

PHASE_0_ARCHITECTURE

- [SUOP ERP v1.0 — Phase 0: Enterprise Foundation Architecture](#)
 - [Table of Contents](#)
 - [1. Purpose and Scope](#)
 - [2. Architectural Principles](#)
 - [3. Folder Structure](#)
 - [4. Layered Architecture](#)
 - [5. Dependency Graph](#)
 - [6. Shared Libraries](#)
 - [7. Base Classes and Interfaces](#)
 - [8. Middleware Stack](#)
 - [9. Workflow Engine](#)
 - [10. Notification Engine](#)
 - [11. Audit Framework](#)
 - [12. Validation Framework](#)
 - [13. Error Framework](#)
 - [14. Configuration Framework](#)
 - [15. Transaction Helper](#)
 - [16. File Upload Service](#)
 - [17. Background Job Framework](#)
 - [18. Event Bus](#)
 - [19. Shared Types and Enums](#)
 - [20. Shared DTOs](#)
 - [21. Shared Domain Events](#)
 - [22. Testing Strategy](#)
 - [23. Coding Standards](#)
 - [24. Definition of Done — Phase 0](#)
 - [25. Open Questions for Approval](#)
 - [Approval Block](#)

SUOP ERP v1.0 — Phase 0:

Enterprise Foundation

Architecture

Field	Value
Document Version	1.0
Status	DRAFT — Awaiting Stakeholder Approval
Author	Lead Software Architect, SUOP v1.0
Date	2026-07-11
Supersedes	All prior implementation decisions
Approval Required From	Project Owner

Table of Contents

- 1. [Purpose and Scope](#)
- 2. [Architectural Principles](#)
- 3. [Folder Structure](#)
- 4. [Layered Architecture](#)
- 5. [Dependency Graph](#)
- 6. [Shared Libraries](#)
- 7. [Base Classes and Interfaces](#)
- 8. [Middleware Stack](#)
- 9. [Workflow Engine](#)
- 10. [Notification Engine](#)
- 11. [Audit Framework](#)
- 12. [Validation Framework](#)
- 13. [Error Framework](#)
- 14. [Configuration Framework](#)

15. [Transaction Helper](#)
 16. [File Upload Service](#)
 17. [Background Job Framework](#)
 18. [Event Bus](#)
 19. [Shared Types and Enums](#)
 20. [Shared DTOs](#)
 21. [Shared Domain Events](#)
 22. [Testing Strategy](#)
 23. [Coding Standards](#)
 24. [Definition of Done — Phase 0](#)
 25. [Open Questions for Approval](#)
-

1. Purpose and Scope

1.1 Purpose

This document defines the **enterprise foundation architecture** for SUOP ERP v1.0. It establishes the shared platform upon which every future ERP module — Organization, Auth, Procurement, Inventory, Manufacturing, Quality, Finance, Retail, Restaurant, and AI — will be built.

The foundation is built **once** and **reused** by every module. No module implementation may bypass or duplicate the foundation. If a module appears to need functionality the foundation does not provide, the correct response is to **extend the foundation**, not to build a parallel mechanism inside the module.

1.2 Scope

Phase 0 covers the following platform capabilities. **No business module code is written in Phase 0.**

- Folder structure and project layout
- Layered architecture and dependency rules
- Shared libraries and base classes
- Middleware stack (auth, RBAC, tenant, audit, validation, error, logging, rate-limit)

- Workflow engine (state machine framework)
- Notification engine (in-app, email, SMS, WhatsApp)
- Audit framework (immutable audit trail)
- Validation framework (schema + business rule)
- Error framework (typed errors, error codes, response envelope)
- Configuration framework (env validation, feature flags)
- Transaction helper (Prisma transaction wrapper)
- File upload service (S3-compatible)
- Background job framework (cron + queued jobs)
- Event bus (in-process pub/sub for domain events)
- Shared types, enums, DTOs, events
- Testing strategy and infrastructure
- Coding standards

1.3 Non-Goals

Phase 0 does **not** include:

- Database schema changes (separate Schema Review Report will be produced)
- Stock Ledger architecture (separate dedicated document will be produced)
- Any business module implementation
- Migration of existing mock code
- UI changes

1.4 Approval Gate

Phase 0 deliverables are **documentation only**. No production code is written until this document is approved in full. After approval, implementation of Phase 0 foundation code begins, and only after Phase 0 code is complete and tested may Phase 1 (Organization module) begin.

2. Architectural Principles

2.1 Layered Architecture with Strict Dependency Direction

```
HTTP Request
  ↓
Routes (thin – only URL → controller mapping)
  ↓
Controllers (thin – request parsing, response shaping)
  ↓
Services (business logic, orchestration, workflows)
  ↓
Repositories (data access, Prisma queries)
  ↓
Database (PostgreSQL via Prisma)
```

Rule: Dependencies flow downward only. A repository may never import a service. A service may never import a controller. A controller may never call Prisma directly.

Enforcement: ESLint `no-restricted-imports` rule enforces layer boundaries at lint time. CI fails on violation.

2.2 Reuse Over Duplication

Any logic that will be needed by more than one module must be promoted to the foundation. This includes:

- CRUD operations → `BaseRepository`
- Validation → `ValidationFramework` + zod schemas
- State transitions → `WorkflowEngine`
- Audit logging → `AuditFramework` (automatic via middleware)
- Notifications → `NotificationEngine` (fired via domain events)
- Pagination, filtering, sorting → `BaseService.list()` + shared DTOs
- Permission checks → `@requirePermission` decorator + RBAC middleware

2.3 State Machine for Every Lifecycle

No status field may be mutated directly. Every entity with a lifecycle (PurchaseOrder, GRN, NCR, CAPA, COA, ProductionOrder, Batch, etc.) must declare a state machine using the `WorkflowEngine`. The engine validates transitions, fires pre/post hooks, emits domain events, and writes audit logs.

2.4 Multi-Tenancy from Day One

Every business table includes `tenantId`. Every repository query automatically scopes by tenant. Cross-tenant access requires explicit elevation and is audit-logged at a higher severity.

2.5 Optimistic Concurrency

Every business table includes `version Int @default(0)`. Updates must use Prisma's `updateMany` with `where: { id, version }` and increment version. Conflicts return `409 Conflict` with the current version.

2.6 Soft Delete with Immutable Audit

No business record is hard-deleted. Soft delete sets `deletedAt` and `deletedBy`. Soft-deleted records are filtered out by default in repositories but remain queryable for audit. True deletion requires a separate “purge” operation that itself writes an audit record describing what was purged and why.

2.7 Event-Driven Side Effects

Side effects (notifications, cache invalidation, downstream updates) are triggered by domain events published on the Event Bus — not by direct service-to-service calls. This decouples modules and makes the system observable.

2.8 Fail Fast, Fail Loud

Missing configuration, missing dependencies, schema drift, and contract violations must cause the application to refuse to start, not silently fall back to demo mode. Every `process.env` read goes through the Configuration Framework, which validates at boot.

2.9 Testability is Non-Negotiable

Every foundation component ships with unit tests. Every public method on a base class or framework has at least one test per branch. Coverage gate in CI: foundation code $\geq 90\%$, business code $\geq 70\%$.

2.10 No Magic, No Surprises

Code must be readable without IDE tooling. Avoid exotic TypeScript patterns, runtime type manipulation, or implicit behavior. Decorators are allowed where they reduce boilerplate (e.g., `@requirePermission`) but must be documented.

3. Folder Structure

3.1 Monorepo Layout

```
suop-erp/
├── apps/
│   ├── web/           # Next.js 16 frontend
│   └── backend/       # Bun + Hono backend
service
├── mobile/           # React Native + Expo
├──
├── packages/
│   └── shared/       # Shared types, enums,
DTOs, events
│   └── prisma/       # Prisma schema,
migrations, client
```

```

|   └─ ui/                                # Shared UI component
library
|   └─ eslint-config/                     # Shared ESLint config
|
└─ docs/                                  # Architecture, ADRs,
runbooks
|   └─ architecture/
|       └─ PHASE_0_ARCHITECTURE.md
|       └─ SCHEMA_REVIEW_REPORT.md
|       └─ STOCK_LEDGER_DESIGN.md
|   └─ adr/                               # Architecture
Decision Records
|   └─ runbooks/
|
└─ docker/
    └─ Dockerfile.web
    └─ Dockerfile.backend
    └─ docker-compose.yml
|
└─ .github/
    └─ workflows/
        └─ ci.yml
        └─ cd-staging.yml
        └─ cd-production.yml
|
└─ package.json                           # Workspace root
└─ turbo.json                             # Turborepo config
└─ README.md

```

Rationale: Monorepo with Turborepo. `apps/` contains deployable applications. `packages/` contains shared code consumed by multiple apps. `packages/shared` is the single source of truth for types and contracts between frontend and backend.

3.2 Backend Folder Structure (apps/backend/)

```
apps/backend/
├─ src/
│   └─ main.ts                    # Bootstrap: load
│                                   config, connect DB, start server
│   └─ app.ts                     # Hono app
│                                   composition, middleware registration
│   └─
│   └─ config/                   # Configuration
framework
│   └─
│   └─ env.ts                    # Env validation (zod)
│   └─ features.ts               # Feature flags
│   └─ index.ts
│   └─
│   └─ core/                     # Foundation framework
│       └─ errors/               # Error framework
│           └─ base-error.ts
│           └─ error-codes.ts
│           └─ error-handler.ts
│           └─ index.ts
│       └─
│       └─ response/             # Response envelope
│           └─ envelope.ts
│           └─ index.ts
│       └─
│       └─ logging/              # Structured logging
│           └─ logger.ts
│           └─ request-context.ts
│           └─ index.ts
│       └─
│       └─ validation/           # Validation framework
│           └─ validate.ts
│           └─ schema-registry.ts
│           └─ index.ts
│       └─
│       └─ audit/                # Audit framework
```

```
| | | └─ audit-logger.ts
| | | └─ audit-middleware.ts
| | | └─ audit-models.ts
| | | └─ index.ts
|
| | | └─ events/ # Event Bus
| | |   └─ event-bus.ts
| | |   └─ event-types.ts
| | |   └─ event-handler.ts
| | |   └─ index.ts
|
| | | └─ notifications/ # Notification engine
| | |   └─ notification-engine.ts
| | |   └─ channels/
| | |     └─ in-app.channel.ts
| | |     └─ email.channel.ts
| | |     └─ sms.channel.ts
| | |     └─ whatsapp.channel.ts
| | |   └─ templates/
| | |   └─ subscription-service.ts
| | |   └─ index.ts
|
| | | └─ workflow/ # Workflow engine
| | |   └─ workflow-engine.ts
| | |   └─ state-machine.ts
| | |   └─ transition-guard.ts
| | |   └─ workflow-registry.ts
| | |   └─ decorators.ts
| | |   └─ index.ts
|
| | | └─ permissions/ # RBAC
| | |   └─ permission-middleware.ts
| | |   └─ permission-checker.ts
| | |   └─ permission-decorator.ts
| | |   └─ permission-registry.ts
| | |   └─ index.ts
|
| | |
```

```

|   |   | └─ tenant/                                # Multi-tenancy
|   |   |   └─ tenant-middleware.ts
|   |   |   └─ tenant-context.ts
|   |   |   └─ index.ts
|   |   |
|   |   | └─ transactions/                          # Transaction helper
|   |   |   └─ transaction.ts
|   |   |   └─ index.ts
|   |   |
|   |   | └─ files/                                # File upload service
|   |   |   └─ file-service.ts
|   |   |   └─ storage-drivers/
|   |   |       └─ s3.driver.ts
|   |   |       └─ local.driver.ts
|   |   |   └─ index.ts
|   |   |
|   |   | └─ jobs/                                # Background job
framework
|   |   |   └─ job-queue.ts
|   |   |   └─ job-scheduler.ts
|   |   |   └─ job-runner.ts
|   |   |   └─ job-registry.ts
|   |   |   └─ index.ts
|   |   |
|   |   | └─ repositories/                        # Base repository
classes
|   |   |   └─ base.repository.ts
|   |   |   └─ soft-delete.repository.ts
|   |   |   └─ tenant.repository.ts
|   |   |   └─ index.ts
|   |   |
|   |   | └─ services/                            # Base service classes
|   |   |   └─ base.service.ts
|   |   |   └─ crud.service.ts
|   |   |   └─ index.ts
|   |   |
|   |   | └─ controllers/                        # Base controller

```

```

| | | └─ base.controller.ts
| | | └─ crud.controller.ts
| | | └─ index.ts
| | |
| | └─ utils/ # Shared utilities
| |   └─ date.ts
| |   └─ money.ts
| |   └─ quantity.ts
| |   └─ crypto.ts
| |   └─ pagination.ts
| |   └─ index.ts
| |
| └─ modules/ # Business modules
(one per domain)
| | └─ organization/
| |   └─ organization.routes.ts
| |   └─ organization.controller.ts
| |   └─ organization.service.ts
| |   └─ organization.repository.ts
| |   └─ organization.workflow.ts
| |   └─ organization.events.ts
| |   └─ organization.schemas.ts
| |   └─ organization.permissions.ts
| |   └─ organization.types.ts
| |   └─ __tests__/
| |     └─ organization.service.test.ts
| |     └─ organization.repository.test.ts
| |     └─ organization.workflow.test.ts
| |     └─ organization.e2e.test.ts
| |
| └─ auth/
| └─ rbac/
| └─ supplier/
| └─ product/
| └─ purchase-order/
| └─ goods-receipt/

```

```

|   |   └─ iqc/
|   |   └─ quality-hold/
|   |   └─ inventory/
|   |   └─ stock-ledger/
|   |   └─ barcode/
|   |   └─ warehouse/
|   |   └─ ... (future modules follow same structure)
|   |
|   └─ routes/                                # Route registration
|   └─ index.ts                              # Mounts all module
routers
|   └─ health.routes.ts
|   |
|   └─ types/                                # Backend-local types
|       └─ express.d.ts
|
└─ tests/
    └─ unit/
    └─ integration/
    └─ e2e/
    └─ fixtures/
    └─ helpers/
    └─ setup.ts
|
└─ package.json
└─ tsconfig.json
└─ vitest.config.ts
└─ README.md

```

3.3 Frontend Folder Structure (apps/web/)

```

apps/web/
└─ src/
    └─ app/                                # Next.js App Router
    |   └─ layout.tsx
    |   └─ page.tsx                        # Composes module

```

```

shell
├── (auth)/
│   ├── login/page.tsx
│   └── logout/page.tsx
└── (dashboard)/
    ├── layout.tsx # Sidebar + header
shell
├── organization/page.tsx
├── suppliers/page.tsx
├── products/page.tsx
└── ... (one route per module)
└── modules/ # One folder per
business module
├── organization/
│   ├── components/
│   │   ├── OrganizationTree.tsx
│   │   ├── CompanyForm.tsx
│   │   ├── PlantForm.tsx
│   │   └── WarehouseForm.tsx
│   ├── hooks/
│   │   ├── use-organizations.ts
│   │   ├── use-create-company.ts
│   │   └── use-update-plant.ts
│   ├── api/
│   │   └── organization-api.ts
│   ├── types.ts
│   └── index.ts
├── auth/
├── rbac/
├── supplier/
├── product/
└── ... (one per backend module)
└── shared/ # Frontend foundation
    └── api/

```

```

|   |   |   |   └─ client.ts           # Axios instance +
interceptors
|   |   |   |   └─ query-client.ts      # React Query client
|   |   |   |   └─ query-keys.ts       # Centralized query
key factory
|   |   |   |   └─ auth/
|   |   |   |   |   └─ auth-context.tsx
|   |   |   |   |   └─ permission-guard.tsx
|   |   |   |   |   └─ require-permission.tsx
|   |   |   |   └─ components/
|   |   |   |   |   └─ ui/              # shadcn/ui base
components
|   |   |   |   |   └─ data-table/      # Reusable data table
|   |   |   |   |   └─ form/            # Form helpers
|   |   |   |   |   └─ feedback/        # Loading, error,
empty states
|   |   |   |   |   └─ layout/
|   |   |   |   |   └─ hooks/
|   |   |   |   |   |   └─ use-debounce.ts
|   |   |   |   |   |   └─ use-pagination.ts
|   |   |   |   |   |   └─ use-permission.ts
|   |   |   |   |   └─ stores/
|   |   |   |   |   |   └─ auth-store.ts
|   |   |   |   |   |   └─ org-context-store.ts
|   |   |   |   |   └─ types/
|   |   |   |   |   |   └─ api-contracts.ts # Mirrors backend DTOs
|   |   |   |   |   └─ utils/
|   |   |   |   └─ lib/                 # Next.js lib
(supabase client, etc.)
|
└─ tests/
  |   └─ unit/
  |   └─ e2e/                          # Playwright
  |   └─ fixtures/
  |
└─ package.json

```

```
├─ tsconfig.json
├─ vitest.config.ts
├─ playwright.config.ts
└─ README.md
```

3.4 Shared Package (packages/shared/)

```
packages/shared/
├─ src/
│   └─ index.ts                # Barrel export
│   └─ types/
│       └─ brans.ts           # Brand types
│           (TenantId, UserId, etc.)
│       └─ common.ts          # Id, Timestamp, Audit
│       └─ api.ts             # ApiSuccess,
│           ApiError, Paginated
│   └─ enums/
│       └─ status.enum.ts
│       └─ permission.enum.ts
│       └─ entity-type.enum.ts
│       └─ audit-action.enum.ts
│       └─ notification-channel.enum.ts
│       └─ index.ts
│   └─ dtos/
│       └─ pagination.dto.ts
│       └─ filter.dto.ts
│       └─ sort.dto.ts
│       └─ audit-log.dto.ts
│       └─ index.ts
│   └─ events/
│       └─ domain-event.base.ts
│       └─ event-names.ts     # Catalog of all event
names
│       └─ index.ts
│   └─ constants/
│       └─ regex.ts           # GST, PAN, email,
```

```

phone patterns
|   |   └─ index.ts
|   └─ utils/
|       └─ date.ts
|       └─ money.ts
|       └─ index.ts
└─ package.json
└─ tsconfig.json
└─ README.md

```

Rationale: `packages/shared` is consumed by both `apps/web` and `apps/backend`. This guarantees the frontend and backend always agree on contract shape. Breaking changes here trigger TypeScript errors on both sides.

3.5 Prisma Package (`packages/prisma/`)

```

packages/prisma/
└─ prisma/
|   └─ schema.prisma                # Single source of
truth
|   └─ migrations/                  # One folder per
migration
|       └─ seed/
|           └─ 00-tenants.seed.ts
|           └─ 01-roles-permissions.seed.ts
|           └─ 02-organization.seed.ts
|           └─ 03-products.seed.ts
|           └─ 04-suppliers.seed.ts
|           └─ index.ts
└─ src/
|   └─ client.ts                    # PrismaClient
singleton
|   └─ extensions/                  # Prisma extensions
(sof delete, tenant)
|   |   └─ soft-delete.ts
|   |   └─ tenant-scope.ts

```

```
| | └─ audit-log.ts
| └─ index.ts
└─ package.json
└─ README.md
```

4. Layered Architecture

4.1 Layer Responsibilities

Layer	Responsibility	Cannot Do
Route	Map URL pattern + HTTP method to controller method	Cannot contain logic
Controller	Parse request, validate input shape, call service, shape response	Cannot call Prisma, cannot contain business rules
Service	Business logic, orchestration, workflow invocation, event publishing	Cannot call Prisma directly (must go through repository)
Repository	Data access via Prisma, multi-tenant scoping, soft delete filtering	Cannot contain business rules, cannot call other services
Database	PostgreSQL	—

4.2 Example Flow — “Create Purchase Order”

```
POST /api/purchase-orders
↓
```

```
PurchaseOrderRoute (routes/purchase-order.routes.ts)
  → just maps to controller.create
  ↓
PurchaseOrderController.create
  → validate body (zod schema)
  → extract tenant/user from request context
  → call purchaseOrderService.create(dto, ctx)
  → shape response envelope
  ↓
PurchaseOrderService.create
  → check permission (RBAC)
  → start transaction (transaction helper)
  → validate business rules (supplier active, products
exist, budget available)
  → invoke workflow: poWorkflow.transition('draft',
'submitted', po)
  → call purchaseOrderRepository.create(data) inside
transaction
  → publish event: PurchaseOrderSubmitted
  → commit transaction
  → return created PO
  ↓
PurchaseOrderRepository.create
  → prisma.purchaseOrder.create(...) with tenantId from
context
  → returns domain entity
  ↓
PostgreSQL
```

4.3 Cross-Cutting Concerns (Middleware Order)

Request →

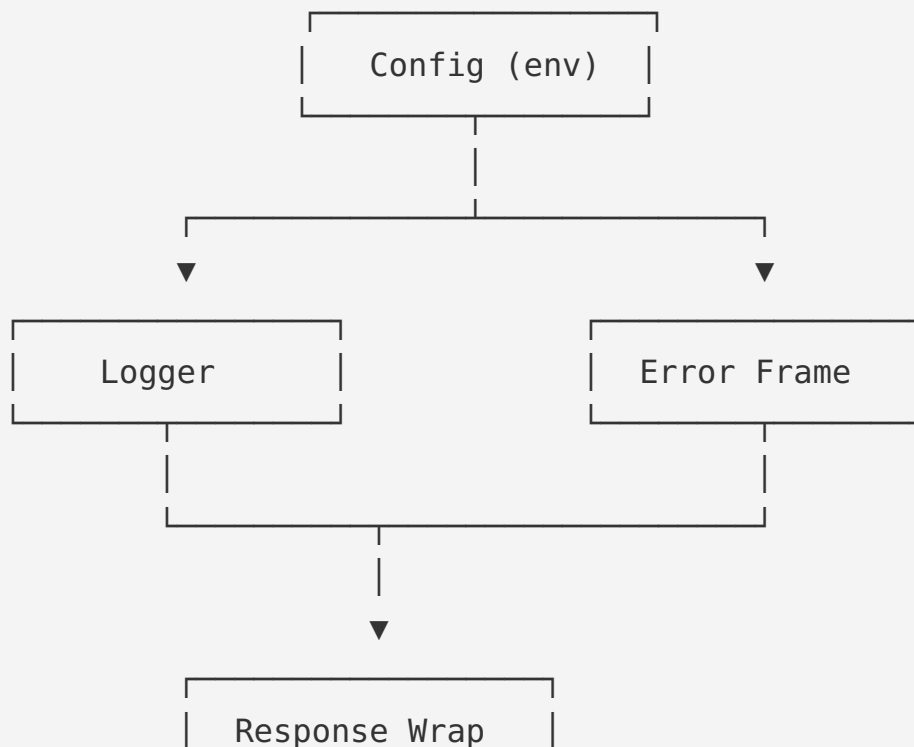
1. RequestIdMiddleware (assigns correlation ID)
2. LoggingMiddleware (logs request start)
3. ErrorBoundaryMiddleware (catches all errors, shapes response)

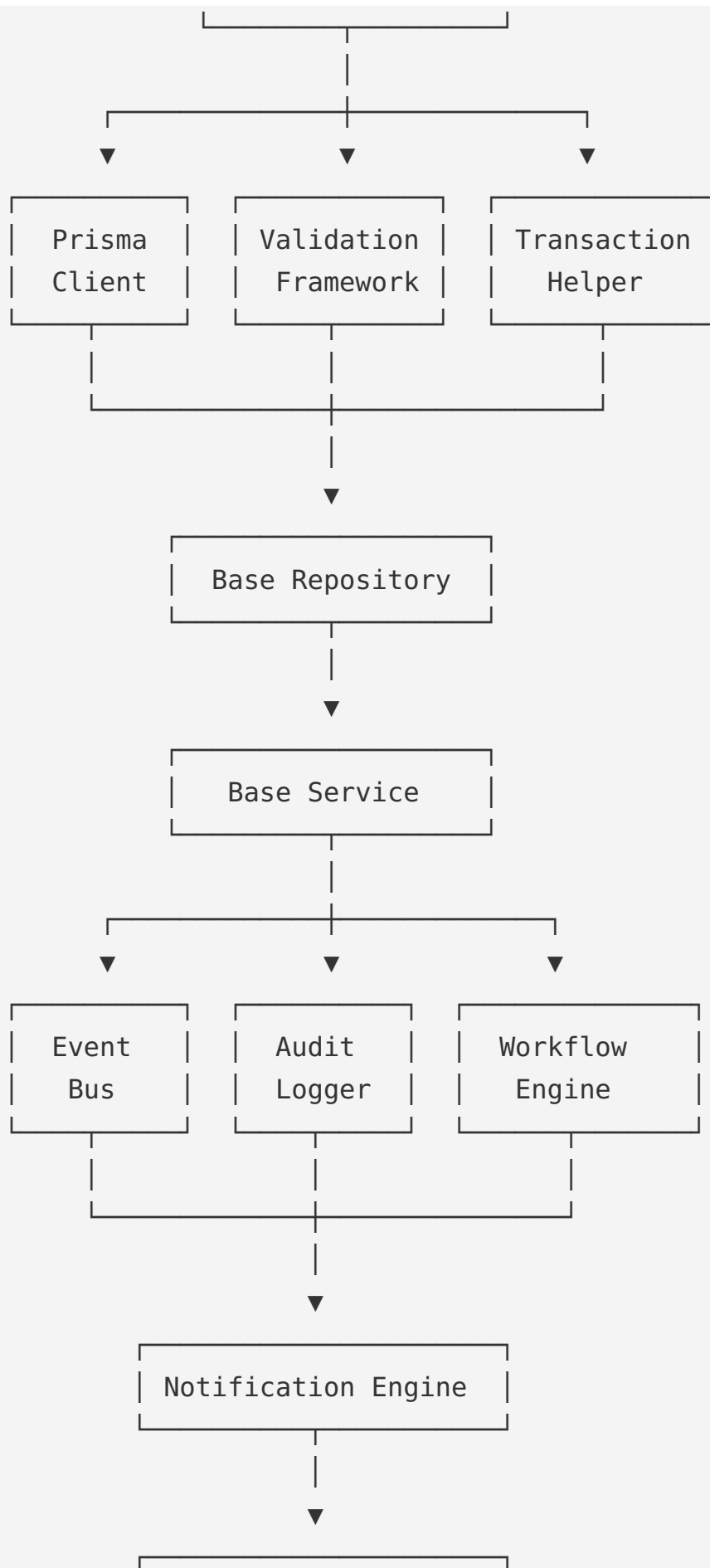
- | | |
|--------------------------|--|
| 4. bodyParserMiddleware | (JSON parsing) |
| 5. AuthMiddleware | (validates JWT, loads user) |
| 6. TenantMiddleware | (loads tenant from JWT or header) |
| 7. RbacMiddleware | (checks route-level permission) |
| 8. AuditMiddleware | (records request metadata for audit log) |
| 9. RateLimitMiddleware | (per-tenant rate limiting) |
| 10. ValidationMiddleware | (per-route zod schema validation) |
| → Handler (Controller) | |

Each middleware runs in order. Any middleware can short-circuit (e.g., Auth fails → 401 returned, no further middleware runs).

5. Dependency Graph

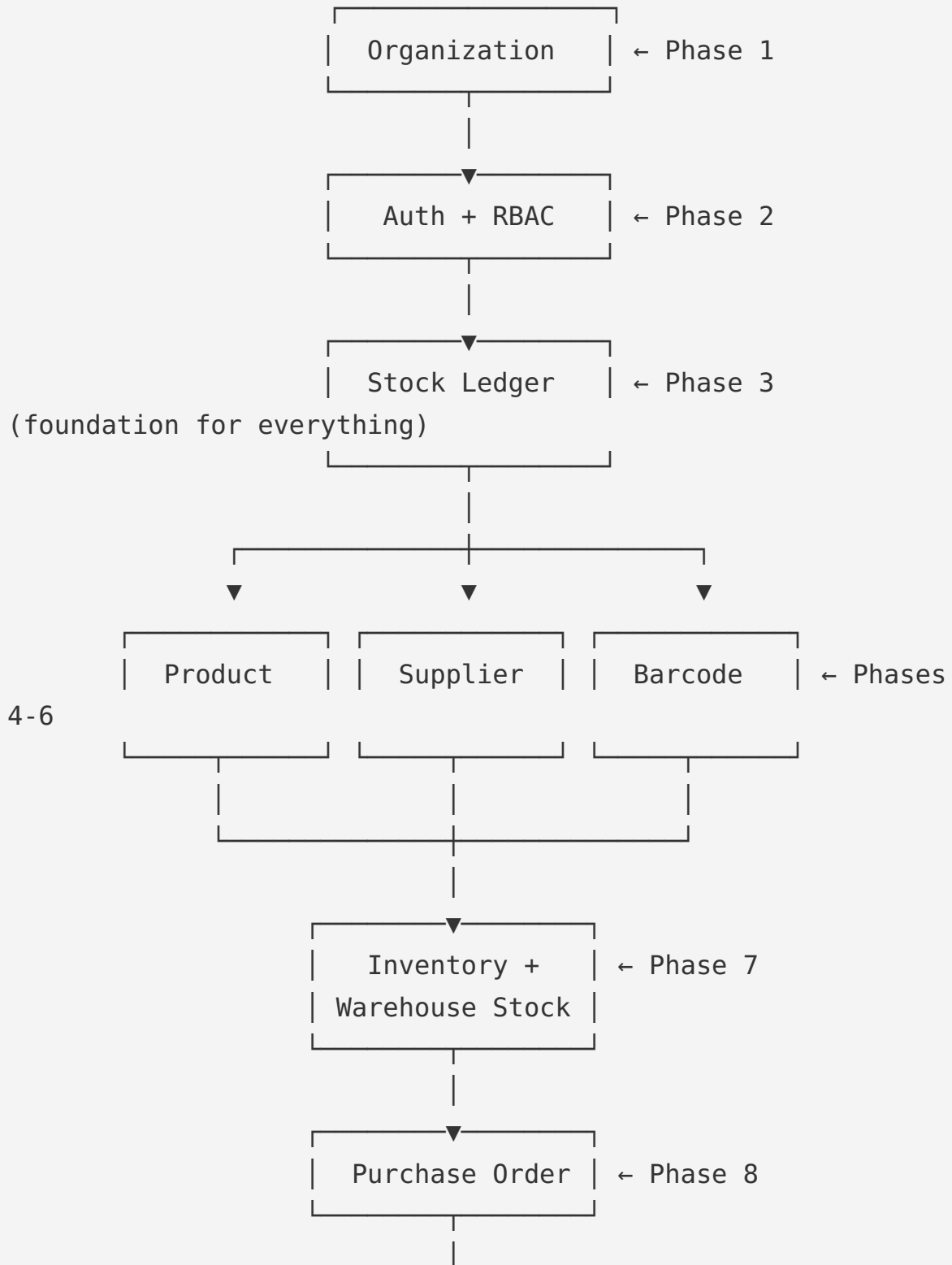
5.1 Foundation Dependency Graph

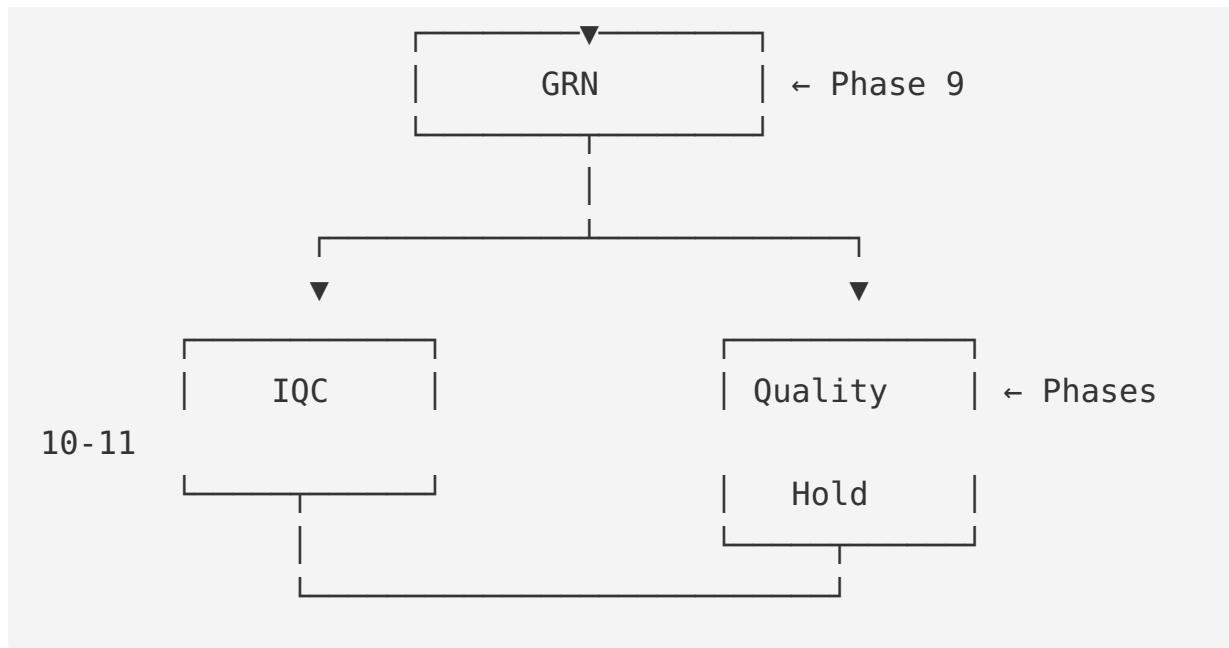




Base Controller

5.2 Module Dependency Graph (for First Journey)





Hard rule: A module may only depend on modules below it in the graph. No upward dependencies. No circular dependencies. Enforced by ESLint `no-restricted-imports` per-module.

6. Shared Libraries

6.1 Library Catalog

Library	Package	Purpose	Consumed By
@suop/shared	packages/ shared	Types, enums, DTOs, events, constants	Backend, Web, Mobile
@suop/prisma	packages/ prisma	PrismaClient, extensions, migrations	Backend only
@suop/ui	packages/ui	Reusable React components (DataTable, Form, etc.)	Web, Mobile (partial)
			All

Library	Package	Purpose	Consumed By
@suop/eslint-config	packages/eslint-config	Shared ESLint rules	

6.2 @suop/shared — Contract Surface

This package exports **only types and constants**. No runtime logic (with rare exceptions like regex patterns and pure functions). This keeps it tree-shakeable and safe to import from both frontend and backend without bundling backend code into the frontend.

Exports: - `Brand<T, B>` — brand types for nominal typing - `EntityId`, `TenantId`, `UserId`, `RoleId`, `ProductId`, etc. — branded strings - `Enum` namespace — all shared enums (see §19) - `DTO` namespace — all shared DTOs (see §20) - `Event` namespace — all domain event names (see §21) - `Regex` — GST, PAN, email, phone patterns - `formatMoney`, `formatDate`, `formatQuantity` — pure formatters

6.3 @suop/prisma — Database Access

Exports: - `prisma` — singleton PrismaClient with extensions registered - `Prisma` — the Prisma namespace (for `Prisma.PromiseReturnType`, etc.) - `withTenant(tenantId)` — helper for cross-tenant queries (rare, audit-logged) - `transaction(fn)` — wraps Prisma `$transaction` with logging + error handling

Prisma extensions registered by default: - **Soft Delete Extension** — overrides `delete` to set `deletedAt`, overrides `findMany`/`findFirst` to filter `deletedAt IS NULL` by default - **Tenant Scope Extension** — automatically injects `tenantId` from `AsyncLocalStorage` context into all `where` clauses - **Audit Log Extension** — writes to `audit_logs` table on every `create`/`update`/`delete`

6.4 @suop/ui — Component Library

Built on top of `shadcn/ui`. Adds enterprise-specific components: - `<DataTable />` — sortable, filterable, paginated table with column definitions - `<EntityForm />` — schema-driven form (zod schema → form

fields) - `<StatusBadge />` — color-coded status pill driven by enum -
`<PermissionGuard>` — hides children if user lacks permission -
`<WorkflowActions />` — renders available actions based on current entity
state - `<AuditTrail />` — renders audit log timeline for an entity -
`<LoadingState />`, `<ErrorState />`, `<EmptyState />` — standard
feedback

7. Base Classes and Interfaces

7.1 BaseRepository<T>

```
abstract class BaseRepository<T extends { id: string;  
    tenantId: string; deletedAt: Date | null; version:  
    number }> {  
    protected constructor(protected readonly prisma:  
        PrismaClient, protected readonly modelName:  
        Prisma.ModelName) {}  
  
    async findById(ctx: TenantContext, id: string): Promise<T  
        | null>  
    async findByIdOrThrow(ctx: TenantContext, id: string):  
        Promise<T> // throws NotFoundError  
    async list(ctx: TenantContext, query: ListQuery):  
        Promise<PaginatedResult<T>>  
    async create(ctx: RequestContext, data: CreatedDto<T>):  
        Promise<T>  
    async update(ctx: RequestContext, id: string, data:  
        UpdatedDto<T>, expectedVersion: number): Promise<T>  
    async softDelete(ctx: RequestContext, id: string,  
        expectedVersion: number): Promise<void>  
    async count(ctx: TenantContext, filter?: FilterDto<T>):  
        Promise<number>  
    async exists(ctx: TenantContext, filter: FilterDto<T>):  
        Promise<boolean>  
}
```

Features: - Multi-tenant scoping automatic (via Prisma extension) - Soft
delete filter automatic - Optimistic locking on `update` and `softDelete`
(throws `ConcurrencyError` on version mismatch) - `RequestContext`

carries `userId`, `tenantId`, `correlationId`, `ip`, `userAgent` for audit -
`ListQuery` supports pagination, sorting, filtering, field selection

7.2 `BaseService<TEntity, TRepository>`

```
abstract class BaseService<TEntity, TRepository> extends
    BaseRepository<TEntity> {
    protected constructor(protected readonly repo:
        TRepository) {}

    async get(ctx: RequestContext, id: string):
        Promise<TEntity>
    async list(ctx: RequestContext, query: ListQuery):
        Promise<PaginatedResult<TEntity>>
    async create(ctx: RequestContext, dto:
        CreatedDto<TEntity>): Promise<TEntity>
    async update(ctx: RequestContext, id: string, dto:
        UpdatedDto<TEntity>, version: number):
        Promise<TEntity>
    async delete(ctx: RequestContext, id: string, version:
        number): Promise<void>

    // Hooks for subclasses to override
    protected async beforeCreate(ctx: RequestContext, dto:
        CreatedDto<TEntity>): Promise<CreatedDto<TEntity>> {
        return dto }
    protected async afterCreate(ctx: RequestContext, entity:
        TEntity): Promise<void> {}
    protected async beforeUpdate(ctx: RequestContext, entity:
        TEntity, dto: UpdatedDto<TEntity>):
        Promise<UpdatedDto<TEntity>> { return dto }
    protected async afterUpdate(ctx: RequestContext, before:
        TEntity, after: TEntity): Promise<void> {}
    protected async beforeDelete(ctx: RequestContext, entity:
        TEntity): Promise<void> {}
    protected async afterDelete(ctx: RequestContext, entity:
        TEntity): Promise<void> {}

    // Validation hook – subclasses implement
    protected abstract validateCreate(ctx: RequestContext,
        dto: CreatedDto<TEntity>): Promise<ValidationResult>
```

```

protected abstract validateUpdate(ctx: RequestContext,
    entity: TEntity, dto: UpdateDto<TEntity>):
    Promise<ValidationResult>
}

```

Features: - `before*` / `after*` hooks for cross-cutting logic without subclassing for every concern - `validateCreate` / `validateUpdate` are abstract — every concrete service must implement - All mutations wrapped in transactions automatically (override `create` / `update` if transaction boundary needs to span multiple operations)

7.3 BaseController<TEntity>

```

abstract class BaseController<TEntity> {
    // Standard CRUD handlers – subclasses register these on routes
    protected async handleGet(ctx: RequestContext):
        Promise<ResponseEnvelope<TEntity>>
    protected async handleList(ctx: RequestContext):
        Promise<ResponseEnvelope<PaginatedResult<TEntity>>>
    protected async handleCreate(ctx: RequestContext):
        Promise<ResponseEnvelope<TEntity>>
    protected async handleUpdate(ctx: RequestContext):
        Promise<ResponseEnvelope<TEntity>>
    protected async handleDelete(ctx: RequestContext):
        Promise<ResponseEnvelope<{ deleted: boolean }>>
}

```

Features: - Controllers are thin — they delegate to services - All responses wrapped in `ResponseEnvelope` (see §13) - All errors thrown by services propagate to `ErrorBoundaryMiddleware` which shapes them - Input validation via zod schema attached to route definition

7.4 BaseWorkflow<TState extends string, TContext>

```

abstract class BaseWorkflow<TState extends string, TContext
    extends { id: string; status: TState; version:
        number }> {
    protected abstract readonly states: TState[]
}

```

```

protected abstract readonly transitions:
    Transition<TState, TContext>[]
protected abstract readonly initialState: TState

async canTransition(ctx: RequestContext, entity:
    TContext, target: TState): Promise<boolean>
async transition(ctx: RequestContext, entity: TContext,
    target: TState, payload?: unknown):
    Promise<TContext>
protected abstract onBeforeTransition(ctx:
    RequestContext, entity: TContext, target: TState,
    payload?: unknown): Promise<void>
protected abstract onAfterTransition(ctx: RequestContext,
    before: TContext, after: TContext, payload?:
    unknown): Promise<void>
}

```

Features: - States and transitions declared declaratively - `canTransition` checks guards (sync + async) - `transition` validates, runs pre-hooks, persists, runs post-hooks, publishes event, writes audit log — all in a transaction - Pre/post hooks let subclasses add behavior without re-implementing the engine

7.5 BaseEventHandler<TEvent extends DomainEvent>

```

abstract class BaseEventHandler<TEvent extends DomainEvent>
{
    abstract readonly eventName: string
    abstract handle(event: TEvent, ctx: EventHandlerContext):
        Promise<void>
    // Optional retry policy
    retries?: number = 3
    backoffMs?: number = 1000
}

```

7.6 BaseJob

```

abstract class BaseJob<TPayload> {
    abstract readonly jobName: string
}

```

```
abstract readonly concurrency: number // max parallel
    instances

abstract readonly maxRetries: number
abstract execute(payload: TPayload, ctx: JobContext):
    Promise<JobResult>
}
```

7.7 BaseMiddleware

```
abstract class BaseMiddleware {
    abstract handle(ctx: RequestContext, next: () =>
        Promise<void>): Promise<void>
}
```

8. Middleware Stack

8.1 RequestIdMiddleware

- Generates UUID v7 for every request
- Stores in `AsyncLocalStorage` for access by logger, audit, etc.
- Echoes back in `X-Request-Id` response header
- If incoming request has `X-Request-Id`, uses that (for distributed tracing)

8.2 LoggingMiddleware

- Logs request start: method, path, userId (if authenticated), tenantId, requestBody (sanitized)
- Logs request end: status, duration, responseSize
- Uses structured JSON logging (Pino)
- Redacts sensitive fields (`password`, `token`, `auth`, `creditCard`)
- Logs at `info` for 2xx, `warn` for 4xx, `error` for 5xx

8.3 ErrorBoundaryMiddleware

- Wraps all downstream middleware in try/catch

- Catches `BaseError` subclasses → shaped response with correct status code
- Catches unknown errors → 500 with generic message, full stack logged
- Never leaks stack traces to client in production
- Publishes `SystemError` event for monitoring

8.4 AuthMiddleware

- Extracts JWT from `Authorization: Bearer <token>` header
- Validates signature, expiry, issuer, audience
- Loads user from DB (cached 60s) — checks `status === 'ACTIVE'`
- Loads tenant from JWT claim
- Sets `RequestContext.user` and `RequestContext.tenant`
- Public routes (`/api/auth/login` , `/api/health`) bypass this middleware
- Missing/invalid token → 401 with `WWW-Authenticate: Bearer` header

8.5 TenantMiddleware

- Reads `tenantId` from `RequestContext` (set by AuthMiddleware)
- If absent (e.g., super-admin), validates user has `system:tenant:cross` permission
- Sets up `AsyncLocalStorage` tenant context — all Prisma queries auto-scope
- Detects tenant mismatch (user from tenant A trying to access tenant B's data) → 403

8.6 RbacMiddleware

- Route declares required permission: `router.post('/purchase-orders', requirePermission('po:create'), ...)`
- Middleware reads user's roles from `RequestContext`
- Checks if any role grants the required permission
- Optional attribute-based check: e.g., user can only approve POs they created (policy engine)
- 403 if denied; audit log entry written at `WARN` severity

8.7 AuditMiddleware

- Records request metadata for every mutating request (POST/PUT/PATCH/DELETE)
- Captures: timestamp, userId, tenantId, action (HTTP method), entity type (from route), entity ID (from path/body), before/after (from service layer)
- Writes to `audit_logs` table in a non-blocking fire-and-forget manner
- For high-severity actions (delete, permission change, blacklist, recall), also publishes `HighSeverityAudit` event for real-time alerting

8.8 RateLimitMiddleware

- Per-tenant + per-IP rate limiting
- Configurable per route (e.g., `/api/auth/login` allows 10/min/IP; `/api/purchase-orders` allows 100/min/tenant)
- Uses Redis sliding window
- 429 response with `Retry-After` header when exceeded

8.9 ValidationMiddleware

- Route attaches zod schema: `router.post('/purchase-orders', validateBody(createPoSchema), ...)`
- Middleware parses body, validates against schema
- On failure → 400 with structured validation errors (field path, message, code)
- Schema is the single source of truth — same schema is used by frontend for client-side validation

9. Workflow Engine

9.1 Purpose

Every entity with a lifecycle must declare a state machine. The engine enforces:

- Only valid transitions are allowed
- Guards (pre-conditions) are checked before transition
- Side effects (hooks) run before and after

transition - Domain events are published on every transition - Audit log entry is written on every transition - All of the above happens in a single database transaction

9.2 State Machine Definition Example (Purchase Order)

```
// modules/purchase-order/purchase-order.workflow.ts
export class PurchaseOrderWorkflow extends
    BaseWorkflow<PoStatus, PurchaseOrderContext> {
    readonly initialState = 'DRAFT'
    readonly states = ['DRAFT', 'SUBMITTED', 'APPROVED',
        'ORDERED', 'PARTIALLY_RECEIVED', 'RECEIVED',
        'CLOSED', 'CANCELLED'] as const

    readonly transitions: Transition<PoStatus,
        PurchaseOrderContext>[] = [
        { from: 'DRAFT', to: 'SUBMITTED', guard: canSubmit,
            onAfter: notifyApprovers },
        { from: 'SUBMITTED', to: 'APPROVED', guard: canApprove,
            onAfter: notifySupplier },
        { from: 'SUBMITTED', to: 'DRAFT', guard: canReject,
            onAfter: notifyRequester },
        { from: 'APPROVED', to: 'ORDERED', onAfter:
            sendPOToSupplier },
        { from: 'ORDERED', to: 'PARTIALLY_RECEIVED', onAfter:
            notifyProcurement },
        { from: 'ORDERED', to: 'RECEIVED', onAfter:
            triggerGrnClosure },
        { from: 'PARTIALLY_RECEIVED', to: 'RECEIVED', onAfter:
            triggerGrnClosure },
        { from: 'RECEIVED', to: 'CLOSED', guard:
            allLinesMatchedInvoiced },
        { from: 'DRAFT', to: 'CANCELLED', onAfter:
            notifyRequester },
        { from: 'SUBMITTED', to: 'CANCELLED', guard: canCancel,
            onAfter: notifyRequester },
        { from: 'APPROVED', to: 'CANCELLED', guard:
            canCancelApproved, onAfter:
            notifySupplierAndRequester },
    ]
}
```

```

protected async onBeforeTransition(ctx, entity, target,
    payload) {
    // Re-validate entity is still in expected state
    // (optimistic concurrency)
    const fresh = await this.repo.findByIdOrThrow(ctx,
        entity.id)
    if (fresh.status !== entity.status) throw new
        ConcurrencyError('PO state changed')
}

protected async onAfterTransition(ctx, before, after,
    payload) {
    // Audit log
    await this.auditLogger.log({
        action: 'WORKFLOW_TRANSITION',
        entityType: 'PurchaseOrder',
        entityId: after.id,
        fromState: before.status,
        toState: after.status,
        actorId: ctx.userId,
        tenantId: ctx.tenantId,
    })
    // Domain event
    await this.eventBus.publish({
        name: 'PurchaseOrderTransitioned',
        payload: { poId: after.id, from: before.status, to:
            after.status, actorId: ctx.userId },
    })
}
}

```

9.3 Transition Guard Contract

A guard is an async predicate that receives `(ctx, entity, payload)` and returns `boolean | { allowed: false, reason: string }`. Guards must be **side-effect free** — they only check whether a transition is allowed.

9.4 Workflow Registry

Every workflow is registered in `WorkflowRegistry` at boot. The registry: - Validates no duplicate state machines for the same entity type - Provides `registry.getWorkflowFor('PurchaseOrder')` lookup - Exposes metadata for UI: “given current state, what actions are available?” — used by `<WorkflowActions />` frontend component

9.5 Workflow Persistence

State is stored on the entity itself (`status` field). The workflow engine does not maintain a separate state store — it operates on the entity’s `status` field, ensuring the DB is always the source of truth. This avoids drift between workflow state and entity state.

9.6 Workflow Inspection API

`GET /api/workflows/:entityType/:entityId` returns: - Current state - Available transitions (with guard results) - Transition history (last 20 transitions from audit log)

This powers the UI’s workflow visualization and action buttons.

10. Notification Engine

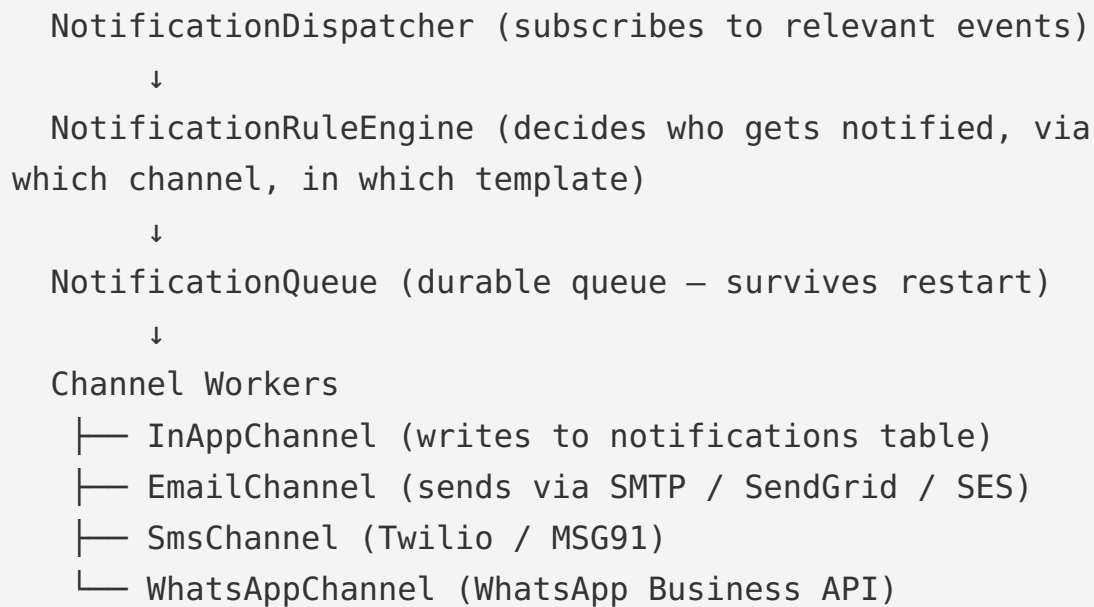
10.1 Purpose

Decouple “things that happen” from “people who need to know about them.” Business logic publishes domain events; the notification engine subscribes to events and routes notifications to the right users via the right channels.

10.2 Architecture

Domain Event Bus





10.3 Notification Rule Configuration

Rules are stored in DB and configurable per tenant:

```
interface NotificationRule {
  id: string
  tenantId: string
  eventName: string // e.g.,
    'PurchaseOrderSubmitted'
  recipientResolver: 'APPROVERS' | 'REQUESTER' |
    'ROLE:PLANT_HEAD' | 'USER' | 'CUSTOM'
  recipientConfig: object // depends on resolver
    - e.g., { roleId: '...' }
  channels: ('IN_APP' | 'EMAIL' | 'SMS' | 'WHATSAPP')[]
  templateId: string
  priority: 'LOW' | 'NORMAL' | 'HIGH' | 'CRITICAL'
  enabled: boolean
}
```

10.4 Templates

Templates use Handlebars syntax with typed contexts:

```
interface EmailTemplate<TContext> {
  id: string
  name: string
  subject: string // Handlebars
  htmlBody: string // Handlebars
  textBody: string // Handlebars
                  (fallback)
  contextSchema: z.ZodSchema<TContext>
}
```

Templates stored in DB, version-controlled. Editing a template creates a new version; old versions retained for audit.

10.5 Subscription Service

Users can subscribe/unsubscribe from notification types (e.g., “I want PO approval emails but not SMS”). Per-user preference overrides rule defaults for non-critical notifications. Critical notifications (food safety recall, security alert) cannot be unsubscribed.

10.6 Durable Delivery

- Notifications written to `notification_outbox` table inside the same transaction as the business event
- Channel workers poll the outbox and deliver
- Retry with exponential backoff (3 attempts, then dead-letter queue)
- Delivery status tracked: `PENDING` → `SENT` → `DELIVERED` / `FAILED`
- Audit log records every delivery attempt

10.7 Initial Channels

- **In-App**: writes to `notifications` table; frontend polls every 30s via React Query; future: WebSocket push
 - **Email**: SMTP via Nodemailer (dev) / SES (prod)
 - **SMS**: Twilio (configurable)
 - **WhatsApp**: WhatsApp Business Cloud API (configurable, future)
-

11. Audit Framework

11.1 Purpose

Every mutation (create, update, delete, workflow transition, approval, permission change) must produce an immutable audit log entry. The audit log is the legal record of system activity and must survive even if the underlying business record is deleted.

11.2 Audit Log Schema (Conceptual)

```
interface AuditLog {
  id: string // UUID v7 (sortable)
  timestamp: Date
  tenantId: string
  // Actor
  actorType: 'USER' | 'SYSTEM' | 'API_KEY' | 'JOB'
  actorId: string | null
  actorName: string | null
  actorRole: string | null
  ipAddress: string | null
  userAgent: string | null
  correlationId: string // request ID
  // Action
  action: AuditAction // CREATE, UPDATE,
  // DELETE, APPROVE, TRANSITION, LOGIN, etc.
  severity: 'INFO' | 'WARN' | 'CRITICAL'
  // Entity
  entityType: string // 'PurchaseOrder',
  // 'User', etc.
  entityId: string | null
  entityCode: string | null // human-readable:
  // 'PO-2026-000001'
  // Change
  before: object | null // JSON snapshot
  after: object | null // JSON snapshot
  diff: object | null // field-level diff
```

```
// Context
reason: string |
      null           // user-supplied reason for
      action
metadata: object     // workflow transition
              info, etc.
}
```

11.3 Immutability

- Audit log table is **append-only**. No UPDATE, no DELETE.
- Enforced at database level via PostgreSQL row-level security policy:
`FOR INSERT ONLY`
- The `audit_logs` table is excluded from soft-delete extension
- Backup strategy: audit logs are exported daily to cold storage (S3 Glacier) and never deleted

11.4 Automatic vs Manual Logging

Automatic (via AuditMiddleware + Prisma extension): - Every CRUD mutation - Every workflow transition - Every login/logout - Every permission check failure

Manual (via `auditLogger.log()` from services): - Business events that don't change data (e.g., "PO sent to supplier via email") - High-severity events (e.g., "Critical NCR auto-quarantined batch X") - Cross-cutting events that the middleware can't detect

11.5 Audit Query API

- `GET /api/audit/entity/:entityType/:entityId` — full history of one entity
- `GET /api/audit/actor/:userId` — all actions by one user
- `GET /api/audit/timeline?`
`from=...&to=...&action=...&severity=...` — filtered timeline
- All queries require `audit:read` permission
- Critical-severity audit logs require `audit:read:critical` permission

11.6 Audit Trail UI

The `<AuditTrail entityId=... entityType=... />` component renders a reverse-chronological timeline with: - Timestamp (relative + absolute) - Actor name + role - Action + state transition arrow - Field-level diff (collapsible) - Reason / notes

12. Validation Framework

12.1 Two Layers of Validation

Layer 1 — Schema Validation (zod): - Validates request shape (body, params, query) - Run by `ValidationMiddleware` before controller executes - Schema is co-located with the route definition - On failure: 400 with structured error
(`{ field: 'poLines.0.quantity', message: 'Must be positive', code: 'INVALID' }`) - Same zod schema is exported to frontend for client-side pre-validation

Layer 2 — Business Rule Validation (Service): - Validates business rules that span multiple entities - Run inside service before mutation - Examples: - “Supplier must be ACTIVE to add to PO” - “Total PO value cannot exceed department budget” - “Cannot close PO with un-received lines” - On failure: throws `BusinessRuleError` with `code`, `message`, `entityId` — caught by `ErrorBoundaryMiddleware` → 422 response

12.2 Schema Registry

Every zod schema is registered in `SchemaRegistry` at boot: - Validates no duplicate schema names - Exposes `GET /api/_internal/schemas/:name` endpoint — frontend fetches schema at build time to generate TypeScript types and form fields - Schemas are versioned (breaking changes require new schema name or version suffix)

12.3 Reusable Schema Fragments

Common patterns are extracted to shared schema fragments: - `schemas/identifier.ts` — UUID v7 validation - `schemas/money.ts` — `Decimal(14, 2)` with non-negative constraint - `schemas/quantity.ts` — `Decimal(14, 3)` with non-negative constraint - `schemas/gst.ts` — Indian GST number format - `schemas/pan.ts` — Indian PAN format - `schemas/pagination.ts` — `page`, `pageSize`, `sort`, `filter` parameters - `schemas/audit.ts` — `reason`, `comment`, `notes`

12.4 Validation Result Shape

```
type ValidationResult =
  | { valid: true }
  | { valid: false; errors: ValidationError[] }

interface ValidationError {
  field: string // dot-path:
  // 'poLines.0.quantity'
  message: string // human-readable
  code: string // 'REQUIRED' |
  // 'INVALID_FORMAT' | 'BUSINESS_RULE' | 'CONCURRENCY'
  metadata?: object // e.g., { min: 0,
  // actual: -5 }
}
```

13. Error Framework

13.1 Error Hierarchy

```
BaseError (abstract)
├─ ValidationError (400)
│   └─ FieldValidationError
├─ AuthenticationError (401)
│   └─ TokenExpiredError
```

```

├─ AuthorizationError (403)
│   └─ PermissionDeniedError
├─ NotFoundError (404)
├─ ConcurrencyError (409)          // optimistic lock
conflict
├─ ConflictError (409)            // duplicate / state
conflict
├─ BusinessRuleError (422)        // business rule
violated
├─ RateLimitError (429)
├─ ExternalServiceError (502)     // upstream service
failure
├─ DatabaseError (500)
└─ InternalError (500)

```

13.2 Error Code Catalog

Every error type has a stable, documented code. Codes are namespaced by domain:

```

VALIDATION.REQUIRED_FIELD
VALIDATION.INVALID_FORMAT
VALIDATION.BUSINESS_RULE
AUTH.TOKEN_EXPIRED
AUTH.INVALID_CREDENTIALS
RBAC.PERMISSION_DENIED
PO.BUDGET_EXCEEDED
PO.SUPPLIER_INACTIVE
GRN.QUANTITY_EXCEEDS_PO
STOCK.INSUFFICIENT_STOCK
STOCK.NEGATIVE_BALANCE
CONCURRENCY.VERSION_MISMATCH
...

```

Codes are catalogued in `packages/shared/src/enums/error-code.enum.ts`. Frontend uses codes to render localized messages.

13.3 Response Envelope

Every API response (success or error) uses the same envelope:

```
// Success
{
  success: true,
  data: T,
  message?: string,           // human-readable
                             success message
  meta?: {                   // pagination, etc.
    page: number,
    pageSize: number,
    total: number,
    correlationId: string,
  },
}

// Error
{
  success: false,
  error: {
    code: string,             //
    'PO.SUPPLIER_INACTIVE'
    message: string,          // 'Supplier SUP-001
    'is not active'
    details?: ValidationErrors[], // for validation
    errors
    entityId?:
      string,                 // for entity-specific
      errors
    retryAfter?: number,      // for rate limit
  },
  meta: {
    correlationId: string,
  },
}
```

13.4 Error Logging

- 4xx errors logged at `WARN` level
- 5xx errors logged at `ERROR` level with full stack trace
- All errors correlated by `correlationId` from request context
- 5xx errors published to `SystemError` event bus → notification engine alerts engineering channel (Slack/Teams)

13.5 Frontend Error Handling

- Axios interceptor unwraps `ResponseEnvelope`
- On error, interceptor throws typed `ApiError` with code + message
- React Query `onError` handler shows toast for transient errors, redirects to login for 401, shows full-page error for 5xx
- Validation errors are mapped to form fields via react-hook-form

14. Configuration Framework

14.1 Environment Variable Validation

All env vars validated at boot via zod. App refuses to start if validation fails.

```
// config/env.ts
const envSchema = z.object({
  NODE_ENV: z.enum(['development', 'staging', 'production']),
  PORT: z.coerce.number().default(3030),

  DATABASE_URL: z.string().url(),
  DATABASE_POOL_SIZE: z.coerce.number().default(10),

  JWT_SECRET: z.string().min(32),
  JWT_ISSUER: z.string().default('suop-erp'),
  JWT_ACCESS_TTL_MIN: z.coerce.number().default(15),
  JWT_REFRESH_TTL_DAYS: z.coerce.number().default(30),
```

```

    REDIS_URL: z.string().url(),

    S3_ENDPOINT: z.string().url(),
    S3_BUCKET: z.string(),
    S3_ACCESS_KEY: z.string(),
    S3_SECRET_KEY: z.string(),

    SMTP_HOST: z.string(),
    SMTP_PORT: z.coerce.number(),
    SMTP_USER: z.string(),
    SMTP_PASS: z.string(),

    TWILIO_ACCOUNT_SID: z.string().optional(),
    TWILIO_AUTH_TOKEN: z.string().optional(),

    SENTRY_DSN: z.string().url().optional(),
  })

  export const env = envSchema.parse(process.env)

```

14.2 Feature Flags

Feature flags stored in DB, cached in memory, refreshable via admin API.
 Use cases: - Roll out new module to specific tenants - Disable broken feature without deploy - A/B test workflow changes

```

// config/features.ts
class FeatureFlags {
  async isEnabled(tenantId: string, flagName: string):
    Promise<boolean>
  async setFlag(tenantId: string, flagName: string,
    enabled: boolean, reason: string): Promise<void>
  async listFlags(tenantId: string): Promise<FeatureFlag[]>
}

```

Flags checked in code via:

```
if (await features.isEnabled(ctx.tenantId,
    'NEW_RECALL_ENGINE')) {
    // use new engine
} else {
    // use legacy
}
```

14.3 Configuration Sources (in priority order)

1. Environment variables (highest priority — for secrets + deployment-specific)
2. Database (feature flags, per-tenant config)
3. Default values (lowest priority)

15. Transaction Helper

15.1 Purpose

Database transactions must be: - Explicit (no auto-commit magic) - Logged (long-running transactions warned) - Retryable (deadlock + serialization failure → retry with backoff) - Nestable-aware (Prisma interactive transactions don't support nesting — detect and throw clear error)

15.2 API

```
// transactions/transaction.ts
async function transaction<T>(
  ctx: RequestContext,
  fn: (tx: PrismaTransaction) => Promise<T>,
  options?: { isolationLevel?: 'READ_COMMITTED' |
    'SERIALIZABLE'; maxRetries?: number; timeoutMs?:
    number }
): Promise<T>
```

15.3 Retry Policy

- Retry on `P2034` (transaction deadlocked) and `P2038` (serialization failure)
- Exponential backoff: 50ms, 100ms, 200ms, 400ms (max 4 attempts)
- On final failure: throws `DatabaseError` with `code: 'TXN.RETRY_EXHAUSTED'`

15.4 Transaction Context

Inside a transaction, the `RequestContext` carries the `PrismaTransaction` client. All repository methods accept `ctx` and use `ctx.db` (which is either the singleton Prisma client or the transaction client) — so repositories don't need to know they're inside a transaction.

15.5 Long Transaction Detection

Transactions exceeding 5 seconds log a `WARN`. Transactions exceeding 30 seconds log an `ERROR` and publish `SlowTransaction` event. Configurable per-route.

15.6 Outbox Pattern

For mutations that also need to publish domain events, the outbox pattern is used:

1. Inside transaction:
 - Mutate business tables
 - Write event to `event_outbox` table (same transaction)
 2. After commit:
 - Background job reads from `event_outbox`, publishes to event bus, marks as published
 3. Guarantees at-least-once delivery of events even if the bus is temporarily unavailable
-

16. File Upload Service

16.1 Purpose

Unified file storage abstraction for evidence attachments, COA PDFs, calibration certs, audit reports, profile photos, etc.

16.2 API

```
interface FileService {
  async upload(ctx: RequestContext, file: UploadInput):
    Promise<UploadedFile>
  async download(ctx: RequestContext, fileId: string):
    Promise<DownloadedFile>
  async delete(ctx: RequestContext, fileId: string):
    Promise<void>
  async getSignedUrl(ctx: RequestContext, fileId: string,
    ttlSeconds: number): Promise<string>
}

interface UploadInput {
  filename: string
  mimeType: string
  size: number
  stream: ReadableStream           // or Buffer
  category:
    FileCategory                   // COA, EVIDENCE,
    CALIBRATION_CERT, etc.
  linkedEntityType?: string
  linkedEntityId?: string
}

interface UploadedFile {
  id: string
  filename: string
  mimeType: string
  size: number
}
```

```
storageKey: string           // S3 key or
                        local path
url: string                  // signed URL
                        (short-lived)
checksum: string            // SHA-256
uploadedAt: Date
uploadedBy: string
}
```

16.3 Storage Drivers

- **S3Driver** (default): S3 / MinIO / Supabase Storage (S3-compatible)
- **LocalDriver** (dev only): writes to local filesystem — never used in production

Driver selected via `FILE_STORAGE_DRIVER` env var. CI fails if production build uses LocalDriver.

16.4 File Metadata Persistence

File metadata stored in `file_uploads` table. The actual file blob is in object storage. The DB record tracks: - `id`, `tenantId`, `filename`, `mimeType`, `size`, `checksum` - `storageKey` (S3 key), `storageDriver` - `category`, `linkedEntityType`, `linkedEntityId` - `uploadedBy`, `uploadedAt`, `deletedAt` (soft delete)

16.5 Security

- All uploads require authentication
- File size limits per category (e.g., evidence photos max 10MB, COA PDFs max 5MB)
- MIME type whitelist per category
- Virus scan via ClamAV (async, blocks download if scan pending)
- Signed URLs short-lived (default 15 min) — never expose direct S3 URLs to clients
- File deletion soft-deletes metadata; actual blob deleted after retention period (default 90 days)

16.6 Image Processing (Future)

For images, automatic thumbnail generation (100px, 500px, original) via Sharp. EXIF metadata stripped. Optional watermarking.

17. Background Job Framework

17.1 Purpose

Long-running or scheduled work must run outside the request/response cycle. Examples: - Escalation checker (every 5 min): find overdue CAPAs, escalate - Notification dispatcher (every 30s): drain notification outbox - Audit log archival (daily): export audit logs to cold storage - Stock reorder check (hourly): find products below reorder level - Mock recall drill (scheduled): run mock recall exercise

17.2 Job Types

Scheduled Jobs (cron): - Defined via cron expression - Single instance per tenant (distributed lock via Redis prevents duplicates) - Example:

```
escalation-checker: '* /5 * * * *
```

Queued Jobs: - Triggered by application code:

```
jobs.enqueue('SendWelcomeEmail', { userId })
```

 - Processed by workers - Retry with exponential backoff - Example: user registers → enqueue welcome email job

17.3 Job Queue Implementation

- BullMQ on Redis (primary)
- In-memory fallback for development
- Job payload stored as JSON; large payloads (>32KB) stored in S3, payload references the S3 key

17.4 Job Lifecycle

```
ENQUEUED → RUNNING → COMPLETED
    ↘ FAILED (retries remaining)
        ↘ DEAD_LETTER_QUEUE (max retries
exhausted)
```

17.5 Job Observability

- `GET /api/_internal/jobs` — list jobs with filters (status, name, tenant)
- `GET /api/_internal/jobs/:id` — job detail with retry history
- `POST /api/_internal/jobs/:id/retry` — manually retry dead-lettered job (requires admin)
- Failed jobs publish `JobFailed` event → notification engine alerts on-call

17.6 Job Definition Contract

Every job class extends `BaseJob<TPayload>` : - `jobName` — unique identifier - `concurrency` — max parallel instances per worker (default 1) - `maxRetries` — default 3 - `execute(payload, ctx)` — actual work

Jobs registered in `JobRegistry` at boot. Registry validates no duplicate names.

18. Event Bus

18.1 Purpose

Decouple modules via in-process pub/sub. When a Purchase Order is submitted, the PO module publishes `PurchaseOrderSubmitted`. The notification engine, the audit framework, and any future subscriber react independently. The PO module knows nothing about who's listening.

18.2 In-Process vs Distributed

- **Phase 0 (now):** In-process event bus using `EventEmitter3` + `AsyncLocalStorage`. Subscribers run in the same process. Sufficient for single-instance deployment.
- **Future (when scaling):** Migrate to Redis Pub/Sub or Kafka for multi-instance. The `EventBus` interface stays the same; only the implementation changes.

18.3 Event Bus API

```
interface EventBus {
  async publish<T extends DomainEvent>(event: T):
    Promise<void>
  subscribe<T extends DomainEvent>(eventName: string,
    handler: EventHandler<T>): void
}

abstract class EventHandler<T extends DomainEvent> {
  abstract readonly eventName: string
  abstract handle(event: T, ctx: EventHandlerContext):
    Promise<void>
  retries?: number = 3
  backoffMs?: number = 1000
}
```

18.4 Event Shape

```
abstract class DomainEvent {
  abstract readonly name: string
  readonly id: string // UUID v7
  readonly timestamp: Date
  readonly tenantId: string
  readonly correlationId: string // ties to request
    that triggered it
  readonly actorId: string | null
```

```
    abstract readonly payload: unknown
}
```

18.5 Delivery Semantics

- **At-least-once delivery** — handlers must be idempotent
- **Outbox pattern** — events written to `event_outbox` table in same transaction as business mutation; published after commit. Guarantees no event lost if process crashes mid-transaction.
- **Async handlers** — handlers run on next tick to avoid blocking the request
- **Failure isolation** — if one handler throws, other handlers still run; failure logged + retried

18.6 Event Replay

POST `/api/_internal/events/replay` (admin only) — re-publishes events from outbox for a given time range. Used for: - Recovering from subscriber outages - Backfilling new subscribers with historical events - Debugging

19. Shared Types and Enums

19.1 Brand Types (Nominal Typing)

```
// packages/shared/src/types/brands.ts
declare const brand: unique symbol
type Brand<T, B extends string> = T & { readonly [brand]: B }

export type EntityId = Brand<string, 'EntityId'>
export type TenantId = Brand<string, 'TenantId'>
export type UserId = Brand<string, 'UserId'>
export type RoleId = Brand<string, 'RoleId'>
export type ProductId = Brand<string, 'ProductId'>
export type SupplierId = Brand<string, 'SupplierId'>
```

```
export type PurchaseOrderId = Brand<string,
    'PurchaseOrderId'>
export type GrnId = Brand<string, 'GrnId'>
export type BatchId = Brand<string, 'BatchId'>
// ... one per entity
```

Rationale: Prevents accidental cross-assignment (e.g., passing `UserId` where `TenantId` is expected). TypeScript enforces at compile time; zero runtime cost.

19.2 Common Types

```
interface Identifiable { id: string }
interface Tenanted { tenantId: string }
interface Auditable { createdAt: Date; createdBy: string |
    null; updatedAt: Date; updatedBy: string | null }
interface SoftDeletable { deletedAt: Date | null;
    deletedBy: string | null }
interface Versioned { version: number }
```

19.3 Enums

All enums live in `packages/shared/src/enums/`. Example:

```
export enum AuditAction {
    CREATE = 'CREATE',
    UPDATE = 'UPDATE',
    DELETE = 'DELETE',
    APPROVE = 'APPROVE',
    REJECT = 'REJECT',
    TRANSITION = 'TRANSITION',
    LOGIN = 'LOGIN',
    LOGOUT = 'LOGOUT',
    EXPORT = 'EXPORT',
    PRINT = 'PRINT',
}
```

```
export enum Permission {  
  // Org  
  ORG_READ = 'org:read',  
  ORG_CREATE = 'org:create',  
  ORG_UPDATE = 'org:update',  
  ORG_DELETE = 'org:delete',  
  // Auth  
  AUTH_MANAGE_USERS = 'auth:manage_users',  
  AUTH_MANAGE_ROLES = 'auth:manage_roles',  
  // Product  
  PRODUCT_READ = 'product:read',  
  PRODUCT_CREATE = 'product:create',  
  // ... one per resource:action  
}
```

```
export enum EntityType {  
  ORGANIZATION = 'Organization',  
  USER = 'User',  
  ROLE = 'Role',  
  PRODUCT = 'Product',  
  SUPPLIER = 'Supplier',  
  PURCHASE_ORDER = 'PurchaseOrder',  
  GRN = 'GoodsReceipt',  
  // ...  
}
```

```
export enum NotificationChannel {  
  IN_APP = 'IN_APP',  
  EMAIL = 'EMAIL',  
  SMS = 'SMS',  
  WHATSAPP = 'WHATSAPP',  
}
```

```
export enum WorkflowAction {  
  SUBMIT = 'SUBMIT',  
  APPROVE = 'APPROVE',  
  REJECT = 'REJECT',  
}
```

```
CANCEL = 'CANCEL',  
RECEIVE = 'RECEIVE',  
CLOSE = 'CLOSE',  
}
```

20. Shared DTOs

20.1 Pagination

```
interface PaginationParams {  
    page: number           // 1-indexed, default 1  
    pageSize: number       // default 25, max 200  
}  
  
interface PaginatedResult<T> {  
    data: T[]  
    meta: {  
        page: number  
        pageSize: number  
        total: number  
        totalPages: number  
        hasNext: boolean  
        hasPrev: boolean  
    }  
}
```

20.2 Filtering & Sorting

```
interface FilterOperator {  
    eq?: unknown  
    ne?: unknown  
    in?: unknown[]  
    notIn?: unknown[]  
    gt?: unknown
```

```

gte?: unknown
lt?: unknown
lte?: unknown
contains?: string
startsWith?: string
endsWith?: string
between?: [unknown, unknown]
isNull?: boolean
isNotNull?: boolean
}

type Filter<T> = {
  [K in keyof T]?: FilterOperator | Filter<T[K]>
} & {
  AND?: Filter<T>[]
  OR?: Filter<T>[]
  NOT?: Filter<T>
}

interface SortParam {
  field: string
  direction: 'asc' | 'desc'
}

interface ListQuery {
  pagination?: PaginationParams
  filter?: Filter<any>
  sort?: SortParam[]
  search?: string // full-text search query
  fields?:
    string[] // field selection (sparse
    fieldsets)
}

```

20.3 Audit Log DTO

```
interface AuditLogDto {
  id: string
  timestamp: string
  tenantId: string
  actorType: 'USER' | 'SYSTEM' | 'API_KEY' | 'JOB'
  actorId: string | null
  actorName: string | null
  actorRole: string | null
  action: AuditAction
  severity: 'INFO' | 'WARN' | 'CRITICAL'
  entityType: string
  entityId: string | null
  entityCode: string | null
  before: object | null
  after: object | null
  diff: object | null
  reason: string | null
  correlationId: string
}
```

20.4 Response Envelope DTOs (see §13.3)

21. Shared Domain Events

21.1 Event Catalog

All events catalogued in `packages/shared/src/events/event-names.ts`.
Examples:

```
export const EventName = {
  // Auth
  UserRegistered: 'UserRegistered',
```

```
UserLoggedIn: 'UserLoggedIn',  
UserLoggedOut: 'UserLoggedOut',  
UserLocked: 'UserLocked',  
RoleAssigned: 'RoleAssigned',  
RoleRevoked: 'RoleRevoked',
```

// Organization

```
CompanyCreated: 'CompanyCreated',  
PlantActivated: 'PlantActivated',  
PlantDeactivated: 'PlantDeactivated',
```

// Procurement

```
SupplierCreated: 'SupplierCreated',  
SupplierBlacklisted: 'SupplierBlacklisted',  
PurchaseOrderSubmitted: 'PurchaseOrderSubmitted',  
PurchaseOrderApproved: 'PurchaseOrderApproved',  
PurchaseOrderRejected: 'PurchaseOrderRejected',  
PurchaseOrderSent: 'PurchaseOrderSent',
```

// Goods Receipt

```
GrnCreated: 'GrnCreated',  
GrnPosted: 'GrnPosted',  
GrnQualityHold: 'GrnQualityHold',  
GrnPutawayComplete: 'GrnPutawayComplete',
```

// Quality

```
IqcInspectionStarted: 'IqcInspectionStarted',  
IqcInspectionPassed: 'IqcInspectionPassed',  
IqcInspectionFailed: 'IqcInspectionFailed',  
NcrCreated: 'NcrCreated',  
CapaCreated: 'CapaCreated',  
CapaVerified: 'CapaVerified',  
CoaGenerated: 'CoaGenerated',  
CoaSigned: 'CoaSigned',
```

// Stock

```
StockPosted: 'StockPosted',
```

```

    StockReversed: 'StockReversed',
    StockLow: 'StockLow',
    StockExpired: 'StockExpired',

    // System
    SystemError: 'SystemError',
    SlowTransaction: 'SlowTransaction',
    JobFailed: 'JobFailed',
  } as const

  export type EventName = typeof EventName[keyof typeof
    EventName]

```

21.2 Event Payload Contracts

Each event has a typed payload defined in `packages/shared/src/events/payloads/`. Example:

```

// payloads/purchase-order.payloads.ts
export interface PurchaseOrderSubmittedPayload {
  poId: string
  poNumber: string
  supplierId: string
  supplierName: string
  totalValue: number
  currency: string
  submittedBy: string
  approverRoleIds: string[]
}

export interface PurchaseOrderApprovedPayload {
  poId: string
  poNumber: string
  approvedBy: string
  approvedAt: string
}

```

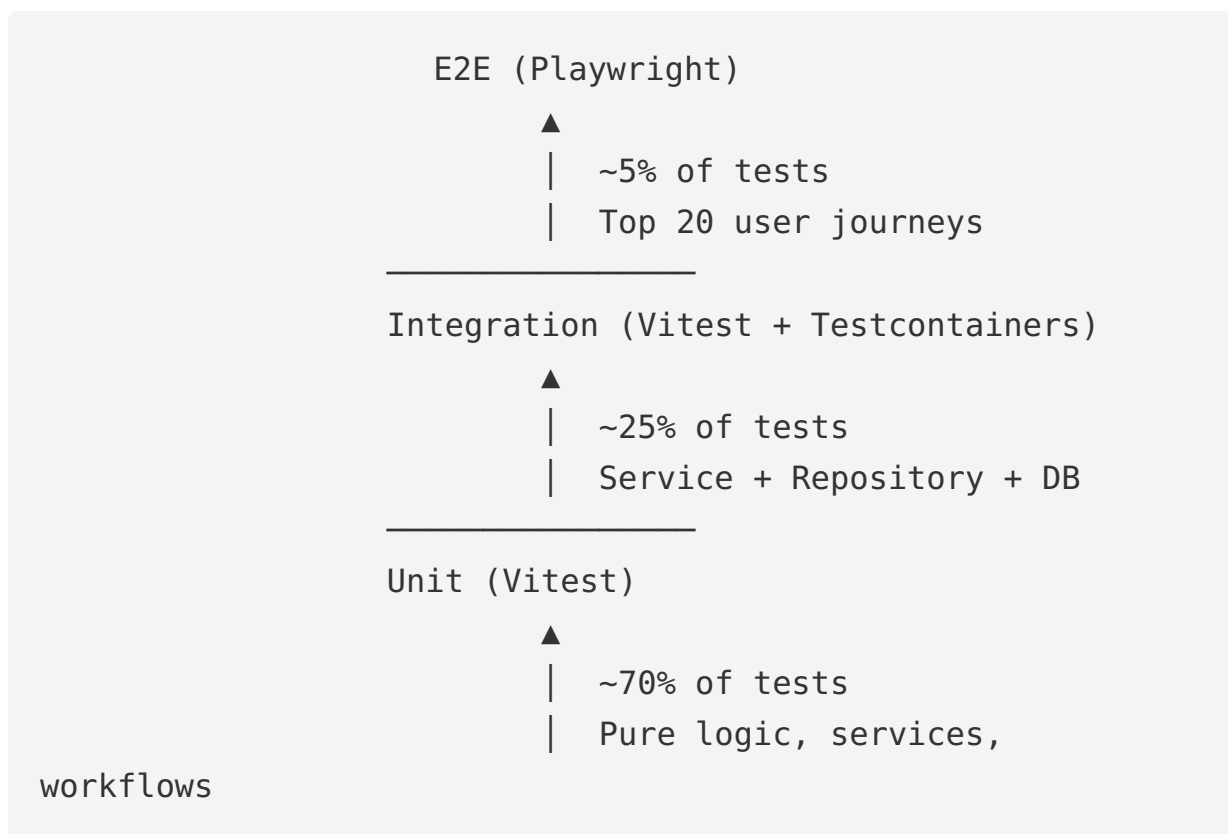
```
nextAction: 'SEND_TO_SUPPLIER' | 'INTERNAL_REVIEW'  
}
```

21.3 Event Versioning

Every event payload includes `version: 1` field. Breaking changes require bumping version + adding new payload type. Subscribers handle both versions during migration window.

22. Testing Strategy

22.1 Test Pyramid



22.2 Unit Tests

- **Target:** 70% coverage for business code, 90% for foundation code
- **Speed:** <10s for full unit suite
- **Tools:** Vitest

- **What to test:**

- Service business rules (validation, calculations, state machines)
- Workflow transitions (valid + invalid)
- Validation schemas (zod)
- Pure utility functions
- Error handling paths

- **What NOT to test:**

- Prisma queries (tested in integration)
- HTTP layer (tested in integration)
- UI rendering (tested in component tests)

22.3 Integration Tests

- **Target:** Every endpoint has at least one happy-path + one error-path integration test
- **Tools:** Vitest + Testcontainers (real PostgreSQL in Docker)
- **Setup:** Per-test transaction rollback (no test pollution)
- **What to test:**
 - Full request → response cycle
 - Repository queries against real DB
 - Multi-tenant isolation (user from tenant A cannot read tenant B's data)
 - Workflow transitions persist correctly
 - Audit log entries written
 - Domain events published

22.4 End-to-End Tests

- **Target:** Top 20 user journeys
- **Tools:** Playwright
- **Examples:**
 - Login → create supplier → create PO → approve PO → receive GRN → IQC pass → stock available
 - Login → create NCR → auto-quarantine batch → create CAPA → verify effectiveness → close
 - Login → generate COA → sign → distribute → archive

22.5 Performance / Load Tests

- **Tools:** k6
- **Targets** (per audit acceptance criteria):
 - Recall identification: <5s for 10M inventory records
 - COA generation: <3s end-to-end
 - Dashboard load: <2s
 - List endpoints: p95 <500ms
- **Run:** Nightly in CI; alert on regression >20%

22.6 Mutation Tests

- **Tools:** Stryker
- **Target:** Foundation code only (workflow engine, audit, permissions)
- **Purpose:** Verify tests actually catch bugs (not just line coverage)

22.7 Contract Tests

- Frontend and backend share `@suop/shared` types
- CI runs `tsc --noEmit` on both — type mismatch fails the build
- OpenAPI spec generated from zod schemas (via `zod-to-openapi`)
- Frontend codegen from OpenAPI spec as a CI check (no drift)

22.8 Test Database

- Dedicated test database, reset between test runs
- Seed data versioned in `tests/fixtures/`
- No mocking of Prisma — use real DB via Testcontainers
- Mock only external services (S3 mock via MinIO, email mock via MailHog)

22.9 Coverage Gate

- **Foundation code (`src/core/`):** ≥90% line coverage, ≥80% branch
- **Business code (`src/modules/`):** ≥70% line coverage, ≥60% branch
- CI fails PRs below threshold
- Coverage trend tracked over time; alert on >5% drop

22.10 Test Naming Convention

```
describe('PurchaseOrderService.create', () => {
  describe('when supplier is active', () => {
    it('creates the purchase order in DRAFT status')
    it('assigns the requester as createdBy')
    it('publishes PurchaseOrderCreated event')
  })
  describe('when supplier is inactive', () => {
    it('throws BusinessRuleError with code
        PO.SUPPLIER_INACTIVE')
  })
  describe('when total value exceeds budget', () => {
    it('throws BusinessRuleError with code
        PO.BUDGET_EXCEEDED')
  })
})
```

23. Coding Standards

23.1 TypeScript Configuration

- `strict: true`
- `noUncheckedIndexedAccess: true`
- `exactOptionalPropertyTypes: true`
- `noFallthroughCasesInSwitch: true`
- `noImplicitReturns: true`
- Target: ES2022
- Module: NodeNext (backend), Bundler (frontend)

23.2 Naming Conventions

Element	Convention	Example
	PascalCase	<code>OrganizationTree.tsx</code>

Element	Convention	Example
Files (components)		
Files (utilities, hooks)	kebab-case	<code>use-organizations.ts</code>
Files (classes, types)	PascalCase	<code>BaseRepository.ts</code>
Variables, functions	camelCase	<code>purchaseOrderService</code>
Constants	SCREAMING_SNAKE_CASE	<code>MAX_PAGE_SIZE</code>
Types, Interfaces, Classes	PascalCase	<code>PurchaseOrderService</code>
Enums	PascalCase (type), SCREAMING_SNAKE_CASE (values)	<code>enum AuditAction { CREATE = 'CREATE' }</code>
Database tables	snake_case	<code>purchase_orders</code>
Database columns	snake_case	<code>created_at</code>
API paths	kebab-case	<code>/api/purchase- orders/:id</code>
Environment variables	SCREAMING_SNAKE_CASE	<code>DATABASE_URL</code>

23.3 File Conventions

- One default export per file for components; named exports for utilities
- Max file length: 400 lines (split if larger)
- Max function length: 50 lines (extract if larger)
- No `any` type without `// eslint-disable-next-line` with justification

- No `// @ts-ignore` — fix the type or use `// @ts-expect-error` with reason

23.4 Import Order (enforced by ESLint)

```
// 1. Node built-ins
import { readFile } from 'node:fs/promises'

// 2. External packages
import { z } from 'zod'
import express from 'express'

// 3. Internal packages (@suop/*)
import { Permission, EntityType } from '@suop/shared'
import { prisma } from '@suop/prisma'

// 4. Internal absolute (@/...)
import { BaseService } from '@core/services'
import { logger } from '@core/logging'

// 5. Relative (./ or ../)
import { PoStatus } from './types'
import { CreatePoSchema } from './schemas'
```

23.5 Layer Boundary Rules (enforced by ESLint)

```
// .eslintrc.js – no-restricted-imports per layer
 '@suop/eslint-config/no-cross-layer-imports': 'error'

// Rules:
// - controllers/ cannot import from repositories/
// - repositories/ cannot import from services/
// - services/ cannot import from controllers/
// - modules/a/ cannot import from modules/b/ (use event bus
    instead)
```

23.6 Commit Message Convention (Conventional Commits)

```
<type>(<scope>): <subject>
```

```
<body>
```

```
<footer>
```

Types: `feat`, `fix`, `refactor`, `test`, `docs`, `chore`, `perf`, `ci` Scope: module name (e.g., `purchase-order`, `auth`, `core-workflow`)

Examples:

```
feat(purchase-order): implement P0 submission workflow
fix(stock-ledger): correct FEFO batch selection edge case
refactor(core-audit): extract audit logger from middleware
test(iqc): add integration tests for fail disposition
docs(architecture): add Phase 0 foundation document
```

23.7 Pull Request Standards

- PR title matches commit convention
- PR description includes: What, Why, How, Testing Done, Screenshots (if UI)
- Max 500 lines changed per PR (split larger work)
- At least one reviewer approval required
- All CI checks green
- No `console.log` in merged code

23.8 Branching Strategy

- `main` — always deployable to production
- `develop` — integration branch
- `feature/<scope>-<description>` — feature branches
- `fix/<scope>-<description>` — bug fix branches

- `hotfix/<scope>-<description>` — production hotfixes (branched from `main`, merged to both `main` and `develop`)

23.9 Code Review Checklist

- ☐ No `any` types
 - ☐ No hardcoded values (use config or constant)
 - ☐ All mutations audit-logged
 - ☐ All endpoints have RBAC
 - ☐ All status changes go through workflow engine
 - ☐ Tests added for new logic
 - ☐ No secrets in code
 - ☐ Error messages user-friendly (no stack traces leaked)
 - ☐ Multi-tenant scoping verified
 - ☐ Soft delete used (no hard delete)
-

24. Definition of Done — Phase 0

Phase 0 is complete when **all** of the following are true:

24.1 Code Implemented (but no business modules)

- ☐ Folder structure created per §3
- ☐ `@suop/shared` package with all types, enums, DTOs, events
- ☐

`@suop/prisma` package with client + extensions (soft delete, tenant scope, audit log)

☐

`@suop/ui` package with base components

☐

Backend foundation:

☐

Config framework (env validation, feature flags)

☐

Error framework (full hierarchy + error codes)

☐

Response envelope

☐

Logging framework (Pino + request context)

☐

Validation framework (zod + registry)

☐

Audit framework (logger + middleware + immutable table)

☐

Event bus (with outbox pattern)

☐

Notification engine (in-app + email channels; SMS/WhatsApp stubbed)

☐

Workflow engine (state machine + registry)

☐

Permission framework (middleware + decorator + registry)

☐

Tenant middleware

☐

Transaction helper

☐

File upload service (S3 driver + local driver)

☐

Background job framework (BullMQ + scheduler)

☐

Base repository, base service, base controller classes

☐

Frontend foundation:

☐

- ☐ Axios client with interceptors
- ☐ React Query client + query key factory
- ☐ Auth context + permission guard
- ☐ Shared UI components (DataTable, EntityForm, StatusBadge, etc.)

24.2 Tests

- ☐ Foundation code unit test coverage $\geq 90\%$
- ☐ Integration tests for every middleware
- ☐ E2E test for login flow (only real flow available)
- ☐ Test database setup via Testcontainers
- ☐ CI pipeline running all tests on every PR

24.3 Documentation

- ☐ This document approved and committed to `docs/architecture/`
- ☐ README.md in every package (`@suop/shared` , `@suop/prisma` , etc.)
- ☐ Architecture Decision Records (ADRs) for major decisions
- ☐ Runbook for common ops tasks (restart, restore backup, etc.)

24.4 DevOps

- ☐ `Dockerfile.web` and `Dockerfile.backend` (multi-stage builds)
- ☐ `docker-compose.yml` for local dev (Postgres + Redis + MinIO + MailHog + backend + web)
- ☐

CI pipeline: lint → typecheck → test → build



CD pipeline: staging on `develop` merge, production on `main` merge (manual approval)



Sentry integration (DSN configured, source maps uploaded)

24.5 Acceptance Tests (Proof Phase 0 Works)

Before any business module work begins, the following must be demonstrable end-to-end:

1. `docker-compose up` starts the entire stack
2. Visit `http://localhost:3000` — landing page loads
3. Visit `http://localhost:3030/api/health` — returns `{ status: 'ok' }`
4. Visit `http://localhost:3030/api/_internal/schemas` — returns list of registered schemas
5. Visit `http://localhost:3030/api/_internal/jobs` — returns empty job list (requires auth → 401 without token)
6. Run `npm test` — all foundation tests pass
7. Run `npm run test:e2e` — login e2e test passes
8. Run `npm run build` — both apps build without errors
9. Audit log table exists in DB with row-level security policy
10. Event outbox table exists
11. Notification outbox table exists
12. Soft-delete Prisma extension verified (delete on a test record sets `deletedAt`, doesn't actually delete)

25. Open Questions for Approval

The following decisions are required before Phase 0 implementation begins. Please approve or modify each:

Q1 — Monorepo Tooling

Recommendation: Turborepo + pnpm workspaces. **Why:** Industry standard for TypeScript monorepos; intelligent caching; simple config. **Alternative:** Nx (more features, more complexity). **Approve / modify?**

Q2 — Backend HTTP Framework

Recommendation: Hono (lightweight, fast, runs on Bun). **Why:** Bun-native, fast, simple middleware composition. **Alternative:** Express (more familiar, more middleware available). **Approve / modify?**

Q3 — Runtime

Recommendation: Bun (already in use). **Why:** Faster startup, built-in test runner, native TypeScript. **Alternative:** Node.js 20 LTS (more mature ecosystem). **Approve / modify?**

Q4 — Database Hosting

Recommendation: Supabase (managed Postgres + free tier sufficient for dev). **Why:** Free managed Postgres with backups; integrates with auth if needed later. **Alternative:** Self-hosted Postgres in Docker (more control, more ops burden). **Decision needed:** Provide Supabase project URL + service key, or instruct self-hosted?

Q5 — Redis Hosting

Recommendation: Upstash Redis (serverless, free tier) for production; in-memory for dev/tests. **Why:** Required for rate limiting, job queue (BullMQ), distributed locks. **Approve / modify?**

Q6 — File Storage

Recommendation: MinIO (S3-compatible, self-hosted in docker-compose) for dev; AWS S3 or Supabase Storage for production. **Why:** S3 API is the de facto standard; MinIO lets dev match prod exactly. **Approve / modify?**

Q7 — Email Provider

Recommendation: MailHog (dev) → AWS SES (prod). **Why:** SES is cheapest at scale; MailHog captures emails locally for inspection.

Alternative: SendGrid (more features, more cost), Postmark. **Approve / modify?**

Q8 — Test Database Strategy

Recommendation: Testcontainers — spin up real Postgres in Docker per test run. **Why:** Tests against real DB; no SQLite/Postgres drift; isolated.

Alternative: Shared test database with transaction rollback (faster, riskier). **Approve / modify?**

Q9 — Frontend State Management

Recommendation: React Query (server state) + Zustand (client UI state).

Why: React Query is the standard for server state; Zustand is minimal for UI-only state (theme, sidebar, etc.). **Alternative:** Redux Toolkit (heavier, more opinionated). **Approve / modify?**

Q10 — Existing Code Disposition

The current `src/app/page.tsx` (37,080 lines, 236 modules, all hardcoded) and `mini-services/suop-backend/index.ts` (14,142 lines, all mock) will not be migrated as-is. Recommendation:

- **Keep** Prisma schema as starting point (will be refined per Schema Review Report)
- **Keep** UI component library (shadcn/ui base)
- **Discard** all mock data arrays (replaced by real API calls)
- **Discard** all backend mock endpoints (replaced by real implementations)
- **Keep** login screen UI (but wire to real auth)
- **Keep** sidebar shell (but rebuild module routing)

Approve / modify?

Q11 — Demo Mode

Recommendation: Remove demo mode entirely from production builds. Demo mode only available when `NODE_ENV=development` and `ENABLE_DEMO_MODE=true`. **Why:** Demo mode is a security risk in production (anyone can click “Explore Demo” and bypass auth). **Approve / modify?**

Q12 — API Versioning

Recommendation: Prefix all routes with `/api/v1/...`. Breaking changes go to `/api/v2/...` with the old version maintained for 6 months. **Why:** Mobile app and any external integrations need stable contracts. **Approve / modify?**

Q13 — Logging Vendor

Recommendation: Pino (structured JSON logs) → stdout → collected by Docker → shipped to Loki/Datadog. **Why:** Pino is fastest; structured logs are greppable; no vendor lock-in. **Alternative:** Winston (slower, more plugins). **Approve / modify?**

Q14 — Monitoring Vendor

Recommendation: Sentry for error tracking; Prometheus + Grafana for metrics; Loki for logs. **Why:** Sentry has best-in-class TS support; Prometheus/Grafana/Loki are open-source and self-hostable. **Alternative:** Datadog (all-in-one, paid). **Approve / modify?**

Q15 — Implementation Cadence

Recommendation: After Phase 0 is approved, I implement Phase 0 code and report back with the standard output format (Database/Backend/Frontend/Workflow/Testing progress %). You review, approve, then I begin Phase 1 (Organization module).

Alternative: Time-boxed sprints (2 weeks each) with demo at end of each. **Approve / modify?**

Approval Block

By approving this document, the Project Owner authorizes:

1. The architectural principles, folder structure, and layering rules defined herein
2. The shared foundation framework specifications (§§6-18)
3. The shared types, enums, DTOs, and events (§§19-21)
4. The testing strategy (§22)
5. The coding standards (§23)
6. The Definition of Done for Phase 0 (§24)
7. The implementation order: Phase 0 (foundation) → Phase 1 (Organization) → ... per the roadmap

Approval does **not** authorize: - Database schema changes (separate Schema Review Report required) - Stock Ledger implementation (separate Stock Ledger Design document required) - Any business module code

Approved by: _____ **Date:** _____

Signature: _____

End of Phase 0 Architecture Document

01_DATA_ARCHITECTURE

- [SUOP ERP v1.0 — Enterprise Data Architecture](#)
 - [1. Purpose](#)
 - [2. Database Platform](#)
 - [3. Database Naming Conventions](#)
 - [4. Primary Key Strategy](#)
 - [5. Tenant Strategy](#)
 - [6. Index Strategy](#)
 - [7. Partition Strategy](#)
 - [8. Archiving Strategy](#)
 - [9. Backup Strategy](#)
 - [10. Restore Strategy](#)
 - [11. Read/Write Patterns](#)
 - [12. Data Lifecycle](#)
 - [13. Historical Data Policy](#)
 - [14. Master Data Policy](#)
 - [15. Transaction Data Policy](#)
 - [16. Reference Data Policy](#)
 - [17. Schema Migration Discipline](#)
 - [18. Database Access Discipline](#)
 - [19. Database Observability](#)
 - [20. Open Questions for Approval](#)
 - [Approval Block](#)

SUOP ERP v1.0 — Enterprise Data Architecture

Field	Value
Document Version	1.0
Status	DRAFT — Awaiting Approval

Field	Value
Date	2026-07-11
Supersedes	All prior database decisions
Approval Required	Project Owner

1. Purpose

This document defines how data is structured, stored, accessed, protected, retired, and recovered in SUOP ERP v1.0. Every Prisma model, every migration, every query, and every archiving decision must conform to this document.

Hard rule: No Prisma schema change is permitted unless it conforms to the conventions defined here. Schema Review Reports must reference this document section by section.

2. Database Platform

2.1 Primary Platform

- **Production:** PostgreSQL 16+ (managed — Supabase, RDS, or Aurora)
- **Staging:** PostgreSQL 16+ (managed, smaller instance)
- **Development:** PostgreSQL 16+ in Docker (via `docker-compose`)
- **Test:** PostgreSQL 16+ via Testcontainers (isolated per test run)

Justification: PostgreSQL is the only open-source database that meets all enterprise ERP requirements: ACID, row-level security, JSONB, full-text search, partitioning, logical replication, point-in-time recovery, extensions (`pgcrypto`, `pg_trgm`, `uuid-oss`).

Forbidden: SQLite (no row-level security, no concurrent write performance), MySQL (weaker transaction isolation, no JSONB), MongoDB (not relational — ERP requires relational integrity).

2.2 Connection Pooling

- **Production:** PgBouncer or Supabase's built-in Supavisor (transaction-mode pooling, pool size 20-50)
- **Application:** Prisma connection pool — `DATABASE_POOL_SIZE=10` per instance, scaled horizontally
- **Max connections:** 100 (Postgres default) → 80 application + 20 reserved for admin/migration
- **Connection lifecycle:** Idle timeout 30s; max lifetime 5min; Prisma logs warnings on slow acquisitions (>1s)

2.3 Character Set & Collation

- **Encoding:** `UTF8` (mandatory — supports all Indian languages + emoji)
 - **Collation:** `en_US.utf8` for default; per-column collation for case-insensitive search
 - **Timezone:** DB stored in `UTC`; application converts to tenant/user timezone for display
-

3. Database Naming Conventions

3.1 Tables

- **Format:** `snake_case`, plural
- **Examples:** `purchase_orders`, `goods_receipts`, `stock_ledger_entries`, `audit_logs`
- **Join tables:** `singular_singular` (e.g., `user_role`, `role_permission`)
- **Prefix conventions:**
 - No prefix for business tables (`purchase_orders`)
 - `_meta_` prefix for framework tables
 - `vw_` prefix for views
 - `fn_` prefix for functions

3.2 Columns

- **Format:** `snake_case`
- **Primary key:** Always `id` (UUID v7)
- **Foreign keys:** `<entity>_id` (e.g., `supplier_id`, `purchase_order_id`)
- **Timestamps:** `created_at`, `updated_at`, `deleted_at` (always)
- **Actor references:** `created_by`, `updated_by`, `deleted_by`
- **Status:** `status` (string, never boolean — must support state machine)
- **Booleans:** `is_<adjective>` (e.g., `is_active`, `is_immutable`)
- **Amounts:** Always `Decimal(14, 2)` for money, `Decimal(14, 3)` for quantities
- **Avoid abbreviations:** `quantity` not `qty`, `purchase_order_id` not `po_id` in schema

3.3 Indexes

- **Format:** `idx_<table>_<columns>` (e.g., `idx_purchase_orders_supplier_id_status`)
- **Unique indexes:** `uq_<table>_<columns>`
- **Composite indexes:** Order columns by selectivity (most selective first)
- **Partial indexes:** `WHERE deleted_at IS NULL` for soft-deleted tables
- **Foreign key indexes:** Every FK gets an index automatically
- **GIN indexes:** For JSONB columns queried with `@>` operator

3.4 Constraints

- **Check constraints:** Named `ck_<table>_<rule>`
- **Unique constraints:** Named `uq_<table>_<columns>`
- **Foreign key constraints:** Named `fk_<from_table>_<to_table>`

3.5 Reserved Columns (every business table)

id	UUID uuid_generate_v7()	PRIMARY KEY DEFAULT
tenant_id	UUID	NOT NULL
version	INTEGER	NOT NULL DEFAULT 0
created_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()
created_by	UUID	NULL
updated_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()
updated_by	UUID	NULL
deleted_at	TIMESTAMPTZ	NULL
deleted_by	UUID	NULL

4. Primary Key Strategy

4.1 Type: UUID v7

- **Format:** 128-bit UUID, time-ordered (RFC 9562)
- **Generation:** PostgreSQL `uuid_generate_v7()` OR application-side generation via `uuidv7` npm package
- **Storage:** `UUID` type (16 bytes — half the size of `TEXT`)

4.2 Why UUID v7 (not v4, not BIGSERIAL)

Property	UUID v4	BIGSERIAL	UUID v7
Globally unique	Yes	No (centralized)	Yes
Time-sortable	No	Yes	Yes
Insert-perf (random PK)	No (B-tree fragmentation)	Yes	Yes (sequential- ish)
Predictable count	No	No (leaks scale)	No

Property	UUID v4	BIGSERIAL	UUID v7
No coord needed	Yes	No	Yes
URL-safe (no int overflow)	Yes	No	Yes

Critical: UUID v7 is time-ordered — inserts are append-mostly at the B-tree leaf, avoiding fragmentation.

4.3 Application-Side vs DB-Side Generation

- **Default:** DB-side (`DEFAULT uuid_generate_v7()`) — single source of truth
- **Exception:** Application-side when the application needs the ID before insert. Use `uuidv7()` npm package.

4.4 No Composite Primary Keys

- Every table has a single `id` UUID PK
- Composite uniqueness enforced via unique constraints
- Rationale: Simplifies ORM, simplifies foreign keys, simplifies audit log references

5. Tenant Strategy

5.1 Multi-Tenancy Model

Chosen model: Shared database, shared schema (with `tenant_id` discriminator)

Model	Pros	Cons	Verdict
Database per tenant	Strong isolation	Expensive, hard analytics	No

Model	Pros	Cons	Verdict
Schema per tenant	Some isolation	Migration pain	No
Shared DB, shared schema, <code>tenant_id</code>	Cheap, easy analytics	Weaker isolation, app-enforced	Yes

5.2 Tenant Isolation Enforcement (3 layers)

Layer 1 — Application (Prisma extension): - AsyncLocalStorage carries `tenantId` from the request - Prisma extension injects `WHERE tenant_id = $currentTenant` into every query - Cross-tenant queries require explicit `withTenant()` + `system:tenant:cross` permission

Layer 2 — Database (PostgreSQL Row-Level Security):

```
ALTER TABLE purchase_orders ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON purchase_orders
  USING (tenant_id =
    current_setting('app.current_tenant_id')::uuid);
```

- Application sets `SET app.current_tenant_id = '...'` on every connection checkout
- Even if app has a bug, DB rejects cross-tenant reads/writes
- Superuser bypasses RLS (for migrations only — application role never superuser)

Layer 3 — Network/API: - JWT contains `tenantId` claim - TenantMiddleware validates token tenant matches request path tenant - Cross-tenant admin operations require permission AND audit log at `CRITICAL` severity

5.3 Tenant Context

```
interface TenantContext {  
    tenantId: TenantId  
    tenantName: string  
    tenantTimeZone: string    // 'Asia/Kolkata'  
    tenantCurrency: string    // 'INR'  
    tenantLocale: string      // 'en-IN'  
    tenantFeatures: string[]  
}
```

5.4 Tenant Provisioning

- New tenant creation is a system-level operation
 - Creates `tenants` row + default admin user + default roles + permissions
 - Audit logged at `CRITICAL` severity; alerts security team
-

6. Index Strategy

6.1 Indexing Principles

1. **Every foreign key gets an index.** Postgres does not auto-create FK indexes.
2. **Every query path gets an index.** EXPLAIN ANALYZE must show index usage. No seq scans on tables >10k rows.
3. **Partial indexes for soft-deleted tables.** `WHERE deleted_at IS NULL`.
4. **Composite indexes for filter+sort.**
5. **No indexes on low-cardinality columns alone.**
6. **Covering indexes for hot queries.** Use `INCLUDE` for index-only scans.

6.2 Standard Index Patterns

```
-- Every business table
CREATE INDEX idx_<table>_tenant_id ON <table> (tenant_id)
    WHERE deleted_at IS NULL;
CREATE INDEX idx_<table>_created_at ON <table> (created_at
    DESC);

-- Lookup by business code
CREATE UNIQUE INDEX uq_<table>_<code>_tenant ON <table>
    (tenant_id, <code>) WHERE deleted_at IS NULL;

-- Foreign keys
CREATE INDEX idx_<table>_<fk_column> ON <table>
    (<fk_column>);

-- Status + tenant (common dashboard filter)
CREATE INDEX idx_<table>_tenant_status ON <table>
    (tenant_id, status) WHERE deleted_at IS NULL;
```

6.3 Index Review Process

- Every Prisma model must declare indexes in the schema
- CI verifies every FK has an index
- Unused indexes (30 days, `idx_scan = 0`) removed
- Index bloat monitored; rebuilt with `REINDEX CONCURRENTLY`

6.4 Full-Text Search

- Postgres `tsvector` + GIN index for FTS
 - `pg_trgm` extension for fuzzy matching (Indian language support)
 - No Elasticsearch in Phase 0 — Postgres FTS sufficient for ERP-scale
-

7. Partition Strategy

7.1 When to Partition

Partition tables expected to exceed **10 million rows** or **50GB**.

7.2 Partition Candidates (Phase 0)

Table	Expected Growth	Partition Strategy
<code>audit_logs</code>	~1M/month	Range partition by <code>created_at</code> (monthly)
<code>stock_ledger_entries</code>	~500K/month	Range partition by <code>posted_at</code> (monthly)
<code>event_outbox</code>	~2M/month	Range partition by <code>created_at</code> (weekly) + retention
<code>notification_outbox</code>	~500K/month	Range partition by <code>created_at</code> (weekly) + retention

7.3 Partition Type

- **Range partitioning** on timestamp columns (most common)
- **List partitioning** only for tenant-isolated tables >100GB (rare)

7.4 Partition Maintenance

- Scheduled job creates next month's partitions 1 week in advance
 - Old partitions dropped via `DROP PARTITION` after retention (instant operation)
 - Tool: `pg_partman` extension
-

8. Archiving Strategy

8.1 Data Categories

Category	Examples	Hot	Warm	Cold
Transactional	purchase_orders, grns, ledger	0-2 years	2-7 years	7+ years
Audit	audit_logs	0-1 year	1-3 years	3-7+ years
Operational	notifications, outbox	0-30 days	—	— (purged)
Master	products, suppliers, users	Always	—	—
Reference	currencies, countries	Always	—	—
Documents	coa_documents, evidence	0-3 years	3-7 years	7+ years (Glacier)

8.2 Archive Process

1. **Identify:** Daily job finds records past hot-window age → `archive_jobs` table
2. **Export:** Export to compressed JSONL in S3 (`s3://suop-archive/<tenant>/<table>/<yyyy-mm>.jsonl.gz`)
3. **Verify:** Re-import to temp table, compare counts + checksums
4. **Purge:** Delete from primary in batches of 1000; audit log entry written
5. **Cold Storage:** 90 days in S3 Standard → Glacier; 7 years → delete (or extend per legal hold)

8.3 Restore from Archive

- Admin API: `POST /api/_internal/archive/restore`

- Reads JSONL from S3, inserts back, resets `archived_at`
 - Audited at `CRITICAL` severity
 - Slow (intentional) — discourages casual use
-

9. Backup Strategy

9.1 Backup Layers

Layer 1 — Managed automated backups (Supabase/RDS): - Daily full backup (2 AM UTC) - PITR via WAL streaming — retention 14 days - Cross-region replica for DR

Layer 2 — Logical backups (pg_dump): - Daily `pg_dump` to S3 (3 AM UTC) - Compressed, encrypted (S3 SSE-KMS) - Retention: 30 days daily, 12 months weekly, 7 years monthly (compliance) - Verified by daily restore-test job

Layer 3 — File storage backups: - S3 versioning on all buckets - S3 cross-region replication to DR region - MinIO (dev) backed up via `mc mirror` to local NAS weekly

9.2 Backup Encryption

- All backups encrypted at rest (S3 SSE-KMS)
- Keys rotated annually
- Decryption keys in secrets manager (different region from backups)

9.3 Backup Integrity

- **Daily:** Restore-test to isolated DB, `pg_checksums`, sample queries
- **Weekly:** Full restore to staging, full E2E suite
- **Monthly:** DR failover drill — promote replica, smoke tests, failback

9.4 Backup Retention Policy

Backup Type	Retention	Storage
PITR WAL	14 days	Supabase managed
Daily logical	30 days	S3 Standard
Weekly logical	12 months	S3 Standard-IA
Monthly logical	7 years	S3 Glacier
File storage	7 years	S3 Glacier

10. Restore Strategy

10.1 Restore Scenarios

Scenario	Method	RTO Target
Accidental row delete (within 14 days)	PITR → extract row → insert	1 hour
Accidental table drop (within 14 days)	PITR → restore table	2 hours
Logical corruption (bad migration)	Restore nightly logical backup	4 hours
Region failure	Promote cross-region replica	30 minutes
Catastrophic (no primary, no replica)	Restore monthly Glacier backup	24 hours

10.2 Restore Procedure (Logical Backup)

1. Provision new Postgres instance (same version)

2. Download backup from S3, verify checksum
3. `pg_restore --clean --if-exists --jobs=4 <backup>`
4. Run `pg_checksums` for integrity
5. Run smoke tests
6. Switch application `DATABASE_URL`
7. Verify application functionality
8. Decommission old instance after 7-day observation

10.3 Restore Procedure (PITR)

1. Use Supabase/RDS console to restore to timestamp T (within 14 days)
2. New instance provisioned with data up to T
3. Application switched to new instance
4. Lost data (T → current) re-entered or replayed from event outbox

10.4 Restore Testing

- Monthly restore drill (full restore to staging)
- Quarterly DR failover drill (promote replica)
- Annual full disaster simulation (region loss, restore from Glacier)

11. Read/Write Patterns

11.1 Read Patterns

Pattern A — Direct lookup by ID: PK index, instant **Pattern B — Filtered list with pagination:** Composite index (`tenant_id`, `created_at` DESC) ; cursor pagination preferred **Pattern C — Aggregation (dashboard):** Partial index; result cached in Redis 5 min **Pattern D — Join (master + transactional):** FK index critical **Pattern E — Full-text search:** `pg_trgm` GIN index for fuzzy match

11.2 Write Patterns

Pattern A — Single insert: ID pre-generated by app for child inserts in same transaction **Pattern B — Update with optimistic locking:**

```
UPDATE purchase_orders
SET status = $1, updated_at = NOW(), updated_by = $2,
    version = version + 1
WHERE id = $3 AND tenant_id = $4 AND version = $5 AND
    deleted_at IS NULL;
-- If 0 rows: either 404 or 409 (version mismatch)
```

Pattern C — Transactional multi-table: All-or-nothing; uses transaction helper + outbox pattern **Pattern D — Bulk insert:** Batch size 500; Prisma `createMany`; transactional **Pattern E — Soft delete:** Sets `deleted_at`, `deleted_by`, increments version

11.3 Forbidden Patterns

- ❌ `SELECT *` in production (explicit column list)
- ❌ N+1 queries (use Prisma `include` or DataLoader)
- ❌ `OFFSET` for deep pagination (cursor-based)
- ❌ `DELETE` without `WHERE` (always soft delete)
- ❌ Cross-tenant queries without explicit `withTenant()` + permission check

12. Data Lifecycle

12.1 Lifecycle Stages

```
Create → Use → Update → Soft Delete → Archive → Cold Storage
→ Purge
```

12.2 Lifecycle Rules per Category

Master Data (products, suppliers, users): - Never archived while referenced by transactional data - Soft delete blocks new references; existing preserved - Hard purge only after 7 years of zero references (compliance audit required)

Transactional Data (POs, GRNs, stock ledger): - Hot: 2 years online - Warm: years 2-7 archived to S3 - Cold: years 7+ to Glacier - Purge: never (legal hold for audits)

Audit Logs: Hot 1y → Warm 1-3y → Cold 3-7+y → Purge: never without legal sign-off

Operational Data (notifications, outbox): Hot 30d → Purge

File Storage: Hot 3y → Warm 3-7y → Cold 7+y → Purge: never without legal sign-off

12.3 Lifecycle Automation

- Daily: `archive-sweeper`, `outbox-purger`
- Weekly: `glacier-transition`
- Monthly: `archive-compressor`
- All jobs audit-logged; failures alert ops

13. Historical Data Policy

13.1 What Counts as “Historical”

- Closed POs, GRNs, NCRs, CAPAs older than 2 years
- Stock ledger entries older than 2 years
- Audit logs older than 1 year

13.2 Access to Historical Data

- Read-only — no edits, no deletes (beyond archive purge)
- Requires `history:read` permission (separate from regular read)
- UI clearly labels with “Historical” badge
- Reports span historical + current via `UNION ALL`

13.3 Historical Snapshots

For trend dashboards: - Pre-computed nightly into `snapshot_<metric>` tables - Immutable once written - Past months never recomputed

13.4 Data Retention Compliance

Regulation	Data Type	Retention
FSSAI	Manufacturing records, COAs	2 years post-expiry
GST	Invoices, GRNs	6 years from filing
Companies Act	Financial records	8 years
Internal	Audit logs	7 years
Internal	Customer complaints	5 years

Configurable per tenant; default to most-strict if not set.

14. Master Data Policy

14.1 What is Master Data

Products, Suppliers, Customers, Warehouses, Plants, Users, Roles, UOMs, Currencies, Tax codes — entities that change rarely and are referenced by many transactional records.

14.2 Master Data Rules

1. **Single source of truth per tenant** — no duplicates; dedup runs nightly
2. **Soft delete only** — never hard delete
3. **Versioning for changes** — material changes create new versions; old preserved with `valid_to`
4. **Approval workflow for creation** — configurable per master type
5. **External ID support** — multiple external IDs (GSTIN, TIN, customer-assigned)
6. **Reference integrity** — UUID FK enforced
7. **Cache strategy** — Redis TTL 5 min; invalidated on update

14.3 Master Data Quality

- Required fields at schema level (NOT NULL)
 - Format validation at service level (GST regex, PAN regex)
 - Duplicate detection — fuzzy match flagged for review
 - Completeness score — % optional fields populated
 - Staleness alerts — 12 months without update
-

15. Transaction Data Policy

15.1 What is Transaction Data

POs, GRNs, stock issues, transfers, stock ledger entries, NCRs, CAPAs, COAs, audit logs, event outbox.

15.2 Transaction Data Rules

1. **Immutable after closure** — corrections via reversal + new transaction (double-entry)
2. **Sequence numbers** — tenant-unique sequence (e.g., `P0-2026-000001`) via Postgres sequence
3. **Status workflow** — every transaction has state machine
4. **Reference integrity** — master record soft-delete blocked if transactions reference it
5. **Posting to ledger** — financial/stock transactions post in same DB transaction
6. **Reversal pattern** — counter-transaction with reference to original; both preserved

15.3 Sequence Number Generation

```
CREATE SEQUENCE po_sequence_<tenant_id> START 1;
```

- Format: `<prefix>-<year>-<6-digit-sequence>` (e.g., `P0-2026-000001`)

- Resets yearly (separate sequence per year per tenant)
 - Generated inside the creating transaction (atomic)
-

16. Reference Data Policy

16.1 What is Reference Data

System-wide constants shared across tenants: countries, currencies, UOMs, tax codes, languages, timezones, product categories.

16.2 Reference Data Rules

1. **Stored in** `reference_*` **tables** — no `tenant_id` column
2. **Tenant overrides** — `tenant_reference_overrides` table
3. **Versioned** — changes create new versions; effective-dated
4. **Seeded via migrations** — updates via migration only (not API)
5. **Cache aggressively** — Redis TTL 1 hour; invalidated via event
6. **Read-only API** — `GET /api/v1/reference/<type>`; no mutations

16.3 Reference Data Updates

- Migration only (code-reviewed)
 - Emergency updates (GST rate change) via admin API with `system:reference:update` permission
 - Each update publishes `ReferenceDataUpdated` event
-

17. Schema Migration Discipline

17.1 Migration Workflow

1. Modify `prisma/schema.prisma`
2. `prisma migrate dev --name <descriptive>` creates migration SQL
3. Review generated SQL — no destructive changes
4. Run locally + test suite
5. PR includes migration file

6. CI runs migration against fresh DB + full test suite
7. Staging auto-deploys on merge
8. Production via CD pipeline with manual approval

17.2 Migration Rules

- **Never destructive in one step** — column drops require 2 migrations (deprecate, then drop after 1 release)
- **Never change column type in one step** — add new, migrate data, drop old
- **Never rename columns** — add new, dual-write, migrate reads, drop old
- **Never rely on application downtime** — all migrations forward-compatible
- **Always include rollback SQL** — paired with every migration; tested in CI

17.3 Migration Review Checklist

☐

No `DROP TABLE` without prior deprecation release

☐

No `DROP COLUMN` without prior deprecation release

☐

No `ALTER TYPE` requiring table rewrite

☐

No `SET NOT NULL` without default for existing rows

☐

New indexes created with `CONCURRENTLY`

☐

FK constraints added with `NOT VALID` then `VALIDATE`

☐

Migration tested against production-size data in staging

18. Database Access Discipline

18.1 Application Role

- Non-superuser role
- CRUD on business tables, SELECT on reference tables
- Cannot TRUNCATE , DROP , or ALTER
- EXECUTE on whitelisted stored functions

18.2 Migration Role

- Separate role for migrations
- CREATE , ALTER , DROP privileges
- CI/CD pipeline only; credentials in secrets manager

18.3 Read Replica Role

- Read-only for reporting/analytics
- Connects to read replica
- Dashboards use this role to offload primary

18.4 Forbidden

- ❌ Application uses superuser role
- ❌ Direct DB access from developer machines to production
- ❌ Long-running queries without statement_timeout
- ❌ Manual schema changes outside migrations

19. Database Observability

19.1 Metrics (Prometheus)

- Connections: active, idle, waiting
- QPS by type (SELECT, INSERT, UPDATE, DELETE)
- Query latency p50, p95, p99
- Transaction duration p50, p95, p99

- Lock waits (count + duration)
- Cache hit ratio
- Index usage
- Table bloat

19.2 Slow Query Log

- `log_min_duration_statement = 500ms`
- Top 10 slowest queries reviewed weekly
- Added to optimization backlog

19.3 Query Plan Analysis

- Weekly `pg_stat_statements` sampling for top 20 expensive queries
- EXPLAIN ANALYZE run automatically; plan stored
- Seq scans on tables >10k rows flagged

19.4 Lock Monitoring

- Alert if any lock wait exceeds 5s
- Alert on deadlocks
- Long-running transactions (>30s) alert ops

20. Open Questions for Approval

#	Decision	Recommendation	Alternatives
Q-D1	Postgres hosting	Supabase Pro	AWS RDS, Aurora
Q-D2	UUID generation	DB-side <code>uuid_generate_v7()</code>	App-side
Q-D3	Partitioning tool	<code>pg_partman</code>	Custom cron
Q-D4		S3 Glacier	

#	Decision	Recommendation	Alternatives
	Archive storage		Azure Blob Cool, GCS Archive
Q-D5	Backup verification	Daily restore-test	Weekly
Q-D6	Reference data updates	Migration-only	Admin API for emergency
Q-D7	Read replica for dashboards	Yes (Supabase)	No (use primary)

Approval Block

Approved by: _____ **Date:** _____

End of Enterprise Data Architecture Document

02_API_STANDARDS

- [SUOP ERP v1.0 — Enterprise API Standards](#)
 - [1. Purpose](#)
 - [2. REST Conventions](#)
 - [3. URL Naming](#)
 - [4. Versioning](#)
 - [5. Pagination](#)
 - [6. Filtering](#)
 - [7. Sorting](#)
 - [8. Bulk APIs](#)
 - [9. Idempotency](#)
 - [10. Error Codes](#)
 - [11. Response Standards](#)
 - [12. Authentication](#)
 - [13. Authorization](#)
 - [14. Rate Limiting](#)
 - [15. API Documentation Strategy](#)
 - [16. Request/Response Logging](#)
 - [17. CORS](#)
 - [18. API Security](#)
 - [19. Open Questions for Approval](#)
 - [Approval Block](#)

SUOP ERP v1.0 — Enterprise API Standards

Field	Value
Document Version	1.0
Status	DRAFT — Awaiting Approval

Field	Value
Date	2026-07-11
Approval Required	Project Owner

1. Purpose

Every API endpoint in SUOP ERP — internal or external, present or future — must conform to this document. The standards ensure that any developer (internal or third-party) can predict how an endpoint behaves without reading its implementation.

Hard rule: No endpoint ships without conforming to these standards. PR review enforces this. CI runs contract tests against every endpoint.

2. REST Conventions

2.1 HTTP Methods

Method	Intent	Idempotent	Safe	Body
GET	Read resource(s)	Yes	Yes	No
POST	Create resource / trigger action	No	No	Yes
PUT	Replace entire resource	Yes	No	Yes
PATCH	Partial update	No	No	Yes (JSON Patch or partial)
DELETE	Soft delete resource	Yes	No	No
HEAD	Check existence (no body)	Yes	Yes	No

Method	Intent	Idempotent	Safe	Body
OPTIONS	CORS preflight	Yes	Yes	No

Rules: - GET must never mutate state. No side effects. No ? action=delete query params. - POST returns 201 Created with Location header pointing to the new resource. - PUT replaces the entire resource — omitted fields reset to defaults (or rejected if required). - PATCH accepts partial updates — only provided fields change. - DELETE is always soft delete (sets deleted_at). Returns 204 No Content . Hard delete is admin-only via separate endpoint.

2.2 HTTP Status Codes

Code	Meaning	When to use
200 OK	Success	GET, PATCH, PUT, action endpoints
201 Created	Resource created	POST that creates new resource
202 Accepted	Request accepted, processing async	Long-running operations (e.g., generate COA PDF)
204 No Content	Success, no body	DELETE, action with no return value
400 Bad Request	Client error — malformed request	Invalid JSON, missing required field
401 Unauthorized	Not authenticated	Missing/invalid JWT
403 Forbidden	Authenticated but not authorized	Missing permission
404 Not Found	Resource does not exist	Invalid ID, soft-deleted record

Code	Meaning	When to use
405 Method Not Allowed	HTTP method not supported	POST to a GET-only endpoint
409 Conflict	State conflict	Optimistic lock version mismatch, duplicate unique
410 Gone	Resource permanently removed	Hard-deleted record
412 Precondition Failed	ETag/If-Match failed	Conditional request failed
422 Unprocessable Entity	Valid JSON, business rule violation	Supplier not active, budget exceeded
429 Too Many Requests	Rate limited	Returned with <code>Retry-After</code> header
500 Internal Server Error	Server error	Unhandled exception
502 Bad Gateway	Upstream service failed	S3 down, email service down
503 Service Unavailable	Temporary outage	Maintenance mode, deploys
504 Gateway Timeout	Upstream timeout	External service timeout

Forbidden: `200 OK` for errors. Always use the correct 4xx/5xx code.

2.3 Safe Methods & Caching

- `GET` and `HEAD` are cacheable
- Responses include `ETag` (hash of response body) and `Last-Modified` headers

- Clients may send `If-None-Match` / `If-Modified-Since` → server returns `304 Not Modified` if unchanged
 - `Cache-Control: private, max-age=60` for short-lived caches (list endpoints)
 - `Cache-Control: no-store` for sensitive data (audit logs, user lists)
-

3. URL Naming

3.1 URL Structure

```
/api/v1/<resource>/<id>/<sub-resource>/<sub-id>?<query-params>
```

- `/api` — fixed prefix for all API routes
- `/v1` — version (see §4)
- `<resource>` — plural kebab-case noun (e.g., `purchase-orders`, `goods-receipts`)
- `<id>` — UUID v7
- `<sub-resource>` — plural kebab-case (e.g., `/purchase-orders/{id}/lines`)
- `<query-params>` — filtering, sorting, pagination (see §§6-8)

3.2 Resource Naming Rules

- **Always plural nouns:** `/purchase-orders` not `/purchase-order`
- **Always kebab-case:** `/goods-receipts` not `/goodsReceipts` or `/goods_receipts`
- **Never verbs in URL:** Use HTTP method to express action. `/purchase-orders/{id}/cancel` is forbidden — use `POST /purchase-orders/{id}/transitions` with `{target: 'CANCELLED'}`.
 - **Exception:** Workflow transitions and business actions that don't fit CRUD use a sub-resource verb: `POST /purchase-orders/{id}/submit`, `POST /purchase-orders/{id}/approve`. These are explicit and documented.

- **Never expose DB internals:** No `/tables/users` or `/queries/run-q-123`. URLs reflect business domain, not schema.
- **ID format:** UUID v7 only. No sequential IDs in URLs (security + scale).

3.3 URL Examples

```
GET    /api/v1/products
GET    /api/v1/products/{id}
POST   /api/v1/products
PATCH /api/v1/products/{id}
DELETE /api/v1/products/{id}

GET    /api/v1/products/{id}/barcodes
POST   /api/v1/products/{id}/barcodes

GET    /api/v1/purchase-orders/{id}/lines
POST   /api/v1/purchase-orders/{id}/lines
PATCH /api/v1/purchase-orders/{id}/lines/{lineId}

POST   /api/v1/purchase-orders/{id}/submit
POST   /api/v1/purchase-orders/{id}/approve
POST   /api/v1/purchase-orders/{id}/reject
POST   /api/v1/purchase-orders/{id}/cancel

GET    /api/v1/products/{id}/audit-trail
GET    /api/v1/products/{id}/workflows/available-transitions
```

3.4 Special Endpoints

- `GET /api/v1/_internal/health` — health check (no auth)
- `GET /api/v1/_internal/ready` — readiness check (no auth, checks DB/Redis)
- `GET /api/v1/_internal/metrics` — Prometheus metrics (auth: API key)

- `GET /api/v1/_internal/schemas/{name}` — fetch zod schema for codegen
- `GET /api/v1/_internal/jobs` — admin job inspection
- `GET /api/v1/reference/{type}` — reference data (no auth, cached)

The `_internal` prefix is reserved for platform endpoints, never business resources.

4. Versioning

4.1 Version in URL

```
/api/v1/purchase-orders  
/api/v2/purchase-orders (future, if breaking change  
needed)
```

- Version is a major version only (v1, v2 — never v1.5)
- Minor changes (additive — new field, new endpoint) do NOT bump version
- Major changes (breaking — removed field, changed type, changed semantics) bump version

4.2 Backward Compatibility

- Old version maintained for **6 months** after new version ships
- Deprecation notice in response header: `Sunset: Sat, 31 Jan 2027 00:00:00 GMT`
- Deprecation notice in API docs and migration guide
- After 6 months, old version returns `410 Gone` with migration instructions

4.3 What Counts as Breaking

Change	Breaking?
Add new endpoint	No

Change	Breaking?
Add new optional field to request body	No
Add new field to response body	No
Remove field from response body	Yes
Change field type	Yes
Change field semantics (e.g., <code>status</code> enum value meaning)	Yes
Change default value of optional field	Yes
Change error response shape	Yes
Change authentication scheme	Yes
Add new required field to request body	Yes

4.4 No Version in Header

URL versioning chosen over header versioning (`Accept: application/vnd.suop.v1+json`): - URL versioning is visible in logs, monitoring, debugging - Easier to route at CDN/load balancer - Easier to test in browser/Postman without setting headers - Trade-off: URL changes are visible to users — acceptable

5. Pagination

5.1 Two Pagination Styles

Style 1 — Offset pagination (for admin tables, simple lists):

```
GET /api/v1/products?page=1&pageSize=25
```

- Use for: small result sets, jump-to-page UI
- Limit: `pageSize` max 200; `page` max 1000 (use cursor for deeper)

- Response includes `meta.total`, `meta.totalPages`, `meta.hasNext`, `meta.hasPrev`

Style 2 — Cursor pagination (for infinite scroll, large datasets, audit logs):

```
GET /api/v1/audit-logs?
cursor=eyJpZCI6IjAxMjM...&pageSize=25
```

- Cursor is base64-encoded JSON of the last row's sort key (e.g., `{id: '...', createdAt: '...'}`)
- Use for: tables >100k rows, infinite scroll UI, append-mostly data (audit logs, stock ledger)
- Response includes `meta.nextCursor` (null if no more pages)
- Cannot jump to arbitrary page — only next/prev

5.2 Default Page Size

- Default: `pageSize=25`
- Max: `pageSize=200` (returns `400` if exceeded)
- Min: `pageSize=1` (returns `400` if `pageSize=0`)

5.3 Pagination Response Shape

```
{
  "success": true,
  "data": [...],
  "meta": {
    "page": 1,
    "pageSize": 25,
    "total": 482,
    "totalPages": 20,
    "hasNext": true,
    "hasPrev": false,
    "correlationId": "01HXY..."
  }
}
```

```
}  
}
```

For cursor pagination:

```
{  
  "success": true,  
  "data": [...],  
  "meta": {  
    "pageSize": 25,  
    "nextCursor": "eyJpZCI6IjAx...",  
    "hasNext": true,  
    "correlationId": "01HXY..."  
  }  
}
```

6. Filtering

6.1 Filter Syntax

Filters are query parameters with operator suffixes:

```
GET /api/v1/purchase-orders?  
status=APPROVED&supplierId=abc-123&totalValue.gte=10000
```

6.2 Operators

Operator	Suffix	Example	SQL equivalent
Equals	(none)	status=APPROVED	status = 'APPROVED'
	.ne	status.ne=DRAFT	

Operator	Suffix	Example	SQL equivalent
Not equals			<code>status != 'DRAFT'</code>
In	<code>.in</code>	<code>status.in=APPROVED,ORDERED</code>	<code>status IN ('APPROVED','ORDERED')</code>
Not in	<code>.nin</code>	<code>status.nin=DRAFT,CANCELLED</code>	<code>status NOT IN (...)</code>
Greater than	<code>.gt</code>	<code>totalValue.gt=1000</code>	<code>total_value > 1000</code>
Greater/ equal	<code>.gte</code>	<code>totalValue.gte=1000</code>	<code>total_value >= 1000</code>
Less than	<code>.lt</code>	<code>totalValue.lt=10000</code>	<code>total_value < 10000</code>
Less/ equal	<code>.lte</code>	<code>totalValue.lte=10000</code>	<code>total_value <= 10000</code>
Between	<code>.between</code>	<code>createdBetween=2026-01-01,2026-06-30</code>	<code>BETWEEN</code>
Contains	<code>.contains</code>	<code>name.contains=kaju</code>	<code>ILIKE '%kaju%'</code>
Starts with	<code>.startsWith</code>	<code>sku.startsWith=KK-</code>	<code>ILIKE 'KK'</code>
Ends with	<code>.endsWith</code>	<code>sku.endsWith=-250</code>	<code>ILIKE '%-250'</code>
Is null	<code>.isNull</code>	<code>deletedAt.isNull=true</code>	<code>deleted_at IS NULL</code>
Is not null	<code>.isNotNull</code>	<code>approvedAt.isNotNull=true</code>	<code>approved_at IS NOT NULL</code>

6.3 Multiple Filters (AND)

Multiple filter params are AND-ed:

```
GET /api/v1/products?  
status=ACTIVE&category=SWEETS&stock.lt=100  
→ status = 'ACTIVE' AND category = 'SWEETS' AND stock < 100
```

6.4 OR Filters

For OR conditions, use the `filter` parameter with a JSON-encoded filter object (advanced):

```
GET /api/v1/products?filter={"OR":[{"status":"ACTIVE"},  
{"status":"DRAFT"}]}
```

This is URL-encoded in practice. Most use cases use AND via simple query params.

6.5 Date Filters

- Dates in ISO 8601 format: `2026-07-11` or `2026-07-11T14:30:00Z`
- Date ranges: `createdAt.between=2026-01-01,2026-06-30`
- All dates stored in UTC; queries interpret naive dates as UTC

6.6 Filter Validation

- Unknown filter fields return `400` with `{field: 'status', message: 'Unknown field'}`
 - Invalid filter values return `400` (e.g., `createdAt.gt=not-a-date`)
 - Filter operators on incompatible types return `400` (e.g., `status.gt=ACTIVE`)
-

7. Sorting

7.1 Sort Syntax

```
GET /api/v1/purchase-orders?sort=-createdAt,supplierName
```

- Prefix `-` for descending (e.g., `-createdAt` = newest first)
- Multiple sort fields comma-separated (priority order)
- Default sort: `-createdAt` (newest first) if no `sort` param

7.2 Sortable Fields

- Each endpoint declares sortable fields in its OpenAPI spec
- Sorting by non-sortable field returns `400`
- Sorting by sensitive field (e.g., `password_hash`) is forbidden — field not in whitelist

7.3 Sort Stability

- Secondary sort by `id` always applied to ensure stable pagination
- E.g., `sort=-createdAt` becomes `ORDER BY created_at DESC, id ASC`

8. Bulk APIs

8.1 Bulk Create

```
POST /api/v1/products/bulk
Content-Type: application/json

{
  "items": [
    { "name": "Kaju Katli 250g", "sku": "KK-250", ... },
    { "name": "Kaju Katli 500g", "sku": "KK-500", ... }
```

```
]
}
```

- Max 100 items per bulk request
- Returns `207 Multi-Status` with per-item result:

```
{
  "success": true,
  "data": {
    "succeeded": [{ "index": 0, "id": "...", "sku":
      "KK-250" }],
    "failed": [{ "index": 1, "errors": [{ "field": "sku",
      "message": "Already exists" }] }],
    "summary": { "total": 2, "succeeded": 1, "failed": 1 }
  }
}
```

- All-or-nothing mode via `?atomic=true` — fails entire batch if any item invalid

8.2 Bulk Update

```
PATCH /api/v1/products/bulk
{
  "updates": [
    { "id": "...", "version": 1, "data": { "status":
      "DISCONTINUED" } },
    { "id": "...", "version": 3, "data": { "status":
      "DISCONTINUED" } }
  ]
}
```

- Each update must include `version` for optimistic locking
- Returns `207 Multi-Status`

8.3 Bulk Delete

```
DELETE /api/v1/products/bulk
{
  "ids": ["...", "...", "..."]
}
```

- Max 100 IDs per request
- Returns 207 Multi-Status

8.4 Bulk Import (CSV/Excel)

```
POST /api/v1/products/import
Content-Type: multipart/form-data

file: products.csv
```

- Async — returns 202 Accepted with job ID
- Client polls GET /api/v1/_internal/jobs/{jobId} for status
- Job result includes row-level errors
- Max 10,000 rows per import; larger batches via S3 upload + signed URL

8.5 Bulk Export

```
GET /api/v1/products/export?format=csv&filter=...
```

- Async — returns 202 Accepted with job ID
 - Job produces file in S3; client receives signed URL when ready
 - Max 1 million rows per export
-

9. Idempotency

9.1 Idempotency Key

For all `POST` and `PATCH` requests that create or mutate state, clients SHOULD send an idempotency key:

```
POST /api/v1/purchase-orders
Idempotency-Key: 7d8b2c1e-9f3a-4b6c-8e2d-1a5f9b3c7e8d
Content-Type: application/json

{ ... }
```

9.2 Server Behavior

- Server stores (`idempotencyKey`, `tenantId`, `requestHash`, `response`, `expiresAt`) in `idempotency_keys` table
- On retry with same key + same request hash: returns stored response (status + body)
- On retry with same key + different request hash: returns `409 Conflict` (idempotency key reuse)
- Keys expire after 24 hours (configurable)
- Keys are scoped to tenant — same key in different tenants are independent

9.3 Required for

- All `POST` to collection endpoints (create)
- All `POST` to action endpoints (submit, approve, etc.)
- All bulk operations
- All payment/financial mutations

9.4 Optional for

- `GET` (always idempotent by nature)
- `DELETE` (idempotent — soft-deleting already-deleted is no-op)

- `PATCH` (idempotent if client includes `version` for optimistic locking)

10. Error Codes

10.1 Error Code Format

Error codes are namespaced strings:

```
<DOMAIN>.<ENTITY>.<RULE>
```

Examples:

```
VALIDATION.REQUIRED_FIELD
VALIDATION.INVALID_FORMAT
VALIDATION.OUT_OF_RANGE
AUTH.TOKEN_EXPIRED
AUTH.INVALID_CREDENTIALS
AUTH.ACCOUNT_LOCKED
RBAC.PERMISSION_DENIED
PO.SUPPLIER_INACTIVE
PO.BUDGET_EXCEEDED
PO.DUPLICATE_LINE_ITEM
GRN.QUANTITY_EXCEEDS_PO_LINE
STOCK.INSUFFICIENT_STOCK
STOCK.NEGATIVE_BALANCE_FORBIDDEN
CONCURRENCY.VERSION_MISMATCH
CONCURRENCY.DEADLOCK_RETRY
RATE_LIMIT.TOO_MANY_REQUESTS
EXTERNAL.S3_UNAVAILABLE
EXTERNAL.EMAIL_SEND_FAILED
```

10.2 Error Code Registry

All error codes catalogued in `packages/shared/src/enums/error-code.enum.ts`. New codes require PR review. Frontend uses codes to render localized messages.

10.3 Error Response Shape

```
{
  "success": false,
  "error": {
    "code": "PO.SUPPLIER_INACTIVE",
    "message":
      "Supplier SUP-001 is not active and cannot be added
      to a purchase order",
    "details": [
      {
        "field": "supplierId",
        "code": "PO.SUPPLIER_INACTIVE",
        "message": "Supplier status is BLACKLISTED",
        "metadata": { "supplierId": "...", "currentStatus":
          "BLACKLISTED" }
      }
    ],
    "entityId": "po-abc-123",
    "retryAfter": null
  },
  "meta": {
    "correlationId": "01HXY..."
  }
}
```

10.4 Validation Error Shape

For `400 Bad Request` from zod validation:

```
{
  "success": false,
  "error": {
    "code": "VALIDATION.REQUIRED_FIELD",
    "message": "Request body validation failed",
    "details": [
      { "field": "poLines[0].quantity", "code": "REQUIRED",
        "message": "Quantity is required" },
      { "field": "poLines[1].unitPrice", "code":
        "INVALID_FORMAT", "message": "Must be a positive
        number", "metadata": { "actual": -5 } }
    ]
  },
  "meta": { "correlationId": "01HXY..." }
}
```

10.5 Field Paths

Field paths use dot + bracket notation: - `name` - `address.city` -
`poLines[0].quantity` - `poLines[2].taxes.cgst.rate`

11. Response Standards

11.1 Success Envelope

```
{
  "success": true,
  "data": { ... },
  "message": "Purchase order created successfully", //
    optional
  "meta": {
    "correlationId": "01HXY...",
    "page": 1, // only for lists
    "pageSize": 25,
    "total": 482,
    "totalPages": 20,
  }
}
```

```
"hasNext": true
}
}
```

11.2 Error Envelope

See §10.3.

11.3 Field Naming in Responses

- **camelCase** in JSON responses (JavaScript convention)
- Database is snake_case; transformation happens in controller layer
- Example: DB column `created_at` → JSON field `createdAt`
- Prisma Client handles this transformation via the `@map` annotation

11.4 Date Format

- All dates in ISO 8601 with timezone: `2026-07-11T14:30:00Z`
- All dates in UTC (Z suffix) — client converts to local timezone for display
- Date-only fields: `2026-07-11` (no time component)
- Duration fields: ISO 8601 duration (`PT5M30S` for 5 min 30 sec)

11.5 Money Fields

- Always returned as objects with amount + currency:

```
{
  "totalValue": {
    "amount": "12500.00",
    "currency": "INR"
  }
}
```

- Amount is **string** (not number) to avoid floating-point precision loss
- Currency is ISO 4217 3-letter code

11.6 Quantity Fields

- Decimal quantities as **string**: "125.500" (preserves precision)
- Integer counts as **number**: 125

11.7 Empty Responses

- 204 No Content — no body (DELETE, action with no return)
- 200 OK with data: null — explicit null when resource exists but has no representation (rare)
- 200 OK with data: [] — empty list (different from null — list exists, has 0 items)

11.8 Sparse Fieldsets

Clients can request specific fields to reduce payload:

```
GET /api/v1/products?fields=id,name,sku,stock
```

- Returns only requested fields
- id always included
- Unknown fields return 400
- Default (no fields param) returns all fields per entity's response schema

12. Authentication

12.1 Authentication Scheme

JWT Bearer tokens:

```
Authorization: Bearer  
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

12.2 JWT Structure

- **Algorithm:** HS256 (HMAC-SHA256) — symmetric, simple
- **Future:** RS256 (asymmetric) when external clients need to verify tokens without sharing secret
- **Header:**

```
{ "alg": "HS256", "typ": "JWT", "kid": "v1" }
```

- **Payload:**

```
{  
  "sub": "user-uuid-v7",  
  "tenantId": "tenant-uuid-v7",  
  "email": "rajesh@sudhamrit.com",  
  "roles": ["quality_manager"],  
  "permissions": ["po:read", "po:create"],  
  "iat": 1720000000,  
  "exp": 1720000900, // 15 min from issue  
  "iss": "suop-erp",  
  "aud": "suop-frontend",  
  "jti": "token-uuid-v7"  
}
```

12.3 Token Lifecycle

Token	TTL	Storage
Access token	15 minutes	In-memory (frontend); never localStorage
Refresh token	30 days	httpOnly secure cookie (frontend); hashed in DB
Device session	90 days	

Token	TTL	Storage
		DB (device_sessions table)
API key	1 year (rotated)	DB (api_keys table, hashed)

12.4 Refresh Flow

1. Client makes request with access token
2. If access token expired (401), client calls `POST /api/v1/auth/refresh` with refresh token cookie
3. Server validates refresh token against DB (revocation list), issues new access token + new refresh token (rotation)
4. Old refresh token immediately invalidated (replay detection)
5. New refresh token in httpOnly cookie

12.5 Token Revocation

- **Access tokens:** Cannot be revoked (short-lived — wait 15 min). Mitigated by short TTL.
- **Refresh tokens:** Revocable via `POST /api/v1/auth/logout` (deletes from DB) or admin force-logout
- **JTI blocklist:** Compromised access tokens' JTIs added to Redis blocklist (TTL = remaining token TTL); checked on every request

12.6 Multi-Factor Authentication (MFA)

- **TOTP-based** (Google Authenticator, Authy)
- **Required for:** Admin roles, finance roles, anyone with `system:*` permissions
- **Optional for:** Other users (configurable per tenant)
- **Flow:**
 1. User logs in with email + password → server returns `MFA_REQUIRED` response with `mfaSessionToken`

2. User enters 6-digit TOTP code → `POST /api/v1/auth/mfa/verify` with `mfaSessionToken` + code
3. Server verifies TOTP, issues access + refresh tokens

12.7 API Keys (for integrations)

- Long-lived (1 year), rotated
- Format: `suop_<random_64_chars>`
- Stored hashed (argon2id) in `api_keys` table
- Scoped to specific permissions (not full user permissions)
- Sent via `X-API-Key` header (not Authorization — distinguish from user JWTs)

13. Authorization

13.1 Permission Model

RBAC (Role-Based Access Control) with optional ABAC (Attribute-Based) checks.

13.2 Permission Format

```
<resource>:<action>
```

Examples:

```
org:read      org:create      org:update
org:delete
po:read       po:create      po:approve
po:cancel
product:read  product:create product:update
product:delete
audit:read    audit:read:critical
system:tenant:cross
```

13.3 Permission Check Points

Layer 1 — Route-level (middleware):

```
router.post('/purchase-orders',
  requirePermission('po:create'),
  poController.create
)
```

- Blocks unauthorized users before controller executes
- Returns `403` if missing permission

Layer 2 — Row-level (service):

```
async approve(ctx, poId) {
  const po = await poRepo.findByIdOrThrow(ctx, poId)
  // User can only approve POs they're assigned as approver for
  if (po.approverId !== ctx.userId && !
    ctx.hasPermission('po:approve:any')) {
    throw new AuthorizationError('You can only approve POs
      assigned to you')
  }
  // ...
}
```

Layer 3 — Field-level (response shaping): - Some fields visible only to certain roles - E.g., `costPrice` on products visible to `finance:*` but not `sales:*` - Controller strips fields based on user permissions

13.4 Permission Hierarchy

- `system:*` — super-admin (rare; only platform team)
- `tenant:admin` — tenant administrator (full access within tenant)
- `<resource>:*` — full access to a resource (e.g., `po:*`)
- `<resource>:<action>` — specific action
- `<resource>:<action>:<scope>` — scoped action (e.g., `po:approve:any` vs `po:approve:own`)

13.5 Frontend Permission Checks

- `<PermissionGuard require="po:create"><Button>New P0</Button></PermissionGuard>` — hides button if user lacks permission
- Sidebar menu items filtered by permission
- Routes blocked at route level (redirect to 403 page)
- **Note:** Frontend checks are UX only — backend enforces truth. Never trust frontend for security.

14. Rate Limiting

14.1 Rate Limit Tiers

Tier	Limit	Window	Applies to
Public	60 req/min	IP	<code>/api/v1/auth/login</code> , <code>/api/v1/auth/register</code>
Authenticated	600 req/min	tenant + user	All authenticated endpoints
Heavy	30 req/min	tenant + user	Export, import, bulk operations
Read	1200 req/min	tenant + user	GET endpoints (more lenient)

14.2 Rate Limit Algorithm

- **Sliding window** via Redis (more accurate than fixed window)
- Key: `rate_limit:{tier}:{identifier}:{minute_bucket}`
- On each request: increment counter, check against limit, set TTL 60s

14.3 Rate Limit Response

```
HTTP/1.1 429 Too Many Requests
Content-Type: application/json
Retry-After: 23
X-RateLimit-Limit: 600
X-RateLimit-Remaining: 0
X-RateLimit-Reset: 1720000923
```

```
{
  "success": false,
  "error": {
    "code": "RATE_LIMIT.TOO_MANY_REQUESTS",
    "message": "Rate limit exceeded. Retry after 23
seconds.",
    "retryAfter": 23
  },
  "meta": { "correlationId": "01HXY..." }
}
```

14.4 Burst Allowance

- Token bucket allows short bursts above steady-state limit
- E.g., 600/min allows 100-token burst, refilled at 10/sec
- Prevents legitimate batch operations from being throttled

15. API Documentation Strategy

15.1 OpenAPI Specification

- Every endpoint documented in OpenAPI 3.1 spec
- Spec generated from zod schemas via `zod-to-openapi`
- Spec committed to repo at `packages/shared/openapi.yaml`
- Spec served at `GET /api/v1/_internal/openapi.json`

- Swagger UI at `/api/docs` (development only — disabled in production)

15.2 Documentation Requirements per Endpoint

- Summary (one line)
- Description (paragraph)
- Parameters (path, query, header) with types + examples
- Request body schema with example
- Response schemas for each status code (200, 201, 400, 401, 403, 404, 409, 422, 429, 500)
- Error codes specific to this endpoint
- Required permission
- Idempotency: required/optional/N/A
- Rate limit tier
- Sample request (curl)
- Sample response

15.3 SDK Generation

- TypeScript SDK auto-generated from OpenAPI spec via `openapi-typescript-codegen`
- SDK published as `@suop/api-client` npm package (internal)
- Frontend uses SDK instead of raw `fetch` — guarantees type safety
- Mobile app uses same SDK

15.4 API Changelog

- Every PR that adds/changes an endpoint updates `CHANGELOG.md`
- Entry format:

```
## [Unreleased]
### Added
- POST /api/v1/purchase-orders/{id}/submit – submit PO for approval
### Changed
- GET /api/v1/products now returns `upi` field (was previously `upicode`)
```

Deprecated

- GET /api/v1/products?include=barcodes – use GET /api/v1/products/{id}/barcodes instead

Removed

- None

- Release process generates `CHANGELOG.md` entries into versioned docs

15.5 Versioned Documentation

- Each API version has its own documentation snapshot
- `docs.api.suop.com/v1/...` for v1 docs
- `docs.api.suop.com/v2/...` for v2 docs
- Old versions preserved indefinitely (link rot forbidden)

16. Request/Response Logging

16.1 Logged per Request

- Method, path, query params (values redacted for sensitive fields)
- Request body (redacted for sensitive fields: `password`, `token`, `creditCard`)
- User ID, tenant ID (if authenticated)
- Response status, duration, response size
- Correlation ID

16.2 Sensitive Field Redaction

- Field names matching `/password|token|secret|key|auth|credit/i` have values replaced with `[REDACTED]`
- PII fields (`email`, `phone`, `aadhaar`, `pan`) hashed (SHA-256) for correlation without exposure
- Redaction config in `logging/redaction-config.ts`

16.3 Sampling

- 100% of mutating requests (POST/PUT/PATCH/DELETE) logged at INFO
 - 10% of GET requests logged at INFO (90% at DEBUG)
 - 100% of error responses (4xx, 5xx) logged at WARN/ERROR
 - 100% of slow requests (>1s) logged at WARN
-

17. CORS

17.1 Allowed Origins

- Production: `https://app.sudhamrit.com`, `https://erp.sudhamrit.com`
- Staging: `https://staging.sudhamrit.com`
- Development: `http://localhost:3000`, `http://localhost:3001`
- Mobile: capacitor origin (`capacitor://localhost`)

17.2 Headers

```
Access-Control-Allow-Origin: <allowed origin or null>
Access-Control-Allow-Methods: GET, POST, PUT, PATCH, DELETE,
OPTIONS
Access-Control-Allow-Headers: Authorization, Content-Type,
Idempotency-Key, X-Request-Id
Access-Control-Expose-Headers: X-Request-Id, X-RateLimit-*,
ETag, Location
Access-Control-Allow-Credentials: true
Access-Control-Max-Age: 86400
```

17.3 Preflight Caching

- `OPTIONS` preflight cached for 24 hours (`Access-Control-Max-Age: 86400`)
 - Reduces `OPTIONS` traffic to ~1 per origin per day per browser
-

18. API Security

18.1 Input Validation

- All inputs validated by zod schema before controller executes
- Unknown fields rejected (`z.object({...}).strict()`)
- String length limits enforced
- Numeric ranges enforced
- Date format validation

18.2 Output Encoding

- All JSON responses encoded by framework (no manual `JSON.stringify` of user data)
- No HTML responses from API (CSRF protection — see Security doc)
- Headers like `X-Content-Type-Options: nosniff` prevent MIME sniffing

18.3 File Upload Validation

- MIME type whitelist per category
- File size limits per category
- Magic byte verification (not just trusting Content-Type header)
- Virus scan via ClamAV before storage

18.4 SQL Injection Prevention

- Prisma parameterized queries everywhere — no raw SQL with string interpolation
- Raw queries (rare) use parameterized `Prisma.sql` template literal
- Code review enforces no `Prisma.$queryRawUnsafe` ever

18.5 Rate Limit per User (not just IP)

- Rate limits keyed on `userId` (authenticated) or `IP` (unauthenticated)
- Prevents single user from bypassing limits via IP rotation

- Failed login attempts: 10 per user per 15 min → account lockout (30 min)

19. Open Questions for Approval

#	Decision	Recommendation	Alternatives
Q-A1	Pagination default	Offset for lists, cursor for logs	Cursor everywhere
Q-A2	Filter syntax	Query param operators (<code>.gt</code> , <code>.in</code>)	JSON-encoded filter
Q-A3	Bulk max items	100	50, 500
Q-A4	Idempotency key TTL	24 hours	1 hour, 7 days
Q-A5	JWT algorithm	HS256 (Phase 0) → RS256 (Phase X)	RS256 from start
Q-A6	Access token TTL	15 minutes	5 min, 30 min
Q-A7	Refresh token TTL	30 days	7 days, 90 days
Q-A8	MFA requirement	Required for admin/finance	Required for all
Q-A9	API docs in production	Disabled (internal only)	Enabled (public)
Q-A10	SDK generation	Auto from OpenAPI	Manual

Approval Block

Approved by: _____ **Date:** _____

End of Enterprise API Standards Document

03_SECURITY_ARCHITECTURE

- [SUOP ERP v1.0 — Enterprise Security Architecture](#)
 - [1. Purpose](#)
 - [2. Threat Model](#)
 - [3. Authentication](#)
 - [4. Authorization](#)
 - [5. JWT Strategy](#)
 - [6. Refresh Token Strategy](#)
 - [7. Password Policy](#)
 - [8. MFA \(see §3.7\)](#)
 - [9. Encryption](#)
 - [10. Secrets Management](#)
 - [11. OWASP Top 10 Coverage](#)
 - [12. SQL Injection Prevention](#)
 - [13. XSS \(Cross-Site Scripting\)](#)
 - [14. CSRF \(Cross-Site Request Forgery\)](#)
 - [15. SSRF \(Server-Side Request Forgery\)](#)
 - [16. Audit Security](#)
 - [17. API Security](#)
 - [18. Infrastructure Security](#)
 - [19. Developer Security](#)
 - [20. Incident Response](#)
 - [21. Compliance](#)
 - [22. Open Questions for Approval](#)
 - [Approval Block](#)

SUOP ERP v1.0 — Enterprise Security Architecture

Field	Value
Document Version	1.0

Field	Value
Status	DRAFT — Awaiting Approval
Date	2026-07-11
Approval Required	Project Owner
Compliance Frameworks	OWASP Top 10, ISO 27001, SOC 2, FSSAI Cybersecurity

1. Purpose

This document defines the complete security architecture for SUOP ERP v1.0. It is informed by OWASP Top 10 (2021), ISO 27001 controls, SOC 2 Trust Services Criteria, and FSSAI cybersecurity guidelines for food businesses.

Hard rule: No code ships without passing security review against this document. CI runs SAST (Static Application Security Testing), dependency scanning, and secret detection on every PR.

2. Threat Model

2.1 Assets to Protect

Asset	Sensitivity	Impact of Breach
User credentials (passwords, tokens)	Critical	Account takeover, data exfiltration
PII (employee, customer data)	High	Legal, regulatory, reputational
Financial data (prices, costs, invoices)	High	Competitive harm, fraud

Asset	Sensitivity	Impact of Breach
Recipe/formula data	Critical	IP theft, competitive loss
Quality data (NCRs, COAs, audit logs)	High	Regulatory non-compliance
Batch genealogy / recall data	Critical	Food safety liability
Supplier contracts / pricing	High	Competitive harm
Audit logs	High	Loss of legal evidence

2.2 Threat Actors

Actor	Capability	Motivation
External attacker	Internet-facing exploit, phishing	Financial, ransomware
Malicious insider	Authenticated access, knows system	Sabotage, data theft
Negligent insider	Authenticated access	Accidental disclosure
Compromised supplier	Limited tenant access	Pivot attack
Script kiddie	Automated scans	Notoriety

2.3 Attack Surfaces

1. **Public API** — `/api/v1/*` (auth required except login)
2. **Frontend** — Next.js app (XSS, CSRF)
3. **Database** — Direct (should be impossible from internet)
4. **File storage** — S3/MinIO (signed URLs, bucket policy)
5. **Admin endpoints** — `/api/v1/_internal/*` (API key required)
6. **CI/CD pipeline** — Compromise deploys malicious code

7. **Developer machines** — Source code, secrets

8. **Third-party integrations** — Email, SMS, payment providers

3. Authentication

3.1 Password Policy

Rule	Requirement
Minimum length	12 characters
Maximum length	128 characters (prevents DoS via long hash)
Complexity	At least 3 of: uppercase, lowercase, digit, special
Common password check	Reject top 10,000 common passwords (HaveIBeenPwned API + local list)
History	Cannot reuse last 10 passwords
Expiry	90 days (configurable per tenant; disabled if MFA enabled)
Lockout	5 failed attempts → 30-min lockout; 3 lockouts in 24h → admin unlock required
Reset	Token-based, single-use, expires in 30 min

3.2 Password Storage

- **Algorithm:** Argon2id (winner of Password Hashing Competition)
- **Parameters:** `memoryCost=19456` (19 MB), `timeCost=2`, `parallelism=1`

- **Tuning target:** ~250ms per hash on production hardware
- **Salt:** 16-byte random salt per password (Argon2 handles internally)
- **Format:** `$argon2id$v=19$m=19456,t=2,p=1$<salt>$<hash>`

Forbidden: MD5, SHA-1, SHA-256 (without salt+pepper), bcrypt with cost < 12, PBKDF2 with < 100k iterations.

3.3 Password Pepper

- Argon2id hash further encrypted with a server-side pepper (32-byte random)
- Pepper stored in secrets manager (not in DB, not in code)
- Compromise of DB alone does not reveal passwords
- Pepper rotation requires password reset for all users (rare; documented procedure)

3.4 Login Flow

1. Client sends `POST /api/v1/auth/login` with `{email, password}`
2. Server loads user by email (rate-limited: 10/min/IP, 5/min/email)
3. Server verifies password with Argon2id (`argon2.verify`)
4. If MFA enabled: return `{ status: 'MFA_REQUIRED', mfaSessionToken }` (5-min TTL)
5. Client sends `POST /api/v1/auth/mfa/verify` with `{mfaSessionToken, code}`
6. Server verifies TOTP, issues access token (15 min) + refresh token cookie (30 days)
7. Login audit logged; suspicious login (new IP, new device) triggers email alert

3.5 Failed Login Protection

- 5 failed attempts per user → 30-min logout
- 10 failed attempts per IP → 1-hour IP block
- Failed attempts logged; 10+ failures in 1 hour trigger security alert
- Account logout requires password reset (forces user to verify identity via email)

3.6 Session Management

- **Access token:** JWT, 15 min TTL, in-memory in frontend (not localStorage — XSS protection)
- **Refresh token:** Random 64-byte token, 30-day TTL, httpOnly secure cookie
- **Refresh token rotation:** Every refresh issues a new refresh token; old one invalidated
- **Refresh token replay detection:** Reuse of invalidated refresh token revokes entire session
- **Device tracking:** Each session tied to a device fingerprint; new device triggers email alert

3.7 Multi-Factor Authentication (MFA)

- **TOTP-based** (RFC 6238) — Google Authenticator, Authy, 1Password
- **Required for:**
 - All admin roles (`tenant:admin` , `system:*`)
 - Finance roles (`finance:*`)
 - Quality roles with `ncr:approve` , `coa:sign` , `recall:initiate`
- **Optional for:** Other users (configurable per tenant)
- **Backup codes:** 10 single-use codes generated at enrollment; hashed in DB
- **Enrollment:** User scans QR code → enters code to confirm → backup codes displayed once
- **Reset:** Admin can reset MFA (audited at `CRITICAL`); user must re-enroll

3.8 Single Sign-On (Future)

- SAML 2.0 (Microsoft Entra, Google Workspace) — Phase X
 - OIDC (OAuth 2.0 extension) — Phase X
 - Out of scope for Phase 0; architecture allows addition without rewrite
-

4. Authorization

4.1 RBAC Model

User → Role → Permission → Resource:Action

- **User:** authenticated identity
- **Role:** collection of permissions (e.g., `quality_manager`, `procurement_officer`)
- **Permission:** specific action on resource (`po:approve`, `product:create`)
- **Resource:** entity type (`purchase_order`, `product`, `user`)

4.2 Permission Catalog

All permissions catalogued in `packages/shared/src/enums/permission.enum.ts`. Examples:

```
export enum Permission {  
  // Organization  
  ORG_READ = 'org:read',  
  ORG_CREATE = 'org:create',  
  ORG_UPDATE = 'org:update',  
  ORG_DELETE = 'org:delete',  
  // Auth  
  AUTH_MANAGE_USERS = 'auth:manage_users',  
  AUTH_MANAGE_ROLES = 'auth:manage_roles',  
  AUTH_RESET_PASSWORD = 'auth:reset_password',  
  // Product  
  PRODUCT_READ = 'product:read',  
  PRODUCT_CREATE = 'product:create',  
  PRODUCT_UPDATE = 'product:update',  
  PRODUCT_DELETE = 'product:delete',  
  // Purchase Order  
  PO_READ = 'po:read',  
  PO_CREATE = 'po:create',  
}
```

```

PO_APPROVE = 'po:approve',
PO_APPROVE_ANY =
    'po:approve:any', // approve any PO, not just
    assigned
PO_CANCEL = 'po:cancel',
// Quality
NCR_CREATE = 'ncr:create',
NCR_APPROVE = 'ncr:approve',
COA_SIGN = 'coa:sign',
RECALL_INITIATE = 'recall:initiate',
// System
SYSTEM_TENANT_CROSS = 'system:tenant:cross',
SYSTEM_REFERENCE_UPDATE = 'system:reference:update',
AUDIT_READ = 'audit:read',
AUDIT_READ_CRITICAL = 'audit:read:critical',
}

```

4.3 Permission Enforcement (3 layers — see API Standards §13)

1. **Route-level middleware:** `requirePermission('po:create')` blocks unauthorized users before controller
2. **Service-level row checks:** Verify user can access specific record (e.g., PO assigned to them)
3. **Field-level response shaping:** Strip fields user lacks permission to see (e.g., `costPrice`)

4.4 Default Roles

Role	Permissions
<code>tenant_admin</code>	All <code>*:read/create/update/delete</code> within tenant
<code>quality_manager</code>	<code>quality:*</code> , <code>ncr:*</code> , <code>capa:*</code> , <code>coa:*</code> , <code>audit:read</code>
<code>quality_inspector</code>	

Role	Permissions
	quality:read, iqc:inspect, ncr:create, coa:read
procurement_officer	po:read/create, supplier:read, grn:read
procurement_manager	po:*, supplier:*, grn:create
warehouse_operator	inventory:read, grn:post, putaway:execute
warehouse_manager	warehouse:*, inventory:*, grn:*
production_manager	production:*, recipe:read, bom:read
plant_head	All within plant scope
finance_manager	finance:*, cost:*, invoice:*
auditor	*:read, audit:read:critical (read-only)
system_admin	system:* (platform team only)

4.5 Permission Inheritance

- Roles are flat (no inheritance) — simpler, more predictable
- Users can have multiple roles; permissions union
- Tenant admin role granted via separate UI (audited at `CRITICAL`)

4.6 Permission Changes Audit

- Every role assignment / revocation audit-logged
 - Every permission change audit-logged
 - High-severity changes (`system:*`, `tenant_admin` grant) trigger security alert
-

5. JWT Strategy

5.1 Token Structure (see API Standards §12.2)

5.2 Signing Key Management

- **Phase 0:** Single HMAC-SHA256 key in secrets manager
- **Key rotation:** Every 90 days; old key valid for 7 days after rotation (overlap)
- **Multi-key support:** JWT header includes `kid` (key ID) to identify signing key
- **Future (RS256):** Private key in HSM/KMS; public key distributed to verifiers

5.3 Token Validation

On every request: 1. Extract `Authorization: Bearer <token>` header 2. Verify signature (correct key, correct algorithm — reject `alg: none` attacks) 3. Verify `exp` not expired 4. Verify `iss` matches `suop-erp` 5. Verify `aud` matches `suop-frontend` (or `suop-mobile` for mobile) 6. Check `jti` not in Redis blocklist 7. Load user from DB (cached 60s) — verify `status === 'ACTIVE'` 8. Set `RequestContext.user` and `RequestContext.tenant`

5.4 Token Compromise Response

If a token is compromised: 1. Add `jti` to Redis blocklist (TTL = remaining token TTL) 2. Revoke all user's refresh tokens (force re-login) 3. Audit log at `CRITICAL` severity 4. Notify user via email + in-app 5. Optional: Force password reset

6. Refresh Token Strategy

6.1 Storage

- **Client:** httpOnly secure cookie (refresh_token=<random> ; SameSite=Strict; Secure; HttpOnly)
- **Server:** Hashed (SHA-256) in refresh_tokens table with userId , expiresAt , deviceFingerprint , revokedAt

6.2 Refresh Flow

1. Access token expired → client receives 401
2. Client calls POST /api/v1/auth/refresh (no body; refresh cookie sent automatically)
3. Server validates cookie token against DB:
 - Exists in DB? (not revoked)
 - Not expired?
 - Device fingerprint matches?
4. Issue new access token + new refresh token
5. **Old refresh token immediately revoked** (replay detection)
6. New refresh token in cookie

6.3 Refresh Token Replay Detection

If a revoked refresh token is used: 1. Server detects reuse (token in DB but revokedAt set) 2. **Revoke entire session** (all refresh tokens for this user) 3. Force re-login 4. Audit log at CRITICAL severity 5. Alert security team

This protects against refresh token theft — attacker using stolen token alerts legitimate user.

6.4 Refresh Token Revocation

- **User logout:** POST /api/v1/auth/logout revokes current refresh token
- **Admin force-logout:** POST /api/v1/_internal/users/{id}/revoke-sessions revokes all sessions

- **Password change:** All refresh tokens revoked (force re-login everywhere)
 - **Account lockout:** All refresh tokens revoked
-

7. Password Policy

(See §3.1 for full policy)

7.1 Password Strength Estimation

In addition to character rules, use `zxcvbn` to estimate password strength: - Score 0-2: Reject (too weak) - Score 3: Acceptable (warning shown) - Score 4: Strong - Score 5: Excellent

7.2 Password Reset Flow

1. User clicks “Forgot password” → enters email
2. Server generates single-use reset token (UUID v7, 30-min TTL)
3. Token stored hashed in `password_reset_tokens` table
4. Email sent with link `https://app.sudhamrit.com/reset-password?token=...`
5. User clicks link, enters new password
6. Server validates token (exists, not expired, not used)
7. Server updates password (Argon2id hash), invalidates token
8. **All existing sessions revoked** (force re-login everywhere)
9. Audit log entry; email notification to user

7.3 Password Reset Protection

- Rate limited: 3 reset requests per email per hour (prevents email bombing)
 - Token single-use (reuse → rejected)
 - Token expires 30 min after issue
 - Reset invalidates all sessions (prevents attacker maintaining access after legitimate reset)
-

8. MFA (see §3.7)

9. Encryption

9.1 Encryption Layers

Layer	Algorithm	Purpose
Transport (HTTPS)	TLS 1.3	Data in transit
Database connection	TLS 1.3	App ↔ DB
Database at rest	AES-256 (Supabase/ RDS managed)	DB files on disk
File storage at rest	AES-256 (S3 SSE-KMS)	S3 objects
Backup encryption	AES-256 (pg_dump -- encrypt or S3 SSE)	Backups
Secrets at rest	KMS-managed	Secrets in secrets manager
Sensitive column encryption	pgcrypto (AES-256- GCM)	PII columns
JWT signing	HS256 (HMAC- SHA256)	Token integrity
Password hashing	Argon2id	Password storage
TOTP secret storage	AES-256-GCM (pgcrypto)	MFA secrets

9.2 TLS Configuration

- **Minimum version:** TLS 1.3 (TLS 1.2 only for legacy clients; disabled by 2027)

- **Cipher suites:** TLS_AES_256_GCM_SHA384, TLS_AES_128_GCM_SHA256, TLS_CHACHA20_POLY1305_SHA256
- **HSTS:** Strict-Transport-Security: max-age=63072000; includeSubDomains; preload
- **Certificate:** Let's Encrypt (auto-renewed) or managed cert (Supabase/AWS)
- **Certificate pinning:** Not used (CA-based trust sufficient; pinning causes client breakage on rotation)

9.3 Sensitive Column Encryption

For PII / regulated data:

```
-- Aadhaar number stored encrypted
ALTER TABLE users ADD COLUMN aadhaar_encrypted BYTEA;
-- Encrypt on insert
INSERT INTO users (... , aadhaar_encrypted) VALUES (... ,
    pgp_sym_encrypt($aadhaar, $key));
-- Decrypt on read
SELECT pgp_sym_decrypt(aadhaar_encrypted, $key) AS aadhaar
    FROM users WHERE id = $1;
```

- Encryption key in secrets manager (not in DB)
- Key rotation requires re-encrypting all rows (documented procedure; tested in DR drill)

9.4 Application-Level Encryption

For especially sensitive data (recipe formulas, supplier contracts): - Encrypt in application code before storing - Decryption key in secrets manager - Key access audited (every decryption logged)

10. Secrets Management

10.1 Secret Hierarchy

Type	Storage	Examples
Application secrets	<code>.env</code> (dev) / secrets manager (prod)	DB URL, JWT secret, API keys
Infrastructure secrets	Secrets manager only	DB password, S3 access keys
User secrets	DB (hashed)	Passwords, MFA secrets
Certificates	Secrets manager + filesystem	TLS certs, signing keys

10.2 Secrets Manager

- **Production:** AWS Secrets Manager or HashiCorp Vault
- **Staging:** Same as production (separate instance)
- **Development:** `.env` file (gitignored, `.env.example` committed)
- **CI/CD:** GitHub Actions secrets

10.3 Secret Rotation

Secret	Rotation Frequency	Method
JWT signing key	90 days	Multi-key overlap (old valid 7 days)
DB password	90 days	Rotate via secrets manager; app reconnects
S3 access keys	90 days	IAM role preferred (no key rotation)

Secret	Rotation Frequency	Method
SMTP password	180 days	Manual
Encryption keys	365 days	Re-encrypt all data (tested in DR drill)
API keys (integrations)	365 days	Issue new + revoke old

10.4 Secret Access Audit

- Every secret read from secrets manager logged
- Unusual access patterns alert security (e.g., secret read from new IP)
- Production secret access requires MFA (for humans)

10.5 Forbidden

- ❌ Secrets in code (even commented out)
- ❌ Secrets in git (commit hooks block via `git-secrets`)
- ❌ Secrets in CI logs (auto-redacted)
- ❌ Secrets in error messages
- ❌ Secrets in URLs (URLs logged everywhere)

11. OWASP Top 10 Coverage

11.1 A01: Broken Access Control

- **Mitigation:** 3-layer authorization (route, service, field) — see §4
- **Tests:** Integration tests verify cross-tenant access blocked, missing permission blocked
- **Audit:** Failed authorization attempts logged at WARN

11.2 A02: Cryptographic Failures

- **Mitigation:** TLS 1.3, AES-256 at rest, Argon2id passwords, no MD5/SHA-1
- **Tests:** SAST scans for weak crypto; CI fails on use

- **Audit:** Annual crypto review

11.3 A03: Injection

- **Mitigation:** Prisma parameterized queries everywhere; no `$queryRawUnsafe`
- **Tests:** SAST scans for raw SQL; integration tests for injection patterns
- **Audit:** Code review checklist item

11.4 A04: Insecure Design

- **Mitigation:** Threat modeling per feature; security design review for new modules
- **Tests:** Architecture review board for new modules
- **Audit:** Quarterly architecture review

11.5 A05: Security Misconfiguration

- **Mitigation:** Infrastructure as Code (Terraform); config validation at boot (zod); no default credentials
- **Tests:** CIS benchmark scanning; automated config drift detection
- **Audit:** Monthly config audit

11.6 A06: Vulnerable & Outdated Components

- **Mitigation:** `npm audit` on every PR; Dependabot for auto-PRs; Snyk for deep scanning
- **Tests:** CI fails on high/critical vulnerabilities
- **Audit:** Weekly dependency review

11.7 A07: Identification & Authentication Failures

- **Mitigation:** Argon2id, MFA, account lockout, session rotation
- **Tests:** Penetration tests for auth flows
- **Audit:** Annual pen test

11.8 A08: Software & Data Integrity Failures

- **Mitigation:** Signed commits; signed container images; SBOM (Software Bill of Materials); subresource integrity for frontend assets
- **Tests:** CI verifies signatures before deploy
- **Audit:** Continuous SBOM monitoring

11.9 A09: Security Logging & Monitoring Failures

- **Mitigation:** Audit framework (see Phase 0 §11); Sentry for errors; Loki for logs; Prometheus for metrics
- **Tests:** Audit log integrity tests
- **Audit:** Log review automation alerts on anomalies

11.10 A10: Server-Side Request Forgery (SSRF)

- **Mitigation:** No user-controlled URLs in outbound HTTP calls; URL whitelist for outbound integrations
- **Tests:** Integration tests with SSRF payloads
- **Audit:** Code review checklist item

12. SQL Injection Prevention

12.1 Parameterized Queries

- All DB access via Prisma — parameterized by default
- Raw queries use `Prisma.sql` template literal (parameterized):

```
//  SAFE  
const result = await prisma.$queryRaw`SELECT * FROM users  
  WHERE email = ${email}`  
  
//  FORBIDDEN – string interpolation  
const result = await prisma.$queryRawUnsafe(`SELECT * FROM  
  users WHERE email = '${email}'`)
```

12.2 CI Enforcement

- ESLint rule `@suop/eslint-config/no-queryRawUnsafe` blocks `$queryRawUnsafe`
- Code review checklist item
- SAST scan (SonarQube) flags raw SQL

12.3 Dynamic Identifier Escaping

For dynamic table/column names (rare — usually a code smell): - Use Prisma's `Prisma.raw` with explicit allowlist - Never interpolate user input into identifiers - Example:

```
const allowedColumns = ['name', 'sku', 'status']
if (!allowedColumns.includes(sortColumn)) throw new
  ValidationError('Invalid sort column')
const result = await prisma.$queryRaw`SELECT $
  {Prisma.raw(sortColumn)} FROM products`
```

13. XSS (Cross-Site Scripting)

13.1 Output Encoding

- React auto-encodes all interpolated values (`{userInput}`) — no raw HTML by default
- `dangerouslySetInnerHTML` forbidden; if needed (rare), sanitize with `DOMPurify` first
- CI scans for `dangerouslySetInnerHTML` usage; requires code review approval

13.2 Content Security Policy

```
Content-Security-Policy:
  default-src 'self';
  script-src 'self' 'wasm-unsafe-eval';
```

```
style-src 'self' 'unsafe-inline';
img-src 'self' data: https:; # allow images from S3
font-src 'self' data:;
connect-src 'self' https://api.sudhamrit.com;
frame-ancestors 'none';
form-action 'self';
base-uri 'self';
object-src 'none';
upgrade-insecure-requests;
```

- No `unsafe-inline` for scripts (strict)
- `unsafe-inline` for styles (Tailwind requires; acceptable risk)
- No `unsafe-eval` for scripts (no `eval()` anywhere)

13.3 Other Security Headers

```
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: camera=(), microphone=(), geolocation=()
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Embedder-Policy: require-corp
Cross-Origin-Resource-Policy: same-origin
```

13.4 Cookie Security

- All cookies: `Secure; HttpOnly; SameSite=Strict` (or `Lax` for navigation cookies)
 - No sensitive data in cookies other than refresh token
 - Cookie prefix: `__Host-` for refresh token cookie (extra protection)
-

14. CSRF (Cross-Site Request Forgery)

14.1 Protection Strategy

- **API:** CSRF not vulnerable — API uses `Authorization: Bearer` header (not cookies), so CSRF cannot forge requests
- **Refresh token endpoint:** Uses cookie, but `SameSite=Strict` prevents CSRF
- **Server-side form submissions:** None (SPA + API architecture)

14.2 Double-Submit Cookie (Future, if Needed)

If server-side rendering with cookies is added: - Issue CSRF token in cookie
+ require in header for mutating requests - Verify match on server

15. SSRF (Server-Side Request Forgery)

15.1 Outbound HTTP Calls

- All outbound URLs validated against allowlist of domains
- Example: email service = `smtp.sendgrid.net`, file storage = `s3.amazonaws.com`
- User input never used as URL for outbound requests

15.2 URL Validation

```
const ALLOWED_OUTBOUND_DOMAINS = [  
  'api.sendgrid.com',  
  's3.ap-south-1.amazonaws.com',  
  's3.amazonaws.com',  
  'minio.local',  
]  
  
function validateOutboundUrl(url: string): void {  
  const parsed = new URL(url)
```

```
if (!ALLOWED_OUTBOUND_DOMAINS.includes(parsed.hostname)) {  
    throw new SecurityError(`Outbound requests to $  
        {parsed.hostname} not allowed`)  
}  
  
// Block internal IPs (prevent SSRF to metadata service)  
if (isInternalIp(parsed.hostname)) {  
    throw new SecurityError(`Outbound requests to internal  
        IPs forbidden`)  
}  
}
```

15.3 Internal IP Blocking

- Block outbound requests to 127.0.0.0/8, 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16, 169.254.0.0/16 (AWS metadata service)
- Prevents SSRF attack from compromising cloud credentials

16. Audit Security

16.1 Audit Log Integrity

- Audit log table is **append-only** (no UPDATE, no DELETE) — enforced by PostgreSQL RLS
- Daily hash chain: each day's audit logs hashed; hash published to external system (S3 with object lock)
- Tampering detected by re-computing hash chain

16.2 Audit Log Access

- `audit:read` permission required for normal audit logs
- `audit:read:critical` required for high-severity logs (security events, permission changes)
- All audit log reads are themselves audit-logged (meta-audit)
- Audit logs never exported without approval

16.3 Audit Log Retention

- Hot: 1 year (online queryable)
 - Warm: 1-3 years (archived, restorable)
 - Cold: 3-7+ years (Glacier, compliance)
 - Never deleted without legal sign-off
-

17. API Security

(see API Standards §18 for full details)

17.1 Key Controls

- All endpoints require authentication (except login, register, health, reference data)
- All endpoints rate-limited
- All inputs validated by zod schema
- All mutations require idempotency key
- All mutations audit-logged
- Sensitive endpoints (delete, blacklist, recall) require MFA re-authentication

17.2 Sensitive Endpoint Re-Authentication

For especially destructive actions: - `DELETE /api/v1/products/{id}` (hard delete — admin only) - `POST /api/v1/suppliers/{id}/blacklist` - `POST /api/v1/recalls` (initiate recall) - `POST /api/v1/_internal/users/{id}/revoke-sessions` - `POST /api/v1/_internal/tenants/{id}/purge` (purge tenant data)

Server requires: 1. Recent authentication (within last 5 minutes) 2. MFA code re-entered 3. Audit log at `CRITICAL` severity 4. Optional: 2-person rule (another admin approves)

18. Infrastructure Security

18.1 Network

- **VPC:** All resources in private VPC; only load balancer has public IP
- **Subnets:** App in private subnet; DB in isolated subnet (no internet egress)
- **Security groups:** Least privilege — app can reach DB only on port 5432; DB cannot reach anything
- **NAT gateway:** App egress to internet via NAT (for outbound integrations)
- **VPC endpoints:** S3, Secrets Manager via VPC endpoint (no public internet)

18.2 Database

- **No public IP** — only accessible from app subnet
- **TLS required** for all connections
- **Row-Level Security** for tenant isolation
- **Application role** has limited privileges (no DDL)
- **Migration role** separate, used only by CI/CD

18.3 Redis

- **Auth required** (`requirepass` or ACL)
- **TLS enabled**
- **No public IP** — only accessible from app subnet
- **Maxmemory policy:** `allkeys-lru` (cache mode; if lost, app rebuilds)

18.4 File Storage (S3 / MinIO)

- **Bucket private** — no public access
- **Access via signed URLs only** (15-min TTL)
- **Bucket policy** denies non-TLS requests
- **Versioning enabled** (recoverable from accidental delete)
- **Cross-region replication** for DR

18.5 Container Security

- **Non-root user** in container (`USER 1001`)
- **Read-only filesystem** (except `/tmp`)
- **No new privileges** flag (`--security-opt=no-new-privileges`)
- **Resource limits** (CPU, memory)
- **Image scanning** via Trivy before deploy
- **Base image:** `node:20-alpine` (small attack surface)

18.6 Kubernetes (Production)

If deploying via K8s: - **Network policies** restrict pod-to-pod communication - **Pod security standards** (restricted) - **RBAC** for K8s API - **Admission controllers** (OPA Gatekeeper) enforce policies - **Secrets** via external-secrets operator (sync from secrets manager)

19. Developer Security

19.1 Developer Machines

- **Disk encryption** mandatory (FileVault for Mac, BitLocker for Windows)
- **Screen lock** after 5 min idle
- **Antivirus** (corporate standard)
- **No production data on dev machines** — use sanitized staging data

19.2 Source Code Security

- **Signed commits** required (GPG or SSH signing)
- **Branch protection** on `main` and `develop`
- **PR review** required (1 reviewer for code, 2 for security-sensitive changes)
- **Secret scanning** via GitHub Advanced Security (or git-secrets locally)
- **Dependency scanning** via Dependabot

19.3 Access Management

- **Production access:** MFA required; JIT (just-in-time) access via approval workflow
 - **Staging access:** All developers
 - **DB access:** Via bastion host + SSH key only; no direct DB access from dev machines
 - **Cloud console:** SSO + MFA; role-based access; audit logged
-

20. Incident Response

20.1 Incident Severity

Severity	Definition	Response Time
SEV-1	Active breach, data loss, ransomware	15 min
SEV-2	Vulnerability being exploited, limited impact	1 hour
SEV-3	Vulnerability discovered, not exploited	4 hours
SEV-4	Hardening opportunity, no active threat	1 week

20.2 Incident Response Plan

1. **Detect** (monitoring alert, user report, security researcher)
2. **Triage** (on-call engineer assesses severity)
3. **Contain** (revoke tokens, block IPs, isolate systems)
4. **Investigate** (logs, audit trail, forensics)
5. **Eradicate** (apply fix, rotate credentials)
6. **Recover** (restore service, verify integrity)
7. **Post-mortem** (within 7 days; root cause, action items)

20.3 Communication

- **Internal:** Slack incident channel; status page for employees
 - **Customer:** Email notification within 72 hours of confirmed breach (GDPR / DPDP requirement)
 - **Regulatory:** Notify FSSAI / CERT-In within 72 hours for applicable incidents
 - **Public:** Blog post after containment (transparency)
-

21. Compliance

21.1 Frameworks

Framework	Applicability	Status
OWASP Top 10	All	Compliant (this document)
ISO 27001	Future (after v1.0)	Architecture ready
SOC 2 Type II	Future (after v1.0)	Architecture ready
FSSAI Cybersecurity	Food business	Compliant
DPDP Act 2023 (India)	Personal data	Compliant
GST	Invoice data	Compliant (retention)

21.2 Data Subject Access Requests (DPDP)

- User can request all their data: `GET /api/v1/_internal/users/me/data-export`
 - User can request deletion: `DELETE /api/v1/_internal/users/me` (soft delete + audit; data retained per compliance)
 - Response within 30 days
-

22. Open Questions for Approval

#	Decision	Recommendation	Alternatives
Q-S1	Password policy	12+ chars, 3 of 4 categories	16+ chars (more strict)
Q-S2	MFA requirement	Admin/finance/recall roles	All users
Q-S3	JWT algorithm	HS256 now → RS256 later	RS256 from start
Q-S4	Column encryption	pgcrypto for PII	Application-level only
Q-S5	Secrets manager	AWS Secrets Manager	HashiCorp Vault
Q-S6	Pen test frequency	Annual + after major changes	Quarterly
Q-S7	CSP strictness	Strict (no unsafe-inline scripts)	Permissive
Q-S8	Sensitive endpoint re-auth	MFA within 5 min	Password re-entry
Q-S9	Incident response	72-hour notification	24-hour (GDPR-style)
Q-S10	Production access	JIT (just-in-time)	Standing access

Approval Block

Approved by: _____ Date: _____

04_DEVOPS_ARCHITECTURE

- [SUOP ERP v1.0 — Enterprise DevOps Architecture](#)
 - [1. Purpose](#)
 - [2. Environments](#)
 - [3. Docker](#)
 - [4. CI/CD](#)
 - [5. Git Strategy](#)
 - [6. Deployment](#)
 - [7. Secrets Management](#)
 - [8. Infrastructure as Code](#)
 - [9. Monitoring](#)
 - [10. Logging](#)
 - [11. Tracing](#)
 - [12. Alerting](#)
 - [13. Environment Configuration](#)
 - [14. Backup & Recovery](#)
 - [15. Open Questions for Approval](#)
 - [Approval Block](#)

SUOP ERP v1.0 — Enterprise DevOps Architecture

Field	Value
Document Version	1.0
Status	DRAFT — Awaiting Approval
Date	2026-07-11
Approval Required	Project Owner

1. Purpose

This document defines the complete DevOps architecture for SUOP ERP v1.0: how code moves from developer laptop to production, how environments are managed, how deployments are released and rolled back, and how the system is monitored.

Hard rule: No code reaches production except via this pipeline. No manual SSH-and-edit. No “hotfix directly on the server.” Every change is a PR → CI → CD pipeline.

2. Environments

2.1 Environment Catalog

Environment	Purpose	Data	Access
Local	Developer machine	Synthetic seed data	Developer
CI	Test runner	Test fixtures	CI system only
Staging	Pre-production testing	Anonymized production snapshot	All developers + QA
Production	Live customer use	Real data	Operations team only

2.2 Environment Parity

Staging must match production as closely as possible: - Same Postgres version - Same Redis version - Same Node/Bun version - Same environment variables (different values) - Same infrastructure (smaller instance sizes for cost) - Same third-party integrations (test accounts)

Divergences allowed: - Instance size (staging smaller) - Backup frequency (staging less frequent) - Number of replicas (staging single instance) - External integrations use sandbox/test accounts

2.3 Environment Promotion

```
Local → CI (auto on PR) → Staging (auto on merge to develop)
→ Production (manual approval on merge to main)
```

- **Local → CI:** Automatic on PR creation
- **CI → Staging:** Automatic on merge to `develop` branch
- **Staging → Production:** Manual approval gate after merge to `main`; deploy during business hours (10 AM - 4 PM IST)

2.4 Production Deploy Windows

- **Standard:** Tuesday + Thursday, 10 AM - 4 PM IST
 - **Emergency hotfix:** Anytime, requires SEV-1/SEV-2 + on-call approval
 - **Blackout periods:** Quarter-end (GST filing), audit periods, festival peaks (Diwali, etc.)
 - **No Friday deploys** (prevents weekend incident response)
-

3. Docker

3.1 Container Strategy

All three apps (web, backend, mobile-API) ship as Docker containers: - **Reproducible** — same image runs in staging and production - **Isolated** — dependencies bundled - **Scalable** — orchestrate via Kubernetes or container service - **Secured** — non-root user, read-only filesystem, scanned images

3.2 Dockerfile — Backend (docker/Dockerfile.backend)

```
# Multi-stage build for smallest image
FROM oven/bun:1.1-alpine AS base
WORKDIR /app

# Stage 1: Install dependencies
FROM base AS deps
COPY package.json bun.lockb ./
COPY apps/backend/package.json ./apps/backend/
COPY packages/shared/package.json ./packages/shared/
COPY packages/prisma/package.json ./packages/prisma/
RUN bun install --frozen-lockfile --production

# Stage 2: Build
FROM base AS builder
COPY --from=deps /app/node_modules ./node_modules
COPY . .
RUN bun run build:backend

# Stage 3: Production image
FROM oven/bun:1.1-alpine AS runner
WORKDIR /app

# Non-root user
RUN addgroup -g 1001 -S nodejs && adduser -S suop -u 1001
USER suop

# Copy built artifacts
COPY --from=builder --chown=suop:nodejs /app/apps/backend/
  dist ./dist
COPY --from=builder --chown=suop:nodejs /app/apps/backend/
  package.json ./package.json
COPY --from=builder --chown=suop:nodejs /app/node_modules ./
  node_modules
COPY --from=builder --chown=suop:nodejs /app/packages ./
  packages
```

```
# Read-only filesystem
RUN mkdir /tmp/suop && chown suop:nodejs /tmp/suop
ENV TMPDIR=/tmp/suop

# Health check
HEALTHCHECK --interval=30s --timeout=3s --retries=3 \
  CMD wget -q --spider http://localhost:3030/api/v1/_internal/health || exit 1

EXPOSE 3030
CMD ["bun", "run", "dist/main.js"]
```

3.3 Dockerfile — Web (docker/Dockerfile.web)

```
FROM node:20-alpine AS base
RUN apk add --no-cache libc6-compat
WORKDIR /app

# Stage 1: Install
FROM base AS deps
COPY package.json package-lock.json ./
COPY apps/web/package.json ./apps/web/
COPY packages/shared/package.json ./packages/shared/
COPY packages/ui/package.json ./packages/ui/
RUN npm ci

# Stage 2: Build
FROM base AS builder
COPY --from=deps /app/node_modules ./node_modules
COPY . .
ENV NEXT_TELEMETRY_DISABLED=1
RUN npm run build:web

# Stage 3: Production (Next.js standalone)
FROM node:20-alpine AS runner
WORKDIR /app
ENV NODE_ENV=production
```

```
ENV NEXT_TELEMETRY_DISABLED=1

RUN addgroup -g 1001 -S nodejs && adduser -S nextjs -u 1001
USER nextjs

COPY --from=builder --chown=nextjs:nodejs /app/apps/
web/.next/standalone ./
COPY --from=builder --chown=nextjs:nodejs /app/apps/
web/.next/static ./apps/web/.next/static
COPY --from=builder --chown=nextjs:nodejs /app/apps/web/
public ./apps/web/public

EXPOSE 3000
ENV PORT=3000
CMD ["node", "apps/web/server.js"]
```

3.4 Docker Compose (Development)

docker/docker-compose.yml :

```
version: '3.9'
services:
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: suop
      POSTGRES_USER: suop
      POSTGRES_PASSWORD: suop_dev_only
    ports: ['5432:5432']
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ['CMD-SHELL', 'pg_isready -U suop']
      interval: 5s
      retries: 5

  redis:
    image: redis:7-alpine
```

```
ports: ['6379:6379']
command: redis-server --requirepass suop_dev_only

minio:
  image: minio/minio
  ports: ['9000:9000', '9001:9001']
  environment:
    MINIO_ROOT_USER: suop
    MINIO_ROOT_PASSWORD: suop_dev_only
  command: server /data --console-address ":9001"
  volumes:
    - miniodata:/data

mailhog:
  image: mailhog/mailhog
  ports: ['1025:1025', '8025:8025']

backend:
  build:
    context: ..
    dockerfile: docker/Dockerfile.backend
  ports: ['3030:3030']
  environment:
    DATABASE_URL: postgresql://
      suop:suop_dev_only@postgres:5432/suop
    REDIS_URL: redis://:suop_dev_only@redis:6379
    # ... (other env vars)
  depends_on:
    postgres:
      condition: service_healthy
    redis:
      condition: service_started
  volumes:
    - ../apps/backend/src:/app/apps/backend/src # hot
      reload in dev
  command: bun run dev

web:
```

```
build:
  context: ..
  dockerfile: docker/Dockerfile.web
ports: ['3000:3000']
environment:
  NEXT_PUBLIC_API_URL: http://localhost:3030
depends_on:
  - backend

volumes:
  pgdata:
  miniodata:
```

3.5 Image Scanning

- **Trivy** scans every image build in CI
- **High/critical vulnerabilities** block deploy
- **Base images** updated monthly (`npm run update:base-images`)
- **SBOM** (Software Bill of Materials) generated via `syft` and stored with image

4. CI/CD

4.1 Pipeline Overview

PR Created

↓

[CI Pipeline]

- ├─ Lint (ESLint + Prettier)
- ├─ Type Check (tsc --noEmit)
- ├─ Unit Tests (Vitest)
- ├─ Integration Tests (Testcontainers)
- ├─ SAST Security Scan (SonarQube)
- ├─ Dependency Scan (npm audit + Snyk)
- ├─ Secret Scan (git-secrets)

```
├─ Build (Next.js + Bun)
├─ E2E Tests (Playwright – only on staging deploy)
└─ Coverage Gate (≥70% business, ≥90% foundation)
↓
PR Merged to develop
↓
[CD Staging Pipeline]
├─ Build production images
├─ Push to container registry
├─ Run migrations (with rollback SQL tested)
├─ Deploy to staging
├─ Smoke tests
└─ Notify Slack
↓
PR Merged to main
↓
[CD Production Pipeline – Manual Approval]
├─ Build production images (re-use staging images if
same commit)
├─ Push to container registry
├─ Deploy to production (blue-green)
├─ Run migrations (in maintenance window if destructive)
├─ Smoke tests
├─ Notify Slack + email
└─ Rollback if smoke tests fail
```

4.2 GitHub Actions — CI

`.github/workflows/ci.yml`:

```
name: CI
on:
  pull_request:
    branches: [main, develop]

jobs:
  lint:
```

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4
- uses: oven-sh/setup-bun@v1
- run: bun install --frozen-lockfile
- run: bun run lint
- run: bun run typecheck

unit-tests:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4
 - uses: oven-sh/setup-bun@v1
 - run: bun install --frozen-lockfile
 - run: bun run test:unit --coverage
 - uses: codecov/codecov-action@v4
 - name: Coverage gate
- run: |
- ```
FOUNDATION_COV=$(bun run scripts/read-coverage.ts
core)
BUSINESS_COV=$(bun run scripts/read-coverage.ts
modules)
if ["$FOUNDATION_COV" -lt 90]; then exit 1; fi
if ["$BUSINESS_COV" -lt 70]; then exit 1; fi
```

integration-tests:

runs-on: ubuntu-latest

services:

postgres:

image: postgres:16-alpine

env:

POSTGRES\_PASSWORD: test

ports: ['5432:5432']

options: >-

- health-cmd pg\_isready
- health-interval 5s
- health-retries 5

redis:

```
 image: redis:7-alpine
 ports: ['6379:6379']
steps:
 - uses: actions/checkout@v4
 - uses: oven-sh/setup-bun@v1
 - run: bun install --frozen-lockfile
 - run: bun run test:integration

security-scan:
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v4
 - uses: snyk/actions/node@master
 with:
 command: test
 args: --severity-threshold=high
 env:
 SNYK_TOKEN: ${ secrets.SNYK_TOKEN }
 - uses: github/codeql-action/init@v3
 with:
 languages: javascript-typescript
 - uses: github/codeql-action/analyze@v3

build:
 needs: [lint, unit-tests, integration-tests, security-
 scan]
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v4
 - uses: oven-sh/setup-bun@v1
 - run: bun install --frozen-lockfile
 - run: bun run build
 - uses: actions/upload-artifact@v4
 with:
 name: build-artifacts
 path: |
```

```
apps/web/.next
apps/backend/dist
```

## 4.3 GitHub Actions — CD Staging

---

`.github/workflows/cd-staging.yml`:

```
name: CD Staging
on:
 push:
 branches: [develop]

jobs:
 deploy-staging:
 runs-on: ubuntu-latest
 environment: staging
 steps:
 - uses: actions/checkout@v4
 - name: Build & push images
 run: |
 docker build -f docker/Dockerfile.backend -t suop/
 backend:staging-${{ github.sha }} .
 docker build -f docker/Dockerfile.web -t suop/
 web:staging-${{ github.sha }} .
 docker push suop/backend:staging-${{ github.sha }}
 docker push suop/web:staging-${{ github.sha }}
 - name: Run migrations
 run: bun run prisma migrate deploy
 env:
 DATABASE_URL: ${ secrets.STAGING_DATABASE_URL }
 - name: Deploy to staging
 run: ./scripts/deploy-staging.sh ${ github.sha }
 - name: Smoke tests
 run: bun run test:smoke --env staging
 - name: Notify Slack
 if: always()
 uses: slackapi/slack-github-action@v1
```

```
with:
 slack-message: "Staging deploy ${github.sha}:
${job.status}"
```

## 4.4 GitHub Actions — CD Production

---

.github/workflows/cd-production.yml:

```
name: CD Production
on:
 push:
 branches: [main]

jobs:
 deploy-production:
 runs-on: ubuntu-latest
 environment:
 name: production
 url: https://app.sudhamrit.com
 steps:
 - uses: actions/checkout@v4
 - name: Manual approval gate
 uses: trstringer/manual-approval@v1
 with:
 approvers: ops-lead,security-lead
 minimum-approvals: 2
 issue-title: "Production deploy: $
${github.sha}"
 - name: Run migrations
 run: bun run prisma migrate deploy
 env:
 DATABASE_URL: ${secrets.PROD_DATABASE_URL}
 - name: Deploy (blue-green)
 run: ./scripts/deploy-production.sh $
${github.sha}
 - name: Smoke tests
 run: bun run test:smoke --env production
 - name: Rollback if smoke fails
```

```
 if: failure()
 run: ./scripts/rollback-production.sh
- name: Notify Slack + email
 if: always()
 run: ./scripts/notify-deploy.sh ${ job.status }
```

## 4.5 Pipeline Gates

| Gate                              | Trigger on Fail                              |
|-----------------------------------|----------------------------------------------|
| Lint fails                        | Block merge                                  |
| Type check fails                  | Block merge                                  |
| Unit tests fail                   | Block merge                                  |
| Coverage < threshold              | Block merge                                  |
| Integration tests fail            | Block merge                                  |
| Security scan finds high/critical | Block merge                                  |
| Build fails                       | Block merge                                  |
| Smoke tests fail (staging)        | Alert, don't block production (staging only) |
| Smoke tests fail (production)     | Auto-rollback                                |
| Manual approval pending           | Wait for human                               |

## 5. Git Strategy

### 5.1 Branching Model — GitHub Flow + Develop

```
main (production)
 ↑
develop (integration)
```

```
↑
feature/<scope>-<description> (short-lived)
↑
hotfix/<scope>-<description> (branched from main)
```

## 5.2 Branch Lifetimes

| Branch Type | Lifetime  | Owner             |
|-------------|-----------|-------------------|
| main        | Permanent | Operations        |
| develop     | Permanent | Engineering       |
| feature/*   | <2 weeks  | Feature developer |
| fix/*       | <1 week   | Bug fixer         |
| hotfix/*    | <24 hours | On-call           |
| release/*   | <1 week   | Release manager   |

## 5.3 Branch Naming

- feature/<scope>-<description> — feature/purchase-order-approval-workflow
- fix/<scope>-<description> — fix/stock-ledger-fefo-edge-case
- hotfix/<scope>-<description> — hotfix/auth-jwt-validation-bypass
- chore/<description> — chore/upgrade-prisma-6.12
- docs/<description> — docs/api-standards-update

## 5.4 Commit Conventions (Conventional Commits)

```
<type>(<scope>): <subject>
```

```
<body>
```

<footer>

Types: `feat`, `fix`, `refactor`, `test`, `docs`, `chore`, `perf`, `ci`, `build`

Scope: module name ( `purchase-order`, `auth`, `core-workflow` )

Examples:

```
feat(purchase-order): implement P0 submission workflow
fix(stock-ledger): correct FEFO batch selection edge case
refactor(core-audit): extract audit logger from middleware
test(iqc): add integration tests for fail disposition
docs(architecture): add Phase 0 foundation document
chore(deps): upgrade Prisma to 6.12
ci(cd-production): add manual approval gate
```

## 5.5 PR Standards

---

- **Title:** Conventional commit format
- **Description:** What, Why, How, Testing Done, Screenshots (if UI)
- **Size:** Max 500 lines changed (split larger work)
- **Reviewers:** 1 for code; 2 for security-sensitive (auth, RBAC, crypto)
- **CI:** All checks green
- **No `console.log`** in merged code
- **Squash & merge** to keep history clean

## 5.6 Protected Branches

---

- `main` and `develop` are protected
  - No force pushes ( `git push --force` blocked )
  - No direct commits (must be via PR)
  - Code owners required for specific paths ( `packages/prisma/schema.prisma`, `apps/backend/src/core/` )
-

## 6. Deployment

---

### 6.1 Deployment Strategy — Blue-Green

---

**Production uses blue-green deployment:** - Two identical environments: `blue` (current production) and `green` (next production) - Deploy new version to `green` - Run smoke tests on `green` - Switch load balancer from `blue` → `green` - Keep `blue` running for 1 hour (fast rollback) - After 1 hour, decommission old `blue`; promote `green` to `blue` for next deploy

**Why blue-green:** - Zero-downtime deploys - Instant rollback (switch back to `blue`) - Full smoke test on production-equivalent infra before cutover

### 6.2 Database Migrations in Deploy

---

Migrations are **forward-only** during deploy: 1. Run migration on `green` environment DB (separate from `blue`) 2. If migration succeeds → continue deploy 3. If migration fails → `green` rolled back; `blue` untouched

For destructive migrations (drop column, change type): 1. **Phase 1 release:** Add new column, dual-write to old + new 2. **Phase 2 release:** Migrate data, read from new column 3. **Phase 3 release:** Stop writing to old column (still in schema) 4. **Phase 4 release (1+ month later):** Drop old column

### 6.3 Rollback Strategy

---

**Application rollback:** - Switch load balancer from `green` back to `blue` - Instant (< 30 seconds) - No data loss (DB is shared between blue/green)

**Database rollback:** - Migrations are forward-only — no automatic rollback - Rollback SQL paired with every migration; run manually if needed - For schema changes: rollback migration creates the inverse operation - For data changes: rollback requires data restore from backup (rare; avoided by careful migration design)

## 6.4 Smoke Tests Post-Deploy

---

```
// tests/smoke/smoke.test.ts
describe('Post-deploy smoke tests', () => {
 it('responds to health check', async () => {
 const res = await fetch(`${BASE_URL}/api/v1/_internal/health`)
 expect(res.status).toBe(200)
 })
 it('login works', async () => {
 const res = await fetch(`${BASE_URL}/api/v1/auth/login`, { method: 'POST', body: ... })
 expect(res.status).toBe(200)
 })
 it('can list products', async () => {
 const res = await fetch(`${BASE_URL}/api/v1/products`, {
 headers: { Authorization: `Bearer ${token}` } })
 expect(res.status).toBe(200)
 expect((await res.json()).data.length).toBeGreaterThan(0)
 })
 // 20-30 critical-path smoke tests
})
```

- 20-30 tests covering critical paths
- Runs in <2 minutes
- Failure triggers auto-rollback

## 6.5 Mobile App Deployment

---

- **Mobile:** React Native + Expo
  - **Build:** EAS Build (Expo Application Services)
  - **Distribution:** EAS Submit to App Store + Play Store
  - **Phased rollout:** 10% → 50% → 100% over 7 days
  - **Rollback:** Cannot rollback mobile apps — must ship new version (Phase 0: forced-update capability)
-

## 7. Secrets Management

---

(see Security Architecture §10)

### 7.1 CI/CD Secrets

---

- Stored in GitHub Actions secrets
- Injected as environment variables in workflow
- Never logged (auto-redacted by GitHub)
- Scoped to environment ( staging secrets vs production secrets)

### 7.2 Runtime Secrets

---

- Application reads secrets from secrets manager at boot
- Cached in memory for 5 minutes; refreshed periodically
- Secret rotation does not require app restart (cached value expires; new value fetched)

---

## 8. Infrastructure as Code

---

### 8.1 Terraform

---

All infrastructure defined in Terraform: - VPC, subnets, security groups - RDS Postgres instance - ElastiCache Redis - S3 buckets - ECS cluster (or EKS) - Load balancers, target groups - CloudWatch alarms - Route53 DNS records - ACM certificates

### 8.2 Terraform Layout

---

```
infra/
├─ modules/
│ ├─ vpc/
│ ├─ rds/
│ ├─ redis/
│ └─ s3/
```

```
| ├── ecs/
| └── alb/
├── environments/
| ├── staging/
| | ├── main.tf
| | ├── variables.tf
| | └── terraform.tfvars
| └── production/
| ├── main.tf
| ├── variables.tf
| └── terraform.tfvars
└── README.md
```

## 8.3 Terraform Workflow

---

1. Edit `.tf` files
2. `terraform plan` locally to preview changes
3. PR with plan output in description
4. CI runs `terraform validate` + `terraform plan`
5. Reviewer approves
6. Merge triggers `terraform apply` in CI (with manual approval for production)

## 8.4 State Management

---

- State stored in S3 backend with DynamoDB lock
- State file encrypted (S3 SSE-KMS)
- State contains sensitive outputs (DB endpoint, etc.) — never logged in CI

## 8.5 Drift Detection

---

- Daily job runs `terraform plan` against production
  - If drift detected → alert ops
  - Drift caused by manual changes (forbidden) → investigate + revert
-

## 9. Monitoring

### 9.1 Monitoring Stack

Layer	Tool	Purpose
Metrics	Prometheus + Grafana	Time-series metrics, dashboards, alerting
Logs	Loki (or ELK)	Structured log aggregation
Traces	Tempo (or Jaeger)	Distributed tracing
Errors	Sentry	Error tracking + stack traces
Uptime	Pingdom / UptimeRobot	External uptime monitoring
Synthetic	Checkly	Synthetic transaction monitoring
Real User	Sentry Performance / Datadog RUM	Frontend performance

### 9.2 Metrics (Prometheus)

Exported by backend at `GET /api/v1/_internal/metrics`:-

- `http_requests_total{method, route, status}` — request count -
- `http_request_duration_seconds{method, route}` — histogram -
- `http_requests_in_flight` — gauge -
- `db_connections_active`, `db_connections_idle` -
- `db_query_duration_seconds{operation}` — histogram -
- `redis_operations_total{operation, status}` -
- `workflow_transitions_total{entity, from, to}` -
- `audit_log_writes_total` -
- `notification_delivery_total{channel, status}` -
- `job_duration_seconds{job_name}` -

```
event_bus_subscribers_total{event_name} - cache_hits_total,
cache_misses_total
```

## 9.3 Dashboards (Grafana)

---

- **System Overview:** Request rate, error rate, latency, CPU, memory, disk
- **Database:** Connections, query latency, slow queries, lock waits, cache hit ratio
- **Business:** POs created/day, GRNs posted/day, NCRs opened/day, COAs generated/day
- **Per-module:** Each major module has its own dashboard
- **Real-time:** Live request rate, error rate, current in-flight requests

## 9.4 Alerting Rules

---

```
Critical alerts (PageDuty / phone call)
- alert: HighErrorRate
 expr: rate(http_requests_total{status=~"5.."}[5m]) /
 rate(http_requests_total[5m]) > 0.05
 for: 5m
 annotations:
 summary: "Error rate > 5% for 5 min"

- alert: ServiceDown
 expr: up{job="backend"} == 0
 for: 1m
 annotations:
 summary: "Backend service down"

- alert: DatabaseDown
 expr: pg_up == 0
 for: 30s
 annotations:
 summary: "PostgreSQL down"

Warning alerts (Slack)
```

```
- alert: HighLatency
 expr: histogram_quantile(0.95,
 http_request_duration_seconds_bucket) > 1
 for: 10m
 annotations:
 summary: "p95 latency > 1s for 10 min"

- alert: DbConnectionsHigh
 expr: pg_stat_database_numbackends > 80
 for: 5m
 annotations:
 summary: "DB connections > 80"

- alert: DiskSpaceLow
 expr: 100 - (node_filesystem_avail_bytes /
 node_filesystem_size_bytes) * 100 > 80
 for: 5m
 annotations:
 summary: "Disk > 80% full"
```

---

## 10. Logging

---

### 10.1 Logging Stack

---

Application (Pino) → stdout → Docker → Fluent Bit → Loki → Grafana

### 10.2 Log Levels

---

Level	Use	Volume
<code>fatal</code>	App cannot start / continue	Rare
<code>error</code>	Unhandled errors, 5xx	Moderate

Level	Use	Volume
warn	4xx errors, deprecation warnings, slow ops	Moderate
info	Normal operations (login, CRUD, workflow transitions)	High
debug	Detailed flow (SQL queries, cache hits)	High (dev only)
trace	Function-level tracing	Very high (dev only)

## 10.3 Structured Logging

Every log line is JSON:

```
{
 "level": "info",
 "time": "2026-07-11T14:30:00.000Z",
 "correlationId": "01HXY...",
 "userId": "01HXY...",
 "tenantId": "01HXY...",
 "msg": "Purchase order created",
 "poId": "01HXY...",
 "poNumber": "PO-2026-000001",
 "supplierId": "01HXY...",
 "totalValue": 12500,
 "durationMs": 145
}
```

## 10.4 Sensitive Data Redaction

- Fields matching `/password|token|secret|key|auth|credit|aadhaar|pan/i` redacted

- PII fields (email, phone) hashed for correlation without exposure
- Redaction config in `logging/redaction-config.ts`

## 10.5 Log Retention

---

- Hot (Loki): 30 days
- Warm (S3): 90 days
- Cold (Glacier): 1 year
- Audit logs: see Data Architecture §13

---

# 11. Tracing

---

## 11.1 Distributed Tracing

---

- OpenTelemetry instrumentation on backend
- Every HTTP request creates a span
- DB queries, Redis ops, external calls create child spans
- Spans export to Tempo (or Jaeger)
- Correlation ID propagated via `traceparent` header

## 11.2 Trace Sampling

---

- 100% of error traces
- 10% of normal traces (random sampling)
- 100% of slow traces (>1s)

## 11.3 Use Cases

---

- “Why is this request slow?” → trace shows which DB query / external call is the bottleneck
  - “Why did this fail?” → trace shows error span with stack trace + context
  - “What’s the impact of this service?” → trace heatmaps show dependencies
-

## 12. Alerting

---

### 12.1 Alert Routing

---

Severity	Channel	Response Time
Critical	PagerDuty → phone call	5 min
High	PagerDuty → Slack	30 min
Medium	Slack	4 hours
Low	Daily digest email	24 hours

### 12.2 Alert Definitions

---

**System alerts:** - Service down - High error rate (>5% for 5 min) - High latency (p95 > 1s for 10 min) - DB connections high - Disk space low - Memory high - CPU sustained high

**Business alerts:** - NCR severity CRITICAL created → notify quality head - Recall initiated → notify all stakeholders - CAPA overdue → notify owner + escalation - Stock below reorder → notify procurement - Compliance certificate expiring in 30 days → notify quality

**Security alerts:** - Failed login attempts spike - Permission denied attempts spike - Cross-tenant access attempts - New device login - API key abuse (unusual usage pattern)

### 12.3 Alert Fatigue Prevention

---

- **Grouping:** Similar alerts grouped (e.g., 100 5xx errors in 5 min = 1 alert)
  - **Inhibition:** Critical alert suppresses warning alerts for same service
  - **Silencing:** Maintenance windows silence non-critical alerts
  - **Auto-resolution:** Alert auto-resolves when metric returns to normal
  - **Weekly review:** On-call reviews alert volume; tunes noise down
-

## 13. Environment Configuration

### 13.1 Configuration Sources

1. **Environment variables** (highest priority) — for secrets + deployment-specific
2. **.env file** (local dev only) — gitignored
3. **Database** — feature flags, per-tenant config
4. **Default values** (lowest priority) — in code

### 13.2 Environment Variable Catalog

Variable	Required	Description	Example
<code>NODE_ENV</code>	Yes	<code>development</code> / <code>staging</code> / <code>production</code>	<code>production</code>
<code>PORT</code>	Yes	Backend port	<code>3030</code>
<code>DATABASE_URL</code>	Yes	Postgres connection string	<code>postgresql://...</code>
<code>DATABASE_POOL_SIZE</code>	No	Prisma pool size	<code>10</code>
<code>REDIS_URL</code>	Yes	Redis connection string	<code>redis://...</code>
<code>JWT_SECRET</code>	Yes	JWT signing key	(32+ chars random)
<code>JWT_ISSUER</code>	No	JWT issuer	<code>suop-erp</code>
<code>S3_ENDPOINT</code>	Yes	S3 / MinIO endpoint	<code>https://s3.ap-south-1.amazonaws.com/...</code>

Variable	Required	Description	Example
<code>S3_BUCKET</code>	Yes	S3 bucket name	<code>suop-prod</code>
<code>S3_ACCESS_KEY</code>	Yes	S3 access key	<code>AKIA...</code>
<code>S3_SECRET_KEY</code>	Yes	S3 secret key	(secret)
<code>SMTP_HOST</code>	Yes	SMTP server	<code>smtp.sendgrid.net</code>
<code>SMTP_PORT</code>	Yes	SMTP port	<code>587</code>
<code>SMTP_USER</code>	Yes	SMTP username	<code>apikey</code>
<code>SMTP_PASS</code>	Yes	SMTP password	(secret)
<code>SENTRY_DSN</code>	No	Sentry DSN	<code>https://...</code>
<code>TWILIO_ACCOUNT_SID</code>	No	Twilio SID	<code>AC...</code>
<code>TWILIO_AUTH_TOKEN</code>	No	Twilio token	(secret)
<code>NEXT_PUBLIC_API_URL</code>	Yes	Frontend API URL	<code>https://api.sudhamrit.com</code>
<code>NEXT_PUBLIC_SUPABASE_URL</code>	No	Supabase URL	<code>https://...supabase.com</code>
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	No	Supabase anon key	<code>eyJ...</code>

### 13.3 Environment Validation

All env vars validated at boot via zod:

```
const envSchema = z.object({
 NODE_ENV: z.enum(['development', 'staging', 'production']),
 // ...
})
```

```
})
export const env = envSchema.parse(process.env)
```

App refuses to start if validation fails — fail fast.

---

## 14. Backup & Recovery

---

(see Disaster Recovery document for full details)

### 14.1 DevOps Responsibilities

---

- Backup scripts in `infra/scripts/backup/`
  - Restore scripts in `infra/scripts/restore/`
  - Daily restore-test job
  - Monthly DR drill
  - Annual full disaster simulation
- 

## 15. Open Questions for Approval

---

#	Decision	Recommendation	Alternatives
Q-DV1	Container orchestration	AWS ECS Fargate	Kubernetes (EKS)
Q-DV2	Blue-green vs canary	Blue-green	Canary
Q-DV3	Deploy frequency	Twice weekly	Daily, weekly
Q-DV4	IaC tool	Terraform	Pulumi, CloudFormation
Q-DV5	Monitoring stack	Prometheus + Grafana + Loki + Tempo	Datadog (all-in-one)

#	Decision	Recommendation	Alternatives
Q-DV6	Error tracking	Sentry	Rollbar, Datadog
Q-DV7	Alert routing	PagerDuty + Slack	OpsGenie
Q-DV8	Image registry	AWS ECR	Docker Hub, GitHub Container Registry
Q-DV9	CI/CD platform	GitHub Actions	GitLab CI, CircleCI
Q-DV10	Mobile deploy	EAS (Expo)	Fastlane + manual

---

## Approval Block

---

**Approved by:** \_\_\_\_\_ **Date:** \_\_\_\_\_

End of Enterprise DevOps Architecture Document

# 05\_PERFORMANCE\_STANDARDS

---

- [SUOP ERP v1.0 — Enterprise Performance Standards](#)
  - [1. Purpose](#)
  - [2. Response Time Targets](#)
  - [3. Database Performance](#)
  - [4. API Performance](#)
  - [5. Frontend Performance](#)
  - [6. Caching Strategy](#)
  - [7. Redis Strategy](#)
  - [8. CDN Strategy](#)
  - [9. Load Testing](#)
  - [10. Stress Testing](#)
  - [11. Capacity Planning](#)
  - [12. Frontend Performance Monitoring](#)
  - [13. Database Performance Monitoring](#)
  - [14. Performance Regression Testing](#)
  - [15. Open Questions for Approval](#)
  - [Approval Block](#)

## SUOP ERP v1.0 — Enterprise Performance Standards

---

Field	Value
Document Version	1.0
Status	DRAFT — Awaiting Approval
Date	2026-07-11
Approval Required	Project Owner

---

# 1. Purpose

This document defines performance standards for every layer of SUOP ERP v1.0. Every endpoint, every database query, every frontend page, and every background job must meet these standards. Performance is a feature — slow ERP = unusable ERP.

**Hard rule:** No endpoint ships without meeting its performance target. CI runs performance regression tests on every PR.

## 2. Response Time Targets

### 2.1 API Response Time Targets

Endpoint Type	p50	p95	p99	Hard Limit
<b>Single record GET</b> ( /products/{id} )	50ms	200ms	500ms	2s
<b>List GET</b> ( /products? page=1&pageSize=25 )	100ms	400ms	1s	3s
<b>Filtered/aggregated list</b> (dashboards)	200ms	800ms	2s	5s
<b>Simple POST</b> ( / products create)	100ms	400ms	1s	3s
<b>Complex POST</b> (PO create with 50 lines)	300ms	1s	2s	5s
<b>Workflow transition</b> ( /pos/{id}/ approve )	200ms	800ms	2s	5s
	500ms	2s	5s	15s

Endpoint Type	p50	p95	p99	Hard Limit
<b>Bulk operations</b> (100 items)				
<b>Long-running async</b> (COA generation, recall)	200ms (return job ID)	500ms	1s	2s
<b>Export to CSV</b> (async)	200ms (return job ID)	500ms	1s	2s
<b>Search</b> (full-text)	200ms	800ms	2s	5s
<b>Audit log query</b>	200ms	800ms	2s	5s
<b>Health check</b>	5ms	20ms	50ms	100ms
<b>Reference data GET</b>	10ms	30ms	50ms	200ms (cached)

## 2.2 Frontend Performance Targets

Metric	Target	Measurement
<b>Largest Contentful Paint (LCP)</b>	<2.5s	Lighthouse on production
<b>First Input Delay (FID)</b>	<100ms	Lighthouse
<b>Cumulative Layout Shift (CLS)</b>	<0.1	Lighthouse
<b>Time to First Byte (TTFB)</b>	<600ms	Lighthouse
<b>Total Blocking Time (TBT)</b>	<300ms	Lighthouse

Metric	Target	Measurement
<b>First Contentful Paint (FCP)</b>	<1.8s	Lighthouse
<b>Lighthouse Performance Score</b>	≥90	Lighthouse
<b>Page transition (SPA)</b>	<300ms	Custom
<b>API call to render</b>	<500ms	Custom
<b>Bundle size (initial)</b>	<300KB gzipped	webpack-bundle-analyzer
<b>Bundle size (per route)</b>	<100KB gzipped	Per-route code splitting

## 2.3 Database Targets

Operation	Target	Hard Limit
<b>Single row lookup by PK</b>	<5ms	<50ms
<b>Indexed range scan (1000 rows)</b>	<20ms	<200ms
<b>Filtered list (25 rows from 100k)</b>	<50ms	<500ms
<b>Filtered list (25 rows from 1M)</b>	<100ms	<500ms
<b>Join query (3 tables)</b>	<50ms	<500ms
<b>Aggregation (COUNT, SUM)</b>	<100ms	<1s
	<500ms	<2s

Operation	Target	Hard Limit
Complex dashboard query		
Full-text search	<200ms	<1s
Bulk insert (500 rows)	<500ms	<2s
Bulk update (500 rows)	<1s	<5s
Migration (small)	<1s	<30s
Migration (large, with backfill)	<5 min	<30 min (zero-downtime required)

## 2.4 Background Job Targets

Job Type	Target Duration
Notification dispatch (single)	<2s
Audit log archival (10k records)	<5 min
Dashboard snapshot computation	<30s
Stock reorder check (full scan)	<2 min
Backup verification	<10 min
Mock recall drill	<60s (recall identification: <5s per acceptance criteria)
COA generation (single)	<3s per acceptance criteria
Compliance dashboard load	<2s per acceptance criteria

## 2.5 Real-Time Update Targets

Operation	Target
Notification delivery (in-app, polling)	<30s (poll interval)
Notification delivery (future: WebSocket)	<2s
Audit log visibility after action	<1s (same request)
Workflow state update propagation	<1s (same request)
Stock balance update after GRN post	<1s (same transaction)

## 3. Database Performance

### 3.1 Index Strategy (see Data Architecture §6)

- Every FK indexed
- Every query path indexed (verified via EXPLAIN ANALYZE in CI)
- Composite indexes for filter+sort patterns
- Partial indexes for soft-deleted tables
- Covering indexes ( INCLUDE ) for hot queries

### 3.2 Query Performance

- **N+1 prevention:** Prisma `include` for eager loading, DataLoader for batch loading
- **No `SELECT *` in production** (Prisma handles by default; raw queries must specify columns)
- **Cursor pagination for >100k row tables** (offset pagination slow at high offsets)
- **Pagination limit: max 200 rows per page** (prevents memory pressure)

### 3.3 Connection Pooling

---

- PgBouncer (transaction mode) between app and Postgres
- Pool size: 20-50 per app instance (tuned per workload)
- Connection lifecycle: idle timeout 30s, max lifetime 5 min
- Health check: pool utilization alarm at 80%

### 3.4 Slow Query Detection

---

- `log_min_duration_statement = 500ms` — logs slow queries
- Daily report of top 10 slowest queries
- Each query added to optimization backlog
- EXPLAIN ANALYZE auto-run on slow queries; plan stored

### 3.5 Query Plan Review

---

- Weekly job samples `pg_stat_statements` for top 20 expensive queries
- EXPLAIN ANALYZE plan stored
- Seq scans on tables >10k rows flagged
- Optimization PRs created from backlog

### 3.6 Table Maintenance

---

- **VACUUM:** Auto-vacuum tuned per table; manual `VACUUM ANALYZE` weekly on hot tables
- **ANALYZE:** After large data loads; ensures planner statistics current
- **REINDEX:** Monthly on high-write tables (using `REINDEX CONCURRENTLY` — no lock)
- **Bloat monitoring:** `pgstattuple` monthly; bloat >30% triggers REINDEX

### 3.7 Partitioning (see Data Architecture §7)

---

- Large tables partitioned by month
  - Old partitions dropped (instant) instead of DELETE
  - `pg_partman` for automatic partition management
-

## 4. API Performance

---

### 4.1 Response Caching

---

**Cache layers:** 1. **HTTP cache (CDN):** Static assets, reference data (1 hour TTL) 2. **Application cache (Redis):** Reference data, dashboard aggregations (5 min TTL) 3. **Database cache:** Postgres shared buffers (tuned by `shared_buffers`)

**What to cache:** - Reference data (countries, currencies, UOMs) — TTL 1 hour - Dashboard aggregations — TTL 5 min (stale-while-revalidate) - User permissions — TTL 60s - Product/supplier master data — TTL 5 min - Static assets — TTL 1 year (with hash-based filenames)

**What NOT to cache:** - Real-time data (stock balances, audit logs) - User-specific data (notifications, tasks) - Sensitive data (financial, PII)

### 4.2 ETag and Conditional Requests

---

- All GET responses include `ETag` (hash of response body)
- Clients send `If-None-Match: <etag>` on subsequent requests
- Server returns `304 Not Modified` if unchanged (no body — saves bandwidth)
- Reduces load for repeated polling (dashboards, notification checks)

### 4.3 Compression

---

- **gzip** for responses >1KB
- **Brotli** (preferred) for browsers that support it
- Disabled for already-compressed content (images, PDFs, videos)
- Content-Type whitelist: `application/json`, `text/html`, `text/css`, `application/javascript`

### 4.4 Request Batching

---

- Frontend batches independent API calls via `Promise.all`
- Backend does not support batch endpoints for unrelated resources (REST principles)

- Bulk endpoints exist for related bulk operations (bulk create, bulk update)

## 4.5 Idempotency Cache

---

- Idempotency keys stored in Redis (24h TTL) instead of DB
  - Repeat requests served from cache (<5ms response)
  - Reduces DB load for retry storms (client retrying on network error)
- 

# 5. Frontend Performance

---

## 5.1 Bundle Size Budgets

---

```
// apps/web/budgets.config.js
module.exports = {
 bundle: {
 'initial': { max: 300 }, // KB gzipped
 'per-route': { max: 100 }, // KB gzipped
 'vendor': { max: 200 }, // KB gzipped
 },
}
```

- CI fails if bundle exceeds budget
- Bundle analyzed via `webpack-bundle-analyzer` in CI
- Tree-shaking verified for unused exports

## 5.2 Code Splitting

---

- **Route-level:** Each route lazy-loaded ( `dynamic(() => import(...))` )
- **Component-level:** Heavy components (data tables, charts) lazy-loaded
- **Library-level:** Large libs ( `chart.js` , `moment` replaced with `date-fns` ) code-split
- **Vendor-level:** React, React-DOM in vendor chunk (long-term cache)

## 5.3 Image Optimization

---

- **Next.js Image component** for all images
- **WebP/AVIF** format (smaller than JPEG/PNG)
- **Responsive srcset** for different screen sizes
- **Lazy loading** for below-the-fold images
- **Blur placeholder** for above-the-fold images

## 5.4 Font Loading

---

- **Subset fonts** (Latin + Devanagari only)
- **Preload critical fonts** ( `<link rel="preload">` )
- **Font-display: swap** (text visible during font load)
- **Self-host fonts** (no Google Fonts request — privacy + perf)

## 5.5 React Performance

---

- `useMemo` / `useCallback` for expensive computations and stable references
- `React.memo` for pure components that re-render unnecessarily
- **Virtual scrolling** for long lists (>100 items) via `@tanstack/react-virtual`
- **Debounced inputs** for search boxes (300ms debounce)
- **Optimistic updates** for mutations (UI updates before API response)

## 5.6 React Query Performance

---

- **Stale time:** 30s for transactional data, 5 min for master data, 1 hour for reference data
- **Cache time:** 5 min (garbage collected if not used)
- **Refetch on focus:** Yes (for transactional data), No (for reference data)
- **Refetch interval:** 30s for notifications, 5 min for dashboards
- **Pagination:** `useInfiniteQuery` for infinite scroll; prefetch next page
- **Optimistic updates:** `onMutate` rolls back on error

## 5.7 SSR vs CSR

---

- **SSR (Server-Side Rendering):** Public pages (login, marketing), SEO-required pages
  - **SSG (Static Site Generation):** Reference data pages, documentation
  - **CSR (Client-Side Rendering):** Authenticated dashboard pages (data is user-specific)
  - **ISR (Incremental Static Regeneration):** Help articles, changelog (revalidate every hour)
- 

## 6. Caching Strategy

---

### 6.1 Cache Layers

---

Browser Cache (HTTP) → CDN Cache → App Cache (Redis) → DB Cache (Postgres shared buffers) → DB

### 6.2 Cache Invalidation Patterns

---

**TTL-based (time):** - Reference data: 1 hour - Master data: 5 min - Dashboard aggregations: 5 min (stale-while-revalidate) - User permissions: 60s

**Event-based (write invalidates cache):** - Product updated → invalidate `product:{id}`, `products:list` - Supplier updated → invalidate `supplier:{id}`, `suppliers:list` - Permission changed → invalidate `user:{id}:permissions`

**Version-based:** - Reference data updates bump version; clients refetch

### 6.3 Cache-Aside Pattern

---

```
async function getProduct(id: string): Promise<Product> {
 const cacheKey = `product:${id}`
```

```
// Try cache
const cached = await redis.get(cacheKey)
if (cached) return JSON.parse(cached)
// Cache miss → DB
const product = await prisma.product.findUnique({ where:
 { id } })
if (!product) throw new NotFoundError()
// Cache for 5 min
await redis.setex(cacheKey, 300, JSON.stringify(product))
return product
}
```

## 6.4 Cache Stampede Prevention

---

- **Probabilistic early expiration:** Refresh cache before TTL expires (5% chance per request in last 10% of TTL)
- **Lock-based:** Only one request refreshes cache; others wait
- **Single-flight:** `Promise` shared across concurrent requests for same key

## 6.5 Cache Warming

---

- On deploy, warm cache for hot endpoints (reference data, top 100 products, top 100 suppliers)
- Background job every 5 min refreshes dashboard aggregations
- Reduces cold-start latency

---

# 7. Redis Strategy

---

## 7.1 Redis Use Cases

---

Use Case	Data Type	TTL	Memory Impact
<b>Application cache</b> (objects)	String (JSON)	5 min - 1 hour	High

Use Case	Data Type	TTL	Memory Impact
<b>Session cache</b> (user permissions)	String (JSON)	60s	Medium
<b>Rate limiting counters</b>	String (int)	60s	Low
<b>Idempotency keys</b>	String (JSON)	24 hour	Medium
<b>JWT blocklist</b>	String	Token TTL	Low
<b>Distributed locks</b>	String + NX	30s	Low
<b>Job queue</b> (BullMQ)	List + Hash	Persistent	Medium
<b>Pub/Sub</b> (future: WebSocket)	Pub/Sub	—	Low
<b>Leaderboards</b> (future: gamification)	Sorted Set	Persistent	Low

## 7.2 Redis Configuration

- **Maxmemory policy:** `allkeys-lru` (cache mode — old keys evicted)
- **Persistence:** RDB snapshot every 5 min + AOF (append-only file) for durability
- **Replication:** Single replica for read scaling (future)
- **TLS:** Required in production
- **Auth:** ACL-based (separate users for cache vs job queue)

## 7.3 Memory Management

---

- **Max memory:** 2GB per instance (sufficient for cache + queues)
- **Eviction policy:** LRU (least recently used)
- **Monitoring:** Memory usage, eviction rate, hit rate
- **Alert:** Memory >80% → scale up; >95% → critical

## 7.4 Cache Hit Rate Target

---

- **Reference data:** >99% (very stable, long TTL)
- **Master data:** >95% (5 min TTL, frequent reads)
- **Dashboard aggregations:** >90% (5 min TTL, periodic refresh)
- **User permissions:** >85% (60s TTL, periodic refresh)

Hit rate <80% triggers review (cache key strategy, TTL tuning).

---

# 8. CDN Strategy

---

## 8.1 CDN Use

---

- **Static assets:** JS, CSS, images, fonts (1 year cache, hash-based filenames)
- **Reference data API:** Cached at edge (1 hour TTL)
- **Public pages:** Login, marketing, documentation
- **Not cached:** Authenticated API responses, user-specific data

## 8.2 CDN Provider

---

- **Cloudflare** (default) — generous free tier, global PoPs, includes DDoS protection
- **Alternative:** AWS CloudFront (deeper AWS integration)

## 8.3 CDN Configuration

---

- **Cache key:** URL + Accept-Encoding (ignore query params for static assets)

- **Cache behavior:** Cache on GET/HEAD only; bypass for POST/PUT/DELETE
- **TTL:** 1 year for static assets with hash filenames; 1 hour for reference API
- **Purge:** API-driven purge on reference data updates
- **Compression:** Brotli (preferred) or gzip
- **Image optimization:** Cloudflare Images or Polish (resize, convert to WebP)

## 8.4 Cache Invalidation

---

- Hash-based filenames auto-invalidate on content change
  - Reference data updates trigger programmatic purge via CDN API
  - Full purge only for emergency (rare; affects all users)
- 

# 9. Load Testing

---

## 9.1 Load Testing Tool

---

- **k6** (Grafana Labs) — scriptable in JavaScript, integrates with Grafana
- **Alternative:** Locust (Python), JMeter (heavy)

## 9.2 Load Test Scenarios

---

**Scenario 1 — Normal Load:** - 100 concurrent users - Read-heavy (90% GET, 10% POST) - Run for 10 min - Pass criteria: p95 <400ms, error rate <0.1%

**Scenario 2 — Peak Load:** - 500 concurrent users - Mix: 80% GET, 15% POST, 5% PATCH - Run for 30 min - Pass criteria: p95 <1s, error rate <1%

**Scenario 3 — Stress Test:** - Ramp from 100 → 2000 concurrent users over 30 min - Find breaking point - Document max throughput + latency at saturation

**Scenario 4 — Spike Test:** - 100 users baseline → 1000 users spike for 1 min → back to 100 - Verify auto-scaling responds - Verify no error storm

**Scenario 5 — Soak Test:** - 200 concurrent users for 8 hours - Detect memory leaks, connection leaks - Verify stable performance over time

### 9.3 Load Test Targets

Endpoint	Target Throughput	Target p95
GET /products (list)	500 req/s	<400ms
GET /products/{id}	1000 req/s	<200ms
POST /products	50 req/s	<400ms
GET /audit-logs (cursor)	200 req/s	<800ms
POST /purchase-orders (50 lines)	10 req/s	<1s
POST /auth/login	50 req/s	<500ms

### 9.4 Load Test Schedule

- **Every PR:** Smoke load test (50 users, 1 min)
- **Nightly on staging:** Peak load test (500 users, 30 min)
- **Weekly:** Stress test to find breaking point
- **Monthly:** Soak test (8 hours)
- **Quarterly:** Full load test suite + capacity review

## 10. Stress Testing

### 10.1 Stress Test Goals

- Find system breaking point
- Identify bottlenecks (CPU, memory, DB, Redis, network)
- Verify graceful degradation (not crash)
- Verify auto-scaling responds
- Document max capacity for capacity planning

## 10.2 Stress Test Procedure

---

1. Start with normal load (100 users)
2. Ramp +100 users every 5 min
3. Monitor: latency, error rate, CPU, memory, DB connections, Redis memory
4. Continue until error rate >5% or latency p95 >5s
5. Document breaking point + bottleneck
6. Scale bottleneck + retest

## 10.3 Graceful Degradation

---

When overloaded, system should: - Return 503 Service Unavailable (not crash) - Maintain functionality for authenticated users (rate limit unauthenticated) - Shed load via rate limiting (429) before running out of resources - Not corrupt data (transactions either complete or roll back)

---

# 11. Capacity Planning

---

## 11.1 Capacity Metrics

---

Resource	Current Utilization	Headroom	Scale Trigger
<b>CPU</b> (backend)	<50%	2x	>70% sustained 5 min → scale up
<b>Memory</b> (backend)	<60%	1.5x	>80% sustained 5 min → scale up
<b>DB connections</b>	<50%	2x	>70% → add app instances or increase pool
<b>DB CPU</b>	<50%	2x	>70% → scale up DB instance

Resource	Current Utilization	Headroom	Scale Trigger
<b>DB storage</b>	<70%	1.5x	>80% → provision more storage
<b>Redis memory</b>	<50%	2x	>70% → scale up or evict more
<b>Redis CPU</b>	<50%	2x	>70% → scale up or read replica

## 11.2 Growth Assumptions

Metric	Current (Year 1)	Year 2	Year 3
Tenants	5	20	50
Users per tenant	50	100	200
Total users	250	2,000	10,000
Transactions/day	10k	50k	200k
Audit logs/month	100k	500k	2M
DB size	5 GB	25 GB	100 GB
File storage	10 GB	50 GB	200 GB

## 11.3 Scaling Strategy

**Vertical scaling (bigger instance):** - DB: Scale up to larger RDS instance (CPU + memory) - Redis: Scale up to larger ElastiCache instance - App: Scale up to larger ECS task (more CPU + memory)

**Horizontal scaling (more instances):** - App: Add ECS tasks behind ALB (stateless — easy) - DB: Read replica for read-heavy workloads (reports, dashboards) - Redis: Read replica for cache (future)

**When to scale:** - CPU >70% sustained → scale horizontally (app) or vertically (DB) - Memory >80% sustained → scale vertically - DB connections >70% → scale horizontally (app) with PgBouncer - Storage >80% → provision more (automatic with managed services)

## 11.4 Auto-Scaling Rules

---

**Backend (ECS):** - Scale out: CPU >70% for 5 min (add 1 task) - Scale in: CPU <30% for 15 min (remove 1 task) - Min: 2 tasks (HA) - Max: 10 tasks (cost control)

**Database (RDS):** - Manual scaling only (vertical) — auto-scaling not recommended for DBs - Alert at 70% CPU; ops reviews weekly

**Redis (ElastiCache):** - Manual scaling only - Alert at 70% memory

---

## 12. Frontend Performance Monitoring

---

### 12.1 Real User Monitoring (RUM)

---

- **Sentry Performance** (or Datadog RUM) on production frontend
- Captures: page load time, route transitions, API call duration, errors
- Aggregated by: route, browser, country, device
- Alerts on regression: LCP >4s for 5% of users

### 12.2 Synthetic Monitoring

---

- **Checkly** runs synthetic transactions every 5 min:
  - Login → dashboard load → create PO → logout
  - Login → NCR dashboard → create NCR → logout
  - 10 critical paths monitored
- Alerts on failure or slowdown

## 12.3 Lighthouse CI

---

- Lighthouse runs on every PR (against preview deployment)
- Performance score <80 blocks merge
- Lighthouse runs weekly on production
- Trends tracked over time

---

## 13. Database Performance Monitoring

---

### 13.1 PostgreSQL Metrics

---

- **Connections:** active, idle, waiting
- **Queries per second** by type
- **Query latency** p50, p95, p99
- **Transaction duration** p50, p95, p99
- **Lock waits** (count + duration)
- **Cache hit ratio** (`blks_hit / (blks_hit + blks_read)`)
- **Index usage** (`pg_stat_user_indexes`)
- **Table bloat** (`pgstattuple`)
- **Slow queries** (>500ms logged)
- **Deadlocks**
- **Replication lag** (if replica exists)

### 13.2 Query Performance

---

- `pg_stat_statements` extension enabled
- Weekly report: top 20 queries by total time, by mean time, by calls
- Slow queries added to optimization backlog
- EXPLAIN ANALYZE plans stored for review

### 13.3 Index Maintenance

---

- Monthly: `pg_stat_user_indexes` review
  - Unused indexes (`idx_scan = 0` over 30 days) → candidates for removal
  - Bloat >30% → `REINDEX CONCURRENTLY`
  - New indexes verified via EXPLAIN ANALYZE in CI
-

## 14. Performance Regression Testing

---

### 14.1 Regression Test Suite

---

- 50 critical endpoints with target latency
- Run on every PR (smoke — 5 endpoints)
- Run nightly on staging (full — 50 endpoints)
- Run weekly on production-equivalent environment (full)

### 14.2 Regression Detection

---

- Latency >20% slower than baseline → fail
- Throughput >20% lower than baseline → fail
- Baseline updated monthly (intentional updates documented)

### 14.3 Performance Budget Enforcement

---

- Bundle size budget per route (CI fails if exceeded)
- Database query count budget per endpoint (CI fails if exceeded)
- Memory usage budget per endpoint (CI fails if exceeded)

---

## 15. Open Questions for Approval

---

#	Decision	Recommendation	Alternatives
Q-P1	API p95 target for list endpoints	400ms	200ms (more strict), 800ms (lenient)
Q-P2	Lighthouse performance score	≥90	≥80, ≥95
Q-P3	Bundle size budget (initial)	300KB gzipped	200KB, 500KB
Q-P4		5 min	1 min, 15 min

#	Decision	Recommendation	Alternatives
	Cache TTL for master data		
Q-P5	Load test frequency	Nightly on staging	Weekly
Q-P6	Stress test frequency	Weekly	Monthly
Q-P7	Auto-scaling for backend	Yes (ECS)	Manual scaling
Q-P8	RUM tool	Sentry Performance	Datadog RUM
Q-P9	Synthetic monitoring	Checkly	Pingdom
Q-P10	Performance regression threshold	20% slower	10%, 30%

---

## Approval Block

---

**Approved by:** \_\_\_\_\_ **Date:** \_\_\_\_\_

End of Enterprise Performance Standards Document

# 06\_DISASTER\_RECOVERY

---

- [SUOP ERP v1.0 — Enterprise Disaster Recovery](#)
  - [1. Purpose](#)
  - [2. Disaster Recovery Objectives](#)
  - [3. Backup Strategy \(Comprehensive\)](#)
  - [4. Recovery Procedures](#)
  - [5. Business Continuity Plan \(BCP\)](#)
  - [6. High Availability](#)
  - [7. Database Recovery](#)
  - [8. File Recovery](#)
  - [9. Cyberattack Recovery](#)
  - [10. DR Testing](#)
  - [11. DR Roles and Responsibilities](#)
  - [12. DR Communication](#)
  - [13. DR Runbooks](#)
  - [14. DR Budget and Resources](#)
  - [15. Open Questions for Approval](#)
  - [Approval Block](#)

## SUOP ERP v1.0 — Enterprise Disaster Recovery

---

Field	Value
Document Version	1.0
Status	DRAFT — Awaiting Approval
Date	2026-07-11
Approval Required	Project Owner

Field	Value
Compliance	ISO 22301 (Business Continuity), FSSAI Cybersecurity

---

# 1. Purpose

---

This document defines how SUOP ERP v1.0 prevents, survives, and recovers from disasters. A “disaster” is any event that disrupts normal operation: hardware failure, region outage, cyberattack, data corruption, human error, or natural disaster.

**Hard rule:** Every recovery procedure in this document must be tested at least annually. Untested recovery plans are wishful thinking, not recovery plans.

---

# 2. Disaster Recovery Objectives

---

## 2.1 RPO and RTO Targets

---

Scenario	RPO (Data Loss)	RTO (Downtime)
Single-row accidental delete	0 (recoverable from PITR)	1 hour
Single-table accidental drop	0 (recoverable from PITR)	2 hours
Logical corruption (bad migration)	Up to 24 hours	4 hours
Application bug (data overwrite)	Up to 1 hour	2 hours
Database instance failure	0 (replica promotion)	30 minutes

Scenario	RPO (Data Loss)	RTO (Downtime)
Availability zone failure	0 (multi-AZ)	5 minutes
Region failure	Up to 1 minute (async replication)	30 minutes
Ransomware / cyberattack	Up to 24 hours (last clean backup)	24 hours
Total system loss (catastrophic)	Up to 24 hours (Glacier restore)	72 hours

## 2.2 Definitions

- **RPO (Recovery Point Objective):** Maximum acceptable data loss measured in time. “RPO of 1 hour” means at most 1 hour of data can be lost.
- **RTO (Recovery Time Objective):** Maximum acceptable downtime. “RTO of 4 hours” means system must be back online within 4 hours of disaster declaration.
- **DR (Disaster Recovery):** The process of restoring service after a disaster.
- **BCP (Business Continuity Plan):** The broader plan for keeping business running during disruption (including manual workarounds).

## 2.3 DR Tier Classification

Tier	Description	RTO	RPO	Cost
<b>Tier 1 (Critical)</b>	Always-on, zero data loss	<5 min	0	High
<b>Tier 2 (Important)</b>	Quick recovery, minimal loss	<1 hour	<15 min	Medium

Tier	Description	RTO	RPO	Cost
<b>Tier 3 (Standard)</b>	Hours of recovery acceptable	<8 hours	<24 hours	Low

**SUOP components by tier:** - **Tier 1:** Auth service, audit log (cannot lose audit data) - **Tier 2:** Backend API, database (transactional), Redis (cache) - **Tier 3:** File storage, search index (rebuildable)

---

## 3. Backup Strategy (Comprehensive)

---

### 3.1 Backup Layers

---

**Layer 1 — Managed Automated Backups (Supabase/RDS):** - **Frequency:** Continuous (WAL streaming) - **Retention:** 14 days - **Capability:** Point-in-time recovery (PITR) to any second in last 14 days - **Cross-region:** Automated backup replication to DR region - **Encryption:** At rest (KMS-managed keys)

**Layer 2 — Logical Backups (pg\_dump):** - **Frequency:** Daily (3 AM UTC — off-peak) - **Retention:** 30 days daily, 12 months weekly, 7 years monthly (compliance) - **Storage:** S3 with SSE-KMS encryption - **Verification:** Daily restore-test job (restores to isolated DB, runs integrity checks) - **Format:** Custom format ( `-Fc` ) with compression

**Layer 3 — File Storage Backups:** - **S3 versioning:** Enabled on all buckets (recoverable from accidental delete) - **Cross-region replication:** All objects replicated to DR region - **Lifecycle:** Standard (3 years) → Standard-IA (3-7 years) → Glacier (7+ years) - **MinIO (dev):** Weekly `mc mirror` to local NAS

**Layer 4 — Configuration Backups:** - **Terraform state:** S3 backend with versioning - **Secrets:** Backed up via secrets manager export (encrypted) — quarterly - **GitHub:** Repo mirrored to Bitbucket (or AWS CodeCommit) weekly

### 3.2 Backup Encryption

---

- All backups encrypted at rest (S3 SSE-KMS or pg\_dump --encrypt )
- Encryption keys in KMS, separate from data
- Decryption keys in secrets manager, different region from backups
- Key rotation: annually
- Key access: audited

### 3.3 Backup Integrity

---

- **Daily:** Restore-test job restores latest logical backup to isolated DB
  - Runs pg\_checksums for physical integrity
  - Runs sample query suite (counts on key tables, sample row validation)
  - Alerts if any check fails
- **Weekly:** Full restore to staging environment
  - Runs full E2E test suite against restored data
  - Validates data consistency (row counts match production)
- **Monthly:** DR failover drill
  - Promote cross-region replica
  - Run smoke tests
  - Failback
- **Annual:** Full disaster simulation
  - Simulate region loss
  - Restore from Glacier backup
  - Document actual RTO and RPO

### 3.4 Backup Retention Policy

---

Backup Type	Retention	Storage	Purpose
PITR WAL	14 days	Supabase managed	Recovery to any second
Daily logical	30 days	S3 Standard	Quick restore for accidental delete

Backup Type	Retention	Storage	Purpose
Weekly logical	12 months	S3 Standard-IA	Mid-term recovery
Monthly logical	7 years	S3 Glacier	Compliance (FSSAI, GST, audit)
File storage	7 years	S3 Glacier	Compliance
Terraform state	Indefinite	S3 versioned	Infrastructure recovery
Secrets	1 year	Secrets manager	Secret recovery

---

## 4. Recovery Procedures

---

### 4.1 Recovery Scenario Catalog

#	Scenario	Trigger	Method	RTO
R1	Accidental row delete (within 14 days)	User report	PITR → extract row → insert back	1 hour
R2	Accidental table drop (within 14 days)	User report	PITR → restore table	2 hours
R3	Logical corruption (bad migration)	Failed deploy	Restore from nightly logical backup	4 hours

#	Scenario	Trigger	Method	RTO
R4	Application bug (data overwrite)	User report	PITR → extract correct data → restore	2 hours
R5	Database instance failure	Monitoring alert	Promote read replica	30 minutes
R6	Availability zone failure	Cloud provider	Auto-failover to standby AZ	5 minutes
R7	Region failure	Cloud provider	Promote cross-region replica	30 minutes
R8	Ransomware / cyberattack	Security team	Restore from last clean backup (offline)	24 hours
R9	Total system loss	Catastrophic event	Restore from Glacier monthly backup	72 hours
R10	File storage loss	S3 outage / corruption	Cross-region replica promotion	1 hour
R11	Redis loss	Redis outage	Rebuild from DB (cache is disposable)	15 minutes
R12	Configuration drift	Manual change detection	Terraform apply (idempotent)	30 minutes

## 4.2 Recovery Procedure — Logical Backup (R3)

---

**Trigger:** Failed deploy, corrupted data, bad migration

**Steps:** 1. Declare incident (Slack `#incidents` channel, page on-call) 2. Stop application writes (maintenance mode — return `503` from LB) 3. Provision new Postgres instance (same version as backup source) 4. Download latest clean logical backup from S3, verify checksum 5. `pg_restore --clean --if-exists --jobs=4 <backup>` to new instance 6. Run `pg_checksums` to verify data integrity 7. Run smoke tests (count of key tables, sample queries) 8. Switch application `DATABASE_URL` to new instance (via secrets manager update) 9. Restart application 10. Verify application functionality (full smoke test suite) 11. Document incident + post-mortem within 7 days 12. Decommission old instance after 7-day observation

**Estimated time:** 4 hours (RTO target met)

## 4.3 Recovery Procedure — PITR (R1, R2, R4)

---

**Trigger:** Accidental delete/drop, data overwrite

**Steps:** 1. Identify recovery timestamp T (just before incident) 2. Use Supabase/RDS console to restore to timestamp T (creates new instance) 3. New instance provisioned with data up to timestamp T 4. Connect to new instance, extract lost row(s) via SQL 5. Connect to production DB, insert lost row(s) via SQL (within transaction) 6. Verify data integrity 7. Decommission PITR instance 8. Audit log entry documenting recovery

**Estimated time:** 1-2 hours (RTO target met)

## 4.4 Recovery Procedure — Region Failover (R7)

---

**Trigger:** Cloud provider region outage

**Steps:** 1. Confirm region outage (Cloud status page, monitoring alerts) 2. Declare SEV-1 disaster 3. Promote cross-region replica to primary (Supabase/RDS console) 4. Update DNS to point to DR region (Route53 weighted policy — 100% to DR) 5. Update application `DATABASE_URL` to DR

region 6. Restart application in DR region 7. Run smoke tests 8. Notify users (status page, email) 9. When primary region recovers: re-establish replication (DR → primary) 10. Failback to primary during maintenance window 11. Document incident + post-mortem

**Estimated time:** 30 minutes (RTO target met) **RPO:** Up to 1 minute (async replication lag)

## 4.5 Recovery Procedure — Ransomware (R8)

---

**Trigger:** Security team detects ransomware encryption

**Steps:** 1. **ISOLATE** — Disconnect all systems from internet (prevent further encryption) 2. **IDENTIFY** — Determine extent of compromise (which systems, which data) 3. **PRESERVE** — Take disk images of compromised systems (forensics) 4. **ERADICATE** — Wipe compromised systems (cannot trust them) 5. **RESTORE** — Restore from last clean backup (verify backup integrity first) - Verify backup was taken BEFORE attack - Restore to fresh infrastructure (not compromised systems) 6. **VERIFY** — Run security scan on restored systems (no malware in DB) 7. **ROTATE** — Rotate all credentials (assume all compromised) 8. **RESUME** — Bring systems online progressively 9. **NOTIFY** — Notify stakeholders, regulators (FSSAI, CERT-In within 72 hours per DPDP) 10. **POST-MORTEM** — Within 7 days; root cause, action items

**Estimated time:** 24 hours (RTO target met) **RPO:** Up to 24 hours (last clean backup)

## 4.6 Recovery Procedure — Total System Loss (R9)

---

**Trigger:** Catastrophic event destroys primary region + DR region (rare)

**Steps:** 1. Provision new infrastructure via Terraform (different cloud provider if needed) 2. Restore monthly Glacier backup (24-hour retrieval time) 3. Restore file storage from Glacier 4. Restore secrets from secrets manager backup 5. Restore configuration from Terraform state 6. Apply pending migrations 7. Run smoke tests 8. Bring systems online 9. Notify stakeholders

**Estimated time:** 72 hours (RTO target met) **RPO:** Up to 1 month (Glacier monthly backup)

### 4.7 Recovery Procedure — Redis Loss (R11)

---

**Trigger:** Redis outage, cache corruption

**Steps:** 1. Redis is disposable — no data loss concern (cache only) 2. Application continues to function (cache miss → DB query) 3. Provision new Redis instance 4. Update application REDIS\_URL 5. Application reconnects automatically 6. Cache warms over time as requests come in 7. Optional: trigger cache warming job for hot data

**Estimated time:** 15 minutes (RTO target met) **RPO:** N/A (cache is rebuildable)

---

## 5. Business Continuity Plan (BCP)

---

### 5.1 BCP Scope

---

When systems are down, business operations must continue (manually if necessary) until systems recover.

### 5.2 Manual Workarounds

---

Operation	Manual Workaround	Responsible
Goods receipt	Paper GRN form; enter into system when recovered	Warehouse manager
Production order	Paper traveler; system updated post-recovery	Production supervisor
Quality inspection	Paper checklist; system updated post-recovery	Quality inspector

Operation	Manual Workaround	Responsible
<b>Stock issue</b>	Paper issue slip; system updated post-recovery	Warehouse operator
<b>Dispatch</b>	Paper delivery challan; system updated post-recovery	Dispatch supervisor
<b>Customer complaint</b>	Email log; system updated post-recovery	Customer service
<b>NCR creation</b>	Paper NCR form; system updated post-recovery	Quality inspector

### 5.3 BCP Roles

Role	Responsibility
<b>Incident Commander</b>	Coordinates response, makes go/no-go decisions
<b>Operations Lead</b>	Manages manual workarounds
<b>Communications Lead</b>	Notifies stakeholders, customers
<b>Technical Lead</b>	Executes recovery procedures
<b>Safety Lead</b>	Ensures food safety during outage

### 5.4 BCP Communication Plan

Audience	Channel	Frequency
Internal team	Slack #incidents	Real-time updates
Employees		Hourly

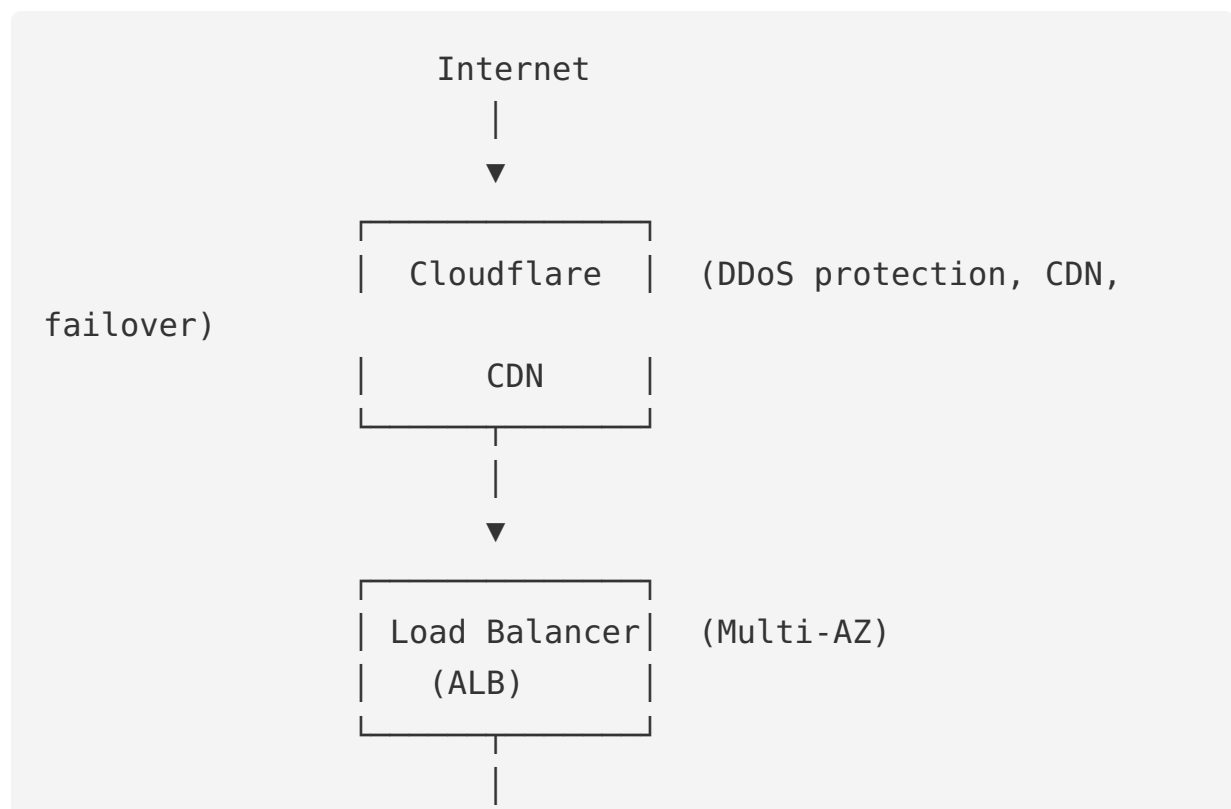
Audience	Channel	Frequency
	Email + Slack #announcements	
Customers	Status page + email	Hourly
Regulators (FSSAI, CERT-In)	Phone + email	Within 72 hours (DPDP)
Public	Status page + blog post	After containment

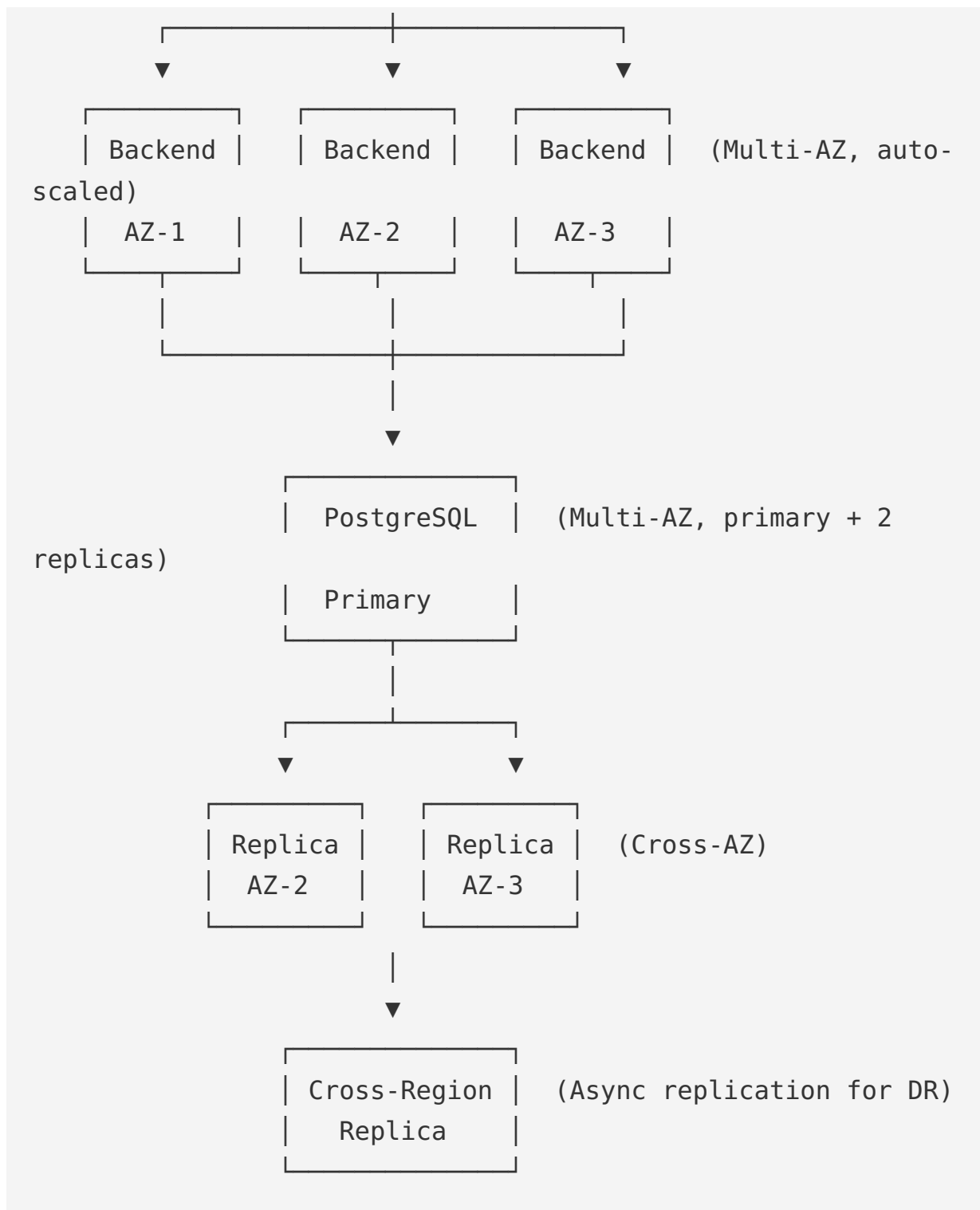
## 5.5 BCP Testing

- **Quarterly:** Tabletop exercise (walk-through of BCP)
- **Semi-annually:** Partial BCP test (one operation manual for 4 hours)
- **Annually:** Full BCP test (all operations manual for 8 hours)

# 6. High Availability

## 6.1 HA Architecture





## 6.2 HA Configuration

- **Multi-AZ:** Primary in AZ-1, synchronous replicas in AZ-2 and AZ-3
- **Auto-failover:** Primary failure → replica promoted automatically (RDS Multi-AZ)
- **Read replicas:** 2 read replicas for read-heavy workloads (dashboards, reports)

- **Cross-region:** Async replica in DR region (RPO ~1 min)
- **Load balancer:** Multi-AZ ALB; health checks route to healthy instances only
- **Backend:** ECS tasks across 3 AZs; min 2 tasks for HA
- **Redis:** Single instance (cache is disposable); future: Redis Cluster for HA

## 6.3 Failure Modes

Failure	Detection	Recovery
Single backend task crash	ALB health check	ALB routes to healthy; ECS starts new task
Single AZ failure	ALB health check	ALB routes to healthy AZs; auto-scaling adds tasks
Primary DB failure	RDS health check	Auto-promote standby replica (<30s)
Redis failure	App health check	App degrades gracefully (cache miss → DB)
Load balancer failure	Cloudflare health check	Cloudflare routes to standby LB
Region failure	External uptime monitor	Manual failover to DR region

## 6.4 No Single Point of Failure

- **Application:** Multiple instances across AZs
- **Database:** Multi-AZ with auto-failover
- **Load balancer:** Managed service (AWS handles HA)
- **DNS:** Route53 with health checks + failover routing
- **CDN:** Cloudflare (globally distributed)
- **Redis:** Single instance today (acceptable — cache); cluster future

---

## 7. Database Recovery

---

### 7.1 Recovery Methods

Method	When	Speed	Granularity
<b>PITR (Point-in-Time Recovery)</b>	Within 14 days of incident	Minutes to hours	Second-level
<b>Logical backup restore</b>	Beyond 14 days	Hours to days	Day-level
<b>Replica promotion</b>	Primary failure	Seconds to minutes	Real-time
<b>Cross-region failover</b>	Region failure	Minutes	Up to 1 min lag
<b>Glacier restore</b>	Catastrophic loss	24+ hours	Month-level

### 7.2 Database Recovery Steps (General)

1. **Identify** — What was lost? When? How?
2. **Isolate** — Stop writes to prevent further loss
3. **Choose method** — PITR vs logical vs replica
4. **Restore** — Execute recovery procedure
5. **Verify** — Run integrity checks + smoke tests
6. **Switch** — Point application to restored DB
7. **Audit** — Log recovery in audit trail
8. **Post-mortem** — Document + prevent recurrence

### 7.3 Database Integrity Verification

After recovery: - `pg_checksums` — physical integrity -  
`pg_stat_user_tables` — row counts match expectations - Foreign key

validation:

`SELECT * FROM child WHERE parent_id NOT IN (SELECT id FROM parent)` - Audit log integrity: re-compute hash chain; verify no tampering - Business rule validation: spot-check critical entities (POs, GRNs, ledger entries)

---

## 8. File Recovery

---

### 8.1 File Storage Recovery

---

**S3 versioning:** Every object has all versions; accidental delete = remove delete marker

**Cross-region replication:** All objects replicated to DR region

**Glacier archive:** 7+ year old files in Glacier; restore takes 1-5 hours (Expedited), 3-5 hours (Standard), 5-12 hours (Bulk)

### 8.2 File Recovery Steps

---

1. Identify lost file (file ID, S3 key)
2. Check S3 versioning — restore previous version (instant)
3. If version unavailable — check DR region replica (instant)
4. If replica unavailable — restore from Glacier (hours)
5. Verify file integrity (checksum)
6. Update application metadata if file ID changed

### 8.3 File Recovery SLAs

---

Scenario	RTO
Accidental delete (S3 versioning)	<5 min
S3 bucket loss (cross-region replica)	<30 min
Region loss (Glacier restore)	<12 hours
Total file storage loss	<24 hours (Expedited Glacier)

---

# 9. Cyberattack Recovery

## 9.1 Cyberattack Scenarios

Attack	Impact	Recovery
Ransomware	Data encrypted	Restore from clean backup; rotate all credentials
Data breach (exfiltration)	Data stolen	Notify regulators (72h), customers; forensics; patch vulnerability
Account takeover	Unauthorized access	Revoke sessions; reset passwords; audit log review
DDoS	Service unavailable	Cloudflare absorbs; rate limit; auto-scale
SQL injection	Data leaked/modified	Patch vulnerability; audit affected data; restore if modified
Insider threat	Data deleted/leaked	Revoke access; forensics; legal action; restore from backup

## 9.2 Cyberattack Recovery Procedure

1. **Detect** — Monitoring alert, user report, security researcher
2. **Isolate** — Disconnect affected systems (without destroying evidence)
3. **Investigate** — Logs, audit trail, forensics
4. **Contain** — Block attacker (revoke tokens, block IPs, rotate credentials)
5. **Eradicate** — Apply fix, remove malware, close vulnerability

6. **Restore** — From clean backup if data corrupted
7. **Verify** — Security scan of restored systems
8. **Resume** — Bring systems online progressively
9. **Notify** — Regulators (72h), customers, public
10. **Post-mortem** — Root cause, action items, lessons learned

## 9.3 Cyberattack Recovery Tools

---

- **Forensics:** AWS GuardDuty, Detective
  - **Backup verification:** Checksums, restore-test
  - **Credential rotation:** Secrets manager bulk rotation
  - **Communication:** Pre-drafted templates (breach notification, customer email)
- 

## 10. DR Testing

---

### 10.1 DR Test Schedule

---

Test	Frequency	Scope
<b>Backup restore-test</b>	Daily	Latest logical backup to isolated DB
<b>Full restore-test</b>	Weekly	Full restore to staging + E2E suite
<b>DR failover drill</b>	Monthly	Promote cross-region replica
<b>Tabletop exercise</b>	Quarterly	Walk-through of DR procedures
<b>Partial BCP test</b>	Semi-annual	One operation manual for 4 hours
<b>Full BCP test</b>	Annually	All operations manual for 8 hours

Test	Frequency	Scope
Full disaster simulation	Annually	Region loss, restore from Glacier

## 10.2 DR Test Documentation

Each test produces a report: - Test scenario - Date + participants - Steps executed - Actual RTO achieved - Actual RPO achieved - Issues encountered - Action items - Lessons learned

## 10.3 DR Test Failures

If DR test fails: - Declare SEV-2 incident - Stop deploying to production until DR capability restored - Root-cause analysis within 7 days - Fix verified before resuming deploys

# 11. DR Roles and Responsibilities

## 11.1 DR Team

Role	Responsibility	Backup
Incident Commander	Coordinates response, go/no-go decisions	Operations Lead
Operations Lead	Executes recovery procedures	Senior Engineer
Communications Lead	Notifies stakeholders	Marketing Lead
Technical Lead	Deep technical recovery	Senior Engineer
Safety Lead	Food safety during outage	Quality Manager

Role	Responsibility	Backup
<b>Legal Lead</b>	Regulatory notifications	External counsel
<b>Customer Liaison</b>	Customer communication	Customer Success

## 11.2 DR Contact Tree

On-call Engineer (24/7)

↓

Incident Commander

↓

- ├─ Operations Lead → Engineering team
- ├─ Communications Lead → Status page, Slack, email
- ├─ Technical Lead → Recovery execution
- ├─ Safety Lead → Food safety decisions
- ├─ Legal Lead → Regulatory notification
- └─ Customer Liaison → Customer communication

## 11.3 DR Activation

Any team member can declare a disaster by: - Paging PagerDuty with “DISASTER” keyword - Creating incident in `#incidents` Slack channel - Calling DR hotline (forwarded to on-call)

Once declared, Incident Commander takes over coordination.

# 12. DR Communication

## 12.1 Internal Communication

- **Slack `#incidents`** : Real-time updates during incident
- **Slack `#announcements`** : Hourly employee updates
- **Status page (internal)**: Current incident status

- **Phone bridge:** Open conference call for DR team

## 12.2 External Communication

---

- **Status page (public):** [status.sudhamrit.com](https://status.sudhamrit.com)
- **Customer email:** Within 1 hour of incident declaration
- **Customer update:** Hourly until resolved
- **Resolution email:** Within 1 hour of resolution
- **Post-mortem blog:** Within 7 days (transparency)

## 12.3 Regulatory Notification

---

Regulator	Trigger	Deadline
<b>FSSAI</b>	Food safety data compromised	72 hours
<b>CERT-In</b>	Cybersecurity incident	6 hours (new 2022 mandate)
<b>DPDP (Data Protection)</b>	Personal data breach	72 hours
<b>GST authorities</b>	Invoice data compromised	24 hours
<b>Local police</b>	Cybercrime (ransomware, hack)	24 hours

## 12.4 Communication Templates

---

Pre-drafted templates for: - Customer incident notification - Customer resolution notification - Regulator notification - Public blog post - Internal employee update - Media statement

Templates in [docs/runbooks/communication-templates/](#).

---

## 13. DR Runbooks

---

Each recovery scenario has a detailed runbook:

Runbook	Scenario
RB-01-row-recovery.md	Single row accidental delete
RB-02-table-recovery.md	Single table accidental drop
RB-03-logical-restore.md	Logical corruption / bad migration
RB-04-pitr-restore.md	PITR recovery
RB-05-db-failover.md	DB instance failure
RB-06-az-failover.md	Availability zone failure
RB-07-region-failover.md	Region failure
RB-08-ransomware.md	Ransomware recovery
RB-09-catastrophic.md	Total system loss
RB-10-file-recovery.md	File storage loss
RB-11-redis-loss.md	Redis loss
RB-12-config-drift.md	Configuration drift
RB-13-ddos.md	DDoS attack
RB-14-data-breach.md	Data breach
RB-15-insider-threat.md	Insider threat

Each runbook includes: - Trigger criteria - Step-by-step procedure - Required tools + access - Verification steps - Rollback (if recovery fails) - Communication templates - Post-recovery tasks

---

## 14. DR Budget and Resources

### 14.1 DR Infrastructure Cost

Resource	Purpose	Monthly Cost (Est.)
Cross-region DB replica	DR region standby	\$200-500
Cross-region S3 replication	File storage DR	\$50-100
Glacier storage (7-year archive)	Long-term retention	\$20-50
PagerDuty (DR team)	Incident alerting	\$50-100
Status page (public)	Customer communication	\$30-50
DR test infrastructure	Monthly drill	\$100-200
<b>Total</b>		<b>\$450-1000/month</b>

### 14.2 DR Time Investment

Activity	Frequency	Time per Session	Annual Hours
Tabletop exercise	Quarterly	2 hours	8
DR failover drill	Monthly	1 hour	12
Full restore test	Weekly	30 min	26
Full BCP test	Annually	8 hours	8
Full disaster simulation	Annually	16 hours	16

Activity	Frequency	Time per Session	Annual Hours
Total			70 hours/year

## 15. Open Questions for Approval

#	Decision	Recommendation	Alternatives
Q-DR1	RPO for region failure	1 minute (async replication)	0 (sync — expensive)
Q-DR2	RTO for ransomware	24 hours	8 hours (more strict)
Q-DR3	DR region	AWS ap-south-1 (Mumbai) primary; ap-south-2 (Hyderabad) DR	Different cloud (Azure)
Q-DR4	DR test frequency	Monthly failover, annual full	Quarterly (less strict)
Q-DR5	Glacier retrieval tier	Standard (3-5 hours)	Expedited (1-5 min, higher cost)
Q-DR6	Customer notification SLA	1 hour	30 min (more strict)
Q-DR7	Regulator notification (CERT-In)	6 hours (per 2022 mandate)	N/A (mandated)
Q-DR8	DR infrastructure	Always-on cross-region replica	Pilot light (start on demand)

#	Decision	Recommendation	Alternatives
Q-DR9	BCP manual workaround scope	Critical operations only	All operations
Q-DR10	DR team size	7 roles	Larger (more redundancy)

---

# Approval Block

---

**Approved by:** \_\_\_\_\_ **Date:** \_\_\_\_\_

End of Enterprise Disaster Recovery Document

# 07\_PRODUCTION\_READINESS\_CHECKLIST

---

- [SUOP ERP v1.0 — Final Production Readiness Checklist](#)
  - [How to Use This Checklist](#)
  - [1. Documentation \(100% complete\)](#)
  - [2. Foundation Framework \(Phase 0 Code\)](#)
  - [3. Database](#)
  - [4. API](#)
  - [5. Security](#)
  - [6. DevOps](#)
  - [7. Performance](#)
  - [8. Disaster Recovery](#)
  - [9. Monitoring & Observability](#)
  - [10. Testing](#)
  - [11. First Business Journey \(Procure-to-Receive\)](#)
  - [12. Production Launch Readiness](#)
  - [Final Production Readiness Score](#)
  - [Approval Block](#)

## SUOP ERP v1.0 — Final Production Readiness Checklist

---

Field	Value
Document Version	1.0
Status	DRAFT — Awaiting Final Approval
Date	2026-07-11
Approval Required	Project Owner
Purpose	Single source of truth for v1.0 production readiness

---

# How to Use This Checklist

This checklist defines **100% completion criteria** for SUOP ERP v1.0 production readiness. When every item is checked, the documentation phase is **permanently frozen** and implementation may begin.

The checklist aggregates requirements from all 8 architecture documents: - Phase 0 Architecture - Data Architecture - API Standards - Security Architecture - DevOps Architecture - Performance Standards - Disaster Recovery - (Plus operational items)

**Each checkbox must be verifiable.** “Verifiable” means there is a test, command, or artifact that proves the item is complete.

## 1. Documentation (100% complete)

#	Item	Verified By	Status
1.1	Phase 0 Architecture approved	Stakeholder signature	<input type="checkbox"/>
1.2	Data Architecture approved	Stakeholder signature	<input type="checkbox"/>
1.3	API Standards approved	Stakeholder signature	<input type="checkbox"/>
1.4	Security Architecture approved	Stakeholder signature	<input type="checkbox"/>
1.5	DevOps Architecture approved	Stakeholder signature	<input type="checkbox"/>
1.6			<input type="checkbox"/>

#	Item	Verified By	Status
	Performance Standards approved	Stakeholder signature	
1.7	Disaster Recovery approved	Stakeholder signature	<input type="checkbox"/>
1.8	Schema Review Report produced	Document exists	<input type="checkbox"/>
1.9	Stock Ledger Design produced	Document exists	<input type="checkbox"/>
1.10	All ADRs (Architecture Decision Records) written	ADR folder populated	<input type="checkbox"/>
1.11	Runbooks for top 10 ops tasks written	Runbook folder	<input type="checkbox"/>
1.12	README.md in every package	File inspection	<input type="checkbox"/>

---

## 2. Foundation Framework (Phase 0 Code)

---

### 2.1 Project Structure

---

#	Item	Verified By	Status
2.1.1	Monorepo setup (Turborepo + pnpm)	<code>pnpm install</code> works	<input type="checkbox"/>

#	Item	Verified By	Status
2.1.2	apps/web , apps/backend , apps/mobile folders	Folder structure	<input type="checkbox"/>
2.1.3	packages/ shared , packages/ prisma , packages/ui folders	Folder structure	<input type="checkbox"/>
2.1.4	ESLint config enforces layer boundaries	bun run lint passes	<input type="checkbox"/>
2.1.5	TypeScript strict mode enabled	tsconfig.json inspection	<input type="checkbox"/>
2.1.6	No any types (without justification)	ESLint rule	<input type="checkbox"/>

## 2.2 Configuration Framework

#	Item	Verified By	Status
2.2.1	Env validation (zod schema)	App refuses to start on missing env	<input type="checkbox"/>
2.2.2	Feature flags implemented	features.isEnabled() works	<input type="checkbox"/>
2.2.3	Config sources prioritized (env > DB > defaults)	Tested	<input type="checkbox"/>

#	Item	Verified By	Status
2.2.4	Secrets manager integration	Secrets loaded at boot	<input type="checkbox"/>

## 2.3 Error Framework

#	Item	Verified By	Status
2.3.1	Error hierarchy (BaseError + 10 subclasses)	Code inspection	<input type="checkbox"/>
2.3.2	Error code catalog	packages/ shared export	<input type="checkbox"/>
2.3.3	Error handler middleware	Errors shaped correctly	<input type="checkbox"/>
2.3.4	5xx errors published to Sentry	Sentry integration	<input type="checkbox"/>

## 2.4 Response Envelope

#	Item	Verified By	Status
2.4.1	Success envelope {success, data, meta}	All endpoints return this	<input type="checkbox"/>
2.4.2	Error envelope {success: false, error, meta}	All errors return this	<input type="checkbox"/>

#	Item	Verified By	Status
2.4.3	Pagination meta	List endpoints include meta	<input type="checkbox"/>
2.4.4	Correlation ID in every response	X-Request-Id header	<input type="checkbox"/>

## 2.5 Logging Framework

#	Item	Verified By	Status
2.5.1	Structured JSON logging (Pino)	Log format verified	<input type="checkbox"/>
2.5.2	Request context (correlation ID, user, tenant)	Logs include context	<input type="checkbox"/>
2.5.3	Sensitive field redaction	Logs don't contain secrets	<input type="checkbox"/>
2.5.4	Log levels (fatal, error, warn, info, debug)	Configurable per env	<input type="checkbox"/>

## 2.6 Validation Framework

#	Item	Verified By	Status
2.6.1	Zod schemas for every endpoint	Schema files exist	<input type="checkbox"/>
2.6.2	Validation middleware	Invalid input → 400 with details	<input type="checkbox"/>
2.6.3	Schema registry	Schemas registered at boot	<input type="checkbox"/>

#	Item	Verified By	Status
2.6.4	Reusable schema fragments	Shared schemas in <code>packages/shared</code>	<input type="checkbox"/>
2.6.5	OpenAPI spec generated from schemas	<code>openapi.yaml</code> exists	<input type="checkbox"/>

## 2.7 Audit Framework

#	Item	Verified By	Status
2.7.1	<code>audit_logs</code> table migrated	DB inspection	<input type="checkbox"/>
2.7.2	Audit log table is append-only	RLS policy enforced	<input type="checkbox"/>
2.7.3	Audit middleware records mutations	All POST/PATCH/DELETE audited	<input type="checkbox"/>
2.7.4	Workflow transitions audit logged	Transitions produce audit entry	<input type="checkbox"/>
2.7.5	Audit query API ( <code>/api/v1/audit/...</code> )	Endpoints work	<input type="checkbox"/>
2.7.6	<code>&lt;AuditTrail / &gt;</code> frontend component	Renders audit timeline	<input type="checkbox"/>
2.7.7			<input type="checkbox"/>

#	Item	Verified By	Status
	Hash chain for audit log integrity	Daily hash published externally	

## 2.8 Event Bus

#	Item	Verified By	Status
2.8.1	EventBus interface	Code inspection	<input type="checkbox"/>
2.8.2	Outbox pattern implemented	event_outbox table + publisher job	<input type="checkbox"/>
2.8.3	Event handler base class	BaseEventHandler exists	<input type="checkbox"/>
2.8.4	Event catalog (all event names)	packages/shared export	<input type="checkbox"/>
2.8.5	At-least-once delivery	Idempotency tested	<input type="checkbox"/>
2.8.6	Event replay API	Admin can replay events	<input type="checkbox"/>

## 2.9 Notification Engine

#	Item	Verified By	Status
2.9.1	Notification rule engine	Rules configurable per tenant	<input type="checkbox"/>
2.9.2	In-app channel	Notifications table populated	<input type="checkbox"/>

#	Item	Verified By	Status
2.9.3	Email channel	Test email sent via SMTP	<input type="checkbox"/>
2.9.4	SMS channel (Twilio)	Stub or working	<input type="checkbox"/>
2.9.5	WhatsApp channel	Stub or working	<input type="checkbox"/>
2.9.6	Durable delivery (outbox pattern)	<code>notification_outbox</code> table	<input type="checkbox"/>
2.9.7	Template system (Handlebars)	Templates versioned	<input type="checkbox"/>
2.9.8	Subscription service	User can unsubscribe	<input type="checkbox"/>
2.9.9	Delivery status tracked	Status visible in UI	<input type="checkbox"/>

## 2.10 Workflow Engine

#	Item	Verified By	Status
2.10.1	<code>BaseWorkflow</code> class	Code inspection	<input type="checkbox"/>
2.10.2	State machine with guards	Invalid transition rejected	<input type="checkbox"/>
2.10.3	Pre/post transition hooks	Hooks fire on transition	<input type="checkbox"/>
2.10.4	Transitions publish events	Events emitted	<input type="checkbox"/>

#	Item	Verified By	Status
2.10.5	Transitions audit logged	Audit entries created	<input type="checkbox"/>
2.10.6	Workflow registry	All workflows registered at boot	<input type="checkbox"/>
2.10.7	Workflow inspection API	<code>/api/v1/workflows/:entityType/:id</code> works	<input type="checkbox"/>
2.10.8	<code>&lt;WorkflowActions / &gt;</code> frontend component	UI shows available actions	<input type="checkbox"/>

## 2.11 Permission Framework

#	Item	Verified By	Status
2.11.1	RBAC middleware	Endpoints protected	<input type="checkbox"/>
2.11.2	<code>@requirePermission</code> decorator	Used on routes	<input type="checkbox"/>
2.11.3	Permission registry	All permissions catalogued	<input type="checkbox"/>
2.11.4	3-layer enforcement (route, service, field)	Tested	<input type="checkbox"/>
2.11.5	<code>&lt;PermissionGuard&gt;</code> frontend component	UI hides unauthorized	<input type="checkbox"/>
2.11.6	Default roles seeded	<code>tenants</code> , <code>roles</code> , <code>permissions</code> seeded	<input type="checkbox"/>

## 2.12 Multi-Tenancy

#	Item	Verified By	Status
2.12.1	<code>tenantId</code> on every business table	Schema inspection	<input type="checkbox"/>
2.12.2	Prisma extension auto-scopes queries	Cross-tenant test blocked	<input type="checkbox"/>
2.12.3	PostgreSQL RLS enabled	DB policy check	<input type="checkbox"/>
2.12.4	TenantMiddleware loads context	AsyncLocalStorage populated	<input type="checkbox"/>
2.12.5	Cross-tenant access audited at CRITICAL	Audit log check	<input type="checkbox"/>

## 2.13 Transaction Helper

#	Item	Verified By	Status
2.13.1	<code>transaction()</code> helper	Code inspection	<input type="checkbox"/>
2.13.2	Retry on deadlock/serialization	Retry test passes	<input type="checkbox"/>
2.13.3	Long transaction detection (>5s warning)	Log warning fires	<input type="checkbox"/>
2.13.4	Outbox pattern in transactions		<input type="checkbox"/>

#	Item	Verified By	Status
		Events published after commit	

## 2.14 File Upload Service

#	Item	Verified By	Status
2.14.1	FileService interface	Code inspection	<input type="checkbox"/>
2.14.2	S3 driver	Files uploaded to S3	<input type="checkbox"/>
2.14.3	Local driver (dev)	Files saved locally	<input type="checkbox"/>
2.14.4	MIME type whitelist	Invalid MIME rejected	<input type="checkbox"/>
2.14.5	File size limits	Large file rejected	<input type="checkbox"/>
2.14.6	Signed URLs (15-min TTL)	URLs expire	<input type="checkbox"/>
2.14.7	Virus scan (ClamAV)	Infected file rejected	<input type="checkbox"/>
2.14.8	File metadata in <code>file_uploads</code> table	DB inspection	<input type="checkbox"/>

## 2.15 Background Job Framework

#	Item	Verified By	Status
2.15.1	BullMQ integration	Queue works	<input type="checkbox"/>
2.15.2	Scheduled jobs (cron)	Jobs fire on schedule	<input type="checkbox"/>

#	Item	Verified By	Status
2.15.3	Queued jobs	<code>jobs.enqueue()</code> works	<input type="checkbox"/>
2.15.4	Retry with backoff	Failed job retried	<input type="checkbox"/>
2.15.5	Dead-letter queue	Failed jobs land in DLQ	<input type="checkbox"/>
2.15.6	Job observability API	<code>/api/v1/_internal/jobs</code> works	<input type="checkbox"/>
2.15.7	Distributed lock (single instance per tenant)	Duplicate job prevented	<input type="checkbox"/>

## 2.16 Base Classes

#	Item	Verified By	Status
2.16.1	<code>BaseRepository</code> with tenant scope, soft delete, optimistic lock	Code inspection	<input type="checkbox"/>
2.16.2	<code>BaseService</code> with hooks + validation contract	Code inspection	<input type="checkbox"/>
2.16.3	<code>BaseController</code> with CRUD handlers	Code inspection	<input type="checkbox"/>
2.16.4	<code>BaseWorkflow</code> with state machine	Code inspection	<input type="checkbox"/>
2.16.5	<code>BaseEventHandler</code>	Code inspection	<input type="checkbox"/>

#	Item	Verified By	Status
2.16.6	BaseJob	Code inspection	<input type="checkbox"/>
2.16.7	BaseMiddleware	Code inspection	<input type="checkbox"/>

## 2.17 Frontend Foundation

#	Item	Verified By	Status
2.17.1	Axios client with interceptors	Code inspection	<input type="checkbox"/>
2.17.2	React Query client + query key factory	Code inspection	<input type="checkbox"/>
2.17.3	Auth context + permission guard	Login works, guards hide UI	<input type="checkbox"/>
2.17.4	Shared UI components (DataTable, EntityForm, StatusBadge, etc.)	Component library exists	<input type="checkbox"/>
2.17.5	Loading/error/empty states	All data views have states	<input type="checkbox"/>
2.17.6	Toast notifications	Mutations show toasts	<input type="checkbox"/>

## 3. Database

### 3.1 Database Setup

#	Item	Verified By	Status
3.1.1	PostgreSQL 16+ provisioned (production)	Live DB connection	<input type="checkbox"/>
3.1.2	PostgreSQL 16+ provisioned (staging)	Live DB connection	<input type="checkbox"/>
3.1.3	PostgreSQL 16+ in Docker (development)	docker-compose up works	<input type="checkbox"/>
3.1.4	Connection pooling (PgBouncer/Supervisor)	Pool config verified	<input type="checkbox"/>
3.1.5	Read replica provisioned	Replica accessible	<input type="checkbox"/>
3.1.6	TLS required for all connections	Connection test	<input type="checkbox"/>

### 3.2 Schema

#	Item	Verified By	Status
3.2.1	All Prisma models have reserved columns (id, tenant_id,	Schema inspection	<input type="checkbox"/>

#	Item	Verified By	Status
	version, created_at, etc.)		
3.2.2	UUID v7 primary keys	Schema inspection	<input type="checkbox"/>
3.2.3	Soft delete columns (deleted_at, deleted_by)	Schema inspection	<input type="checkbox"/>
3.2.4	Optimistic locking (version field)	Schema inspection	<input type="checkbox"/>
3.2.5	Foreign key indexes on every FK	DB inspection	<input type="checkbox"/>
3.2.6	Partial indexes for soft-deleted tables	DB inspection	<input type="checkbox"/>
3.2.7	Row-Level Security enabled	RLS policy check	<input type="checkbox"/>
3.2.8	Check constraints for business rules	DB inspection	<input type="checkbox"/>
3.2.9	Audit log table append-only	RLS policy	<input type="checkbox"/>

### 3.3 Migrations

#	Item	Verified By	Status
3.3.1		DB inspection	<input type="checkbox"/>

#	Item	Verified By	Status
	Initial migration applied to production		
3.3.2	Migration files in prisma/migrations/	Folder inspection	<input type="checkbox"/>
3.3.3	Every migration has rollback SQL	Rollback tested	<input type="checkbox"/>
3.3.4	Migrations forward-compatible (no downtime)	Tested in staging	<input type="checkbox"/>
3.3.5	Migrations use CONCURRENTLY for new indexes	SQL inspection	<input type="checkbox"/>
3.3.6	Seed scripts for reference data	Seed runs	<input type="checkbox"/>
3.3.7	Seed scripts for default roles + permissions	Seed runs	<input type="checkbox"/>

## 3.4 Backups

#	Item	Verified By	Status
3.4.1	PITR enabled (14-day retention)	Backup config	<input type="checkbox"/>
3.4.2	Daily logical backup to S3	Backup job runs	<input type="checkbox"/>

#	Item	Verified By	Status
3.4.3	Weekly logical backup (12-month retention)	Backup config	<input type="checkbox"/>
3.4.4	Monthly logical backup to Glacier (7-year retention)	Backup config	<input type="checkbox"/>
3.4.5	Backup encryption (SSE-KMS)	Encryption verified	<input type="checkbox"/>
3.4.6	Daily restore-test job	Job runs, alerts on fail	<input type="checkbox"/>
3.4.7	Weekly full restore to staging	Restore verified	<input type="checkbox"/>
3.4.8	Cross-region backup replication	Replication verified	<input type="checkbox"/>

## 3.5 Observability

#	Item	Verified By	Status
3.5.1	<code>pg_stat_statements</code> enabled	Extension check	<input type="checkbox"/>
3.5.2	Slow query log (>500ms)	Log inspection	<input type="checkbox"/>
3.5.3	Prometheus metrics exporter	Metrics scraped	<input type="checkbox"/>
3.5.4	Database dashboard in Grafana	Dashboard exists	<input type="checkbox"/>
3.5.5	Lock wait alerting	Alert fires on lock	<input type="checkbox"/>

#	Item	Verified By	Status
3.5.6	Connection pool monitoring	Dashboard shows pool	<input type="checkbox"/>

## 4. API

### 4.1 API Conformance

#	Item	Verified By	Status
4.1.1	All endpoints under <code>/api/v1/</code>	Route inspection	<input type="checkbox"/>
4.1.2	REST conventions (verbs, status codes)	API test	<input type="checkbox"/>
4.1.3	Plural kebab-case URLs	Route inspection	<input type="checkbox"/>
4.1.4	UUID v7 IDs in URLs	Route test	<input type="checkbox"/>
4.1.5	Standard response envelope	API test	<input type="checkbox"/>
4.1.6	Error codes namespaced	Error response test	<input type="checkbox"/>
4.1.7	Pagination (offset + cursor)	List endpoint test	<input type="checkbox"/>
4.1.8	Filtering (operator suffixes)	Filter test	<input type="checkbox"/>
4.1.9	Sorting ( <code>-field</code> for desc)	Sort test	<input type="checkbox"/>
4.1.10	Bulk operations (max 100)	Bulk endpoint test	<input type="checkbox"/>
4.1.11	Idempotency key support	Idempotency test	<input type="checkbox"/>
4.1.12	Rate limiting (4 response headers)	Rate limit test	<input type="checkbox"/>
4.1.13	CORS configured	CORS test	<input type="checkbox"/>
4.1.14	ETag + conditional requests	Cache test	<input type="checkbox"/>

#	Item	Verified By	Status
4.1.15	gzip/Brotli compression	Compression test	<input type="checkbox"/>

## 4.2 Authentication

#	Item	Verified By	Status
4.2.1	JWT auth on all endpoints (except login, health)	Unauth request → 401	<input type="checkbox"/>
4.2.2	Argon2id password hashing	Hash inspection	<input type="checkbox"/>
4.2.3	15-min access token TTL	Token inspection	<input type="checkbox"/>
4.2.4	30-day refresh token (httpOnly cookie)	Cookie inspection	<input type="checkbox"/>
4.2.5	Refresh token rotation	Old token rejected	<input type="checkbox"/>
4.2.6	Refresh token replay detection	Replay → session revoked	<input type="checkbox"/>
4.2.7	MFA (TOTP) for admin/finance/recall	MFA flow tested	<input type="checkbox"/>
4.2.8	Account lockout (5 attempts → 30-min)	Lockout test	<input type="checkbox"/>
4.2.9		Reset flow tested	<input type="checkbox"/>

#	Item	Verified By	Status
	Password reset (token-based, 30-min TTL)		
4.2.10	JWT blocklist for compromised tokens	Blocklist test	<input type="checkbox"/>

## 4.3 Authorization

#	Item	Verified By	Status
4.3.1	RBAC middleware on every endpoint	Permission test	<input type="checkbox"/>
4.3.2	Row-level checks (user can only see own data)	Cross-user test	<input type="checkbox"/>
4.3.3	Field-level response shaping	Sensitive field hidden	<input type="checkbox"/>
4.3.4	Cross-tenant access blocked	Cross-tenant test	<input type="checkbox"/>
4.3.5	Sensitive endpoints require MFA re-auth	Re-auth test	<input type="checkbox"/>
4.3.6	All auth failures audit logged	Audit log check	<input type="checkbox"/>

## 4.4 API Documentation

#	Item	Verified By	Status
4.4.1	OpenAPI 3.1 spec generated	openapi.yaml exists	<input type="checkbox"/>
4.4.2	Every endpoint documented	Spec inspection	<input type="checkbox"/>
4.4.3	TypeScript SDK auto-generated	@suop/api-client package	<input type="checkbox"/>
4.4.4	API changelog maintained	CHANGELOG.md updated	<input type="checkbox"/>
4.4.5	Swagger UI (dev only)	/api/docs works in dev	<input type="checkbox"/>

## 5. Security

### 5.1 Transport Security

#	Item	Verified By	Status
5.1.1	TLS 1.3 enforced	SSL Labs A+ grade	<input type="checkbox"/>
5.1.2	HSTS header (2-year, includeSubDomains, preload)	Header inspection	<input type="checkbox"/>
5.1.3	Auto-renewing certificates	Renewal verified	<input type="checkbox"/>
5.1.4	DB connection over TLS	Connection test	<input type="checkbox"/>

#	Item	Verified By	Status
5.1.5	Redis connection over TLS	Connection test	<input type="checkbox"/>

## 5.2 Application Security

#	Item	Verified By	Status
5.2.1	All inputs validated by zod schema	Validation test	<input type="checkbox"/>
5.2.2	SQL injection prevention (Prisma parameterized)	SAST scan	<input type="checkbox"/>
5.2.3	XSS prevention (React auto-encoding, no dangerouslySetInnerHTML)	Code scan	<input type="checkbox"/>
5.2.4	CSRF protection (SameSite cookies)	Cookie test	<input type="checkbox"/>
5.2.5	SSRF prevention (URL allowlist + IP blocking)	Outbound URL test	<input type="checkbox"/>
5.2.6	Content Security Policy header	Header inspection	<input type="checkbox"/>
5.2.7	Security headers (nosniff, frame-deny, etc.)	Header inspection	<input type="checkbox"/>
5.2.8	Sensitive column encryption (pgcrypto)	DB inspection	<input type="checkbox"/>
5.2.9	Password pepper from secrets manager	Config inspection	<input type="checkbox"/>
5.2.10	No secrets in code/git	Secret scan	<input type="checkbox"/>

## 5.3 Infrastructure Security

#	Item	Verified By	Status
5.3.1	VPC with private subnets	Network config	<input type="checkbox"/>
5.3.2	DB in isolated subnet (no internet)	Network config	<input type="checkbox"/>
5.3.3	Security groups least privilege	SG inspection	<input type="checkbox"/>
5.3.4	S3 buckets private (no public access)	Bucket policy	<input type="checkbox"/>
5.3.5	S3 versioning enabled	Bucket config	<input type="checkbox"/>
5.3.6	S3 cross-region replication	Replication check	<input type="checkbox"/>
5.3.7	Container non-root user	Dockerfile inspection	<input type="checkbox"/>
5.3.8	Container read-only filesystem	Container test	<input type="checkbox"/>
5.3.9	Image scanning (Trivy)	CI scan passes	<input type="checkbox"/>
5.3.10	SBOM generated per image	SBOM artifact	<input type="checkbox"/>

## 5.4 Security Testing

#	Item	Verified By	Status
5.4.1	SAST on every PR (SonarQube)	CI passes	<input type="checkbox"/>
5.4.2	Dependency scanning (Snyk + npm audit)	CI passes	<input type="checkbox"/>
5.4.3	Secret scanning (git-secrets)	CI passes	<input type="checkbox"/>
5.4.4	Container image scanning (Trivy)	CI passes	<input type="checkbox"/>
5.4.5	Annual penetration test		<input type="checkbox"/>

#	Item	Verified By	Status
		Pen test report	
5.4.6	OWASP Top 10 coverage verified	Coverage doc	<input type="checkbox"/>

## 5.5 Audit & Compliance

#	Item	Verified By	Status
5.5.1	All mutations audit logged	Audit log inspection	<input type="checkbox"/>
5.5.2	Audit log immutable (RLS)	UPDATE/DELETE rejected	<input type="checkbox"/>
5.5.3	Audit log hash chain	Hash verified	<input type="checkbox"/>
5.5.4	Audit log retention (7 years)	Retention config	<input type="checkbox"/>
5.5.5	FSSAI compliance (2-year records)	Retention verified	<input type="checkbox"/>
5.5.6	GST compliance (6-year records)	Retention verified	<input type="checkbox"/>
5.5.7	DPDP compliance (DSAR workflow)	DSAR test	<input type="checkbox"/>

## 6. DevOps

### 6.1 CI/CD

#	Item	Verified By	Status
6.1.1	CI pipeline (lint, typecheck, test, build)	CI runs on PR	<input type="checkbox"/>

#	Item	Verified By	Status
6.1.2	CD pipeline (staging auto, production manual)	Deploy verified	<input type="checkbox"/>
6.1.3	Blue-green deployment	Zero-downtime deploy	<input type="checkbox"/>
6.1.4	Auto-rollback on smoke test failure	Rollback tested	<input type="checkbox"/>
6.1.5	Migration tested in CI	Migration runs in CI	<input type="checkbox"/>
6.1.6	All gates blocking	PR blocked on fail	<input type="checkbox"/>

## 6.2 Docker

#	Item	Verified By	Status
6.2.1	Dockerfile.backend (multi-stage)	Build works	<input type="checkbox"/>
6.2.2	Dockerfile.web (multi-stage, Next.js standalone)	Build works	<input type="checkbox"/>
6.2.3	docker-compose.yml (full stack)	docker-compose up works	<input type="checkbox"/>
6.2.4	Non-root user in containers	Container test	<input type="checkbox"/>
6.2.5			<input type="checkbox"/>

#	Item	Verified By	Status
	Health check in Dockerfile	Health endpoint works	

## 6.3 Infrastructure as Code

#	Item	Verified By	Status
6.3.1	Terraform for all infrastructure	terraform plan works	<input type="checkbox"/>
6.3.2	State in S3 backend with lock	State config	<input type="checkbox"/>
6.3.3	Drift detection daily	Drift alert works	<input type="checkbox"/>
6.3.4	Staging + production environments	Both deployable	<input type="checkbox"/>

## 6.4 Git Strategy

#	Item	Verified By	Status
6.4.1	Branch protection on main + develop	Branch settings	<input type="checkbox"/>
6.4.2	Signed commits required	Commit signatures	<input type="checkbox"/>
6.4.3	PR review (1 for code, 2 for security)	PR template	<input type="checkbox"/>
6.4.4	Squash & merge	Merge history	<input type="checkbox"/>
6.4.5	Conventional commits enforced	Commit lint	<input type="checkbox"/>

## 7. Performance

### 7.1 API Performance

#	Item	Verified By	Status
7.1.1	p95 latency targets met (per endpoint type)	Load test	<input type="checkbox"/>
7.1.2	No N+1 queries	Query count test	<input type="checkbox"/>
7.1.3	Cursor pagination for >100k row tables	API test	<input type="checkbox"/>
7.1.4	Page size max 200	API test	<input type="checkbox"/>
7.1.5	ETag on GET responses	Header test	<input type="checkbox"/>
7.1.6	gzip/Brotli enabled	Compression test	<input type="checkbox"/>

### 7.2 Database Performance

#	Item	Verified By	Status
7.2.1	Every FK has an index	DB inspection	<input type="checkbox"/>
7.2.2	EXPLAIN ANALYZE shows no seq scans >10k rows	Plan inspection	<input type="checkbox"/>
7.2.3	Slow query log enabled (>500ms)	Log config	<input type="checkbox"/>
7.2.4		Config inspection	<input type="checkbox"/>

#	Item	Verified By	Status
	Auto-vacuum tuned		
7.2.5	Partitioning for large tables (audit, ledger)	Partition check	<input type="checkbox"/>
7.2.6	Cache hit ratio >95%	pg_stat_database	<input type="checkbox"/>

## 7.3 Frontend Performance

#	Item	Verified By	Status
7.3.1	Lighthouse Performance $\geq 90$	Lighthouse run	<input type="checkbox"/>
7.3.2	LCP <2.5s	Lighthouse	<input type="checkbox"/>
7.3.3	FID <100ms	Lighthouse	<input type="checkbox"/>
7.3.4	CLS <0.1	Lighthouse	<input type="checkbox"/>
7.3.5	Initial bundle <300KB gzipped	Bundle analyzer	<input type="checkbox"/>
7.3.6	Per-route bundle <100KB gzipped	Bundle analyzer	<input type="checkbox"/>
7.3.7	Code splitting per route	Bundle inspection	<input type="checkbox"/>
7.3.8	Image optimization (Next.js Image)	Code inspection	<input type="checkbox"/>

## 7.4 Caching

#	Item	Verified By	Status
7.4.1	Redis cache operational	Cache hit/miss metrics	<input type="checkbox"/>
7.4.2	Reference data cached (1h TTL)	Cache test	<input type="checkbox"/>

#	Item	Verified By	Status
7.4.3	Master data cached (5min TTL)	Cache test	<input type="checkbox"/>
7.4.4	Dashboard aggregations cached (5min TTL)	Cache test	<input type="checkbox"/>
7.4.5	Cache hit rate >80%	Redis metrics	<input type="checkbox"/>
7.4.6	Cache invalidation on update	Update → cache miss	<input type="checkbox"/>
7.4.7	CDN configured (Cloudflare)	CDN test	<input type="checkbox"/>
7.4.8	Static assets cached (1 year)	CDN config	<input type="checkbox"/>

## 7.5 Load Testing

#	Item	Verified By	Status
7.5.1	Normal load test (100 users) passes	k6 run	<input type="checkbox"/>
7.5.2	Peak load test (500 users) passes	k6 run	<input type="checkbox"/>
7.5.3	Stress test (find breaking point) documented	Report exists	<input type="checkbox"/>
7.5.4	Spike test passes	k6 run	<input type="checkbox"/>
7.5.5	Soak test (8 hours) passes	k6 run	<input type="checkbox"/>
7.5.6		CI passes	<input type="checkbox"/>

#	Item	Verified By	Status
	Performance regression tests in CI		

## 8. Disaster Recovery

### 8.1 Backup & Recovery

#	Item	Verified By	Status
8.1.1	Daily restore-test job	Job runs, alerts work	<input type="checkbox"/>
8.1.2	Weekly full restore to staging	Restore verified	<input type="checkbox"/>
8.1.3	Monthly DR failover drill	Drill report	<input type="checkbox"/>
8.1.4	Annual full disaster simulation	Simulation report	<input type="checkbox"/>
8.1.5	Glacier restore tested	Restore time documented	<input type="checkbox"/>

### 8.2 High Availability

#	Item	Verified By	Status
8.2.1	Multi-AZ DB (primary + 2 replicas)	DB config	<input type="checkbox"/>
8.2.2	Auto-failover for DB	Failover test	<input type="checkbox"/>
8.2.3	Cross-region async replica	Replication lag <1min	<input type="checkbox"/>
8.2.4	Multi-AZ backend (min 2 tasks)	ECS config	<input type="checkbox"/>

#	Item	Verified By	Status
8.2.5	Multi-AZ load balancer	ALB config	<input type="checkbox"/>
8.2.6	Route53 health check + failover	DNS config	<input type="checkbox"/>

## 8.3 DR Documentation

---

#	Item	Verified By	Status
8.3.1	15 DR runbooks written	Runbook folder	<input type="checkbox"/>
8.3.2	DR contact tree documented	Doc inspection	<input type="checkbox"/>
8.3.3	BCP manual workaround procedures	BCP doc	<input type="checkbox"/>
8.3.4	Communication templates drafted	Template folder	<input type="checkbox"/>
8.3.5	Regulatory notification procedures	Doc inspection	<input type="checkbox"/>

## 8.4 DR Testing

---

#	Item	Verified By	Status
8.4.1	Tabletop exercise (quarterly)	Exercise report	<input type="checkbox"/>
8.4.2	DR failover drill (monthly)	Drill report	<input type="checkbox"/>
8.4.3	Full restore test (weekly)	Test report	<input type="checkbox"/>
8.4.4	Full BCP test (annual)	Test report	<input type="checkbox"/>
8.4.5	Full disaster simulation (annual)	Simulation report	<input type="checkbox"/>

---

## 9. Monitoring & Observability

### 9.1 Metrics

#	Item	Verified By	Status
9.1.1	Prometheus scraping backend	Metrics endpoint	<input type="checkbox"/>
9.1.2	Grafana dashboards (system, DB, business)	Dashboards exist	<input type="checkbox"/>
9.1.3	Database metrics (connections, latency, locks)	Dashboard	<input type="checkbox"/>
9.1.4	Redis metrics (memory, hit rate)	Dashboard	<input type="checkbox"/>
9.1.5	Business metrics (POs/day, GRNs/day, NCRs/day)	Dashboard	<input type="checkbox"/>

### 9.2 Logs

#	Item	Verified By	Status
9.2.1	Structured JSON logs (Pino)	Log format	<input type="checkbox"/>
9.2.2	Log aggregation (Loki or ELK)	Logs searchable	<input type="checkbox"/>

#	Item	Verified By	Status
9.2.3	Sensitive field redaction	Log inspection	<input type="checkbox"/>
9.2.4	Log retention (30 days hot, 90 days warm)	Retention config	<input type="checkbox"/>
9.2.5	Correlation ID in every log line	Log inspection	<input type="checkbox"/>

## 9.3 Tracing

#	Item	Verified By	Status
9.3.1	OpenTelemetry instrumentation	Trace export	<input type="checkbox"/>
9.3.2	Traces exported to Tempo/Jaeger	Trace inspection	<input type="checkbox"/>
9.3.3	100% error traces captured	Trace test	<input type="checkbox"/>
9.3.4	10% normal traces sampled	Sampling config	<input type="checkbox"/>

## 9.4 Alerting

#	Item	Verified By	Status
9.4.1	PagerDuty integration (critical alerts)	Test page	<input type="checkbox"/>
9.4.2	Slack integration (warning alerts)	Test alert	<input type="checkbox"/>
9.4.3	High error rate alert (>5% for 5min)	Alert test	<input type="checkbox"/>

#	Item	Verified By	Status
9.4.4	Service down alert	Alert test	<input type="checkbox"/>
9.4.5	DB down alert	Alert test	<input type="checkbox"/>
9.4.6	High latency alert (p95 >1s for 10min)	Alert test	<input type="checkbox"/>
9.4.7	Disk space alert (>80%)	Alert test	<input type="checkbox"/>
9.4.8	Business alerts (NCR critical, recall, CAPA overdue)	Alert test	<input type="checkbox"/>
9.4.9	Security alerts (failed login spike, permission denied spike)	Alert test	<input type="checkbox"/>
9.4.10	Alert fatigue prevention (grouping, inhibition)	Config verified	<input type="checkbox"/>

## 9.5 Error Tracking

#	Item	Verified By	Status
9.5.1	Sentry integration	Errors captured	<input type="checkbox"/>
9.5.2	Source maps uploaded	Stack traces readable	<input type="checkbox"/>
9.5.3	Release tracking	Releases tagged in Sentry	<input type="checkbox"/>

#	Item	Verified By	Status
9.5.4	Frontend errors captured	Error test	<input type="checkbox"/>

## 9.6 Uptime Monitoring

#	Item	Verified By	Status
9.6.1	External uptime monitor (Pingdom/ UptimeRobot)	Monitor configured	<input type="checkbox"/>
9.6.2	Synthetic transaction monitoring (Checkly)	Synthetics run	<input type="checkbox"/>
9.6.3	Status page (public)	<code>status.sudhamrit.com</code> live	<input type="checkbox"/>
9.6.4	Real User Monitoring (Sentry Performance)	RUM data flowing	<input type="checkbox"/>

# 10. Testing

## 10.1 Test Infrastructure

#	Item	Verified By	Status
10.1.1	Vitest configured	<code>bun run test:unit</code> works	<input type="checkbox"/>
10.1.2	Testcontainers configured	Integration tests run	<input type="checkbox"/>
10.1.3	Playwright configured	E2E tests run	<input type="checkbox"/>

#	Item	Verified By	Status
10.1.4	k6 configured	Load tests run	<input type="checkbox"/>
10.1.5	Test database isolated	Tests don't pollute	<input type="checkbox"/>

## 10.2 Test Coverage

#	Item	Verified By	Status
10.2.1	Foundation code coverage $\geq 90\%$	Coverage report	<input type="checkbox"/>
10.2.2	Business code coverage $\geq 70\%$	Coverage report	<input type="checkbox"/>
10.2.3	Coverage gate in CI	CI fails below threshold	<input type="checkbox"/>
10.2.4	Mutation tests (Stryker) on foundation	Mutation score $\geq 80\%$	<input type="checkbox"/>

## 10.3 Test Types

#	Item	Verified By	Status
10.3.1	Unit tests for every service method	Test count	<input type="checkbox"/>
10.3.2	Unit tests for every workflow transition	Test count	<input type="checkbox"/>
10.3.3	Integration tests for every endpoint	Test count	<input type="checkbox"/>
10.3.4	E2E tests for top 20 user journeys	Test count	<input type="checkbox"/>
10.3.5	Performance regression tests	Nightly run	<input type="checkbox"/>

#	Item	Verified By	Status
10.3.6	Contract tests (frontend ↔ backend)	CI passes	<input type="checkbox"/>
10.3.7	Smoke tests post-deploy	Deploy triggers tests	<input type="checkbox"/>

## 11. First Business Journey (Procure-to-Receive)

### 11.1 Organization Module

#	Item	Verified By	Status
11.1.1	Schema migrated	DB inspection	<input type="checkbox"/>
11.1.2	Repository implemented	Code inspection	<input type="checkbox"/>
11.1.3	Service with business rules	Code inspection	<input type="checkbox"/>
11.1.4	Workflow (Plant lifecycle)	State machine test	<input type="checkbox"/>
11.1.5	API endpoints (CRUD + workflow)	API tests	<input type="checkbox"/>
11.1.6	Frontend wired to API	UI test	<input type="checkbox"/>
11.1.7	RBAC enforced	Permission test	<input type="checkbox"/>
11.1.8	Audit logging	Audit log check	<input type="checkbox"/>
11.1.9	Notifications (plant activation)	Notification test	<input type="checkbox"/>
11.1.10	Unit + integration tests	Tests pass	<input type="checkbox"/>

## 11.2 Auth & RBAC Module

#	Item	Verified By	Status
11.2.1	Schema migrated	DB inspection	<input type="checkbox"/>
11.2.2	Real auth (Argon2id + JWT)	Auth test	<input type="checkbox"/>
11.2.3	MFA (TOTP)	MFA test	<input type="checkbox"/>
11.2.4	Refresh token rotation	Rotation test	<input type="checkbox"/>
11.2.5	User lifecycle state machine	Workflow test	<input type="checkbox"/>
11.2.6	RBAC middleware on every endpoint	Permission test	<input type="checkbox"/>
11.2.7	Frontend login wired to real API	Login test	<input type="checkbox"/>
11.2.8	RBAC UI (roles, permissions) wired	UI test	<input type="checkbox"/>
11.2.9	Audit logging (login, logout, permission changes)	Audit check	<input type="checkbox"/>
11.2.10	Notifications (welcome, password reset)	Notification test	<input type="checkbox"/>
11.2.11		Coverage report	<input type="checkbox"/>



## 12. Production Launch Readiness

### 12.1 Final Pre-Launch

#	Item	Verified By	Status
12.1.1	Production environment provisioned	Terraform applied	<input type="checkbox"/>
12.1.2	DNS configured	DNS lookup works	<input type="checkbox"/>
12.1.3	TLS certificate valid	SSL Labs test	<input type="checkbox"/>
12.1.4	All env vars set in secrets manager	Config check	<input type="checkbox"/>
12.1.5	Database migrated	Migration applied	<input type="checkbox"/>
12.1.6	Seed data loaded (roles, permissions, reference data)	Seed runs	<input type="checkbox"/>
12.1.7	Initial tenant + admin user created	Login test	<input type="checkbox"/>
12.1.8	Smoke tests pass on production	Test run	<input type="checkbox"/>
12.1.9	Monitoring alerts firing	Alert test	<input type="checkbox"/>

#	Item	Verified By	Status
12.1.10	Status page live	Page accessible	<input type="checkbox"/>
12.1.11	Backup jobs scheduled	Jobs run	<input type="checkbox"/>
12.1.12	DR drill completed	Drill report	<input type="checkbox"/>
12.1.13	On-call rotation set up	PagerDuty schedule	<input type="checkbox"/>
12.1.14	Customer support trained	Training complete	<input type="checkbox"/>
12.1.15	Documentation for end users	User guide exists	<input type="checkbox"/>

## 12.2 Sign-off

Role	Name	Signature	Date
Project Owner			
Lead Architect			
Engineering Lead			
Operations Lead			
Security Lead			
QA Lead			

## Final Production Readiness Score

**Calculation:** - Total checkboxes: ~350 - Each checkbox worth ~0.285% -  
 Production Ready % = (checked / total) × 100

**Status:** - ☐ 0% — Documentation phase - ☐ 25% — Foundation framework complete - ☐ 50% — First business journey complete - ☐ 75% — All modules wired - ☐ 90% — Testing + monitoring complete - ☐ 100% — Production ready (sign-off required)

**Target: 100% before any code is “done”**

---

## Approval Block

---

By approving this checklist, the Project Owner authorizes:

- 1. **Permanent freeze** of all architecture documentation
- 2. **No additional documentation** unless explicitly requested
- 3. **All future responses contain production code only**
- 4. Implementation begins with Phase 0 Foundation code

**Approved by:** \_\_\_\_\_ **Date:** \_\_\_\_\_

**Signature:** \_\_\_\_\_

**Once approved, the documentation phase is permanently frozen. Implementation begins.**

---

End of Final Production Readiness Checklist